Advanced Development Guide

24.0.0.500

NEXACRO

TOBESOFT strives to maintain the accuracy of information provided in this document and will consistently modify and supplement it. However, we cannot guarantee accuracy of information in this document. TOBESOFT assumes no responsibility for errors and omissions. No liability is assumed for damages resulting from the use of information contained herein.

Users of this document implicitly agree that they understand the contents of this disclaimer.

Each trademark herein, including the product name of each company, is a solely registered trademark of each company who developed it, and is protected by patent laws and the copyright laws, and should be used for the purpose of reference only.

TOBESOFT Co.,Ltd.
2-5F INTOPS Building, 617 Bongeunsa-Ro, Gangnam-Gu,
Seoul 06083, Korea
www.tobesoft.com

Table of Contents

Copyright and Disclaimer ii

Table of Contents iii

Part I. Advanced Features 1

1. Developing Protocol Adaptor 2

1.1 Creating Protocol Adaptor Module & Using It in the App 2

1.1.1 Creating Protocol Adaptor Module 2

1.1.2 Editing Protocol Adaptor Module 3

1.1.3 Deploying the Protocol Adaptor Module 5

1.1.4 Installing the Module & Registering the Protocol Service 6

1.1.5 Calling Service through the Protocol Adaptor 7

1.2 Additional Configurable Interface Functions 8

1.2.1 Redefinable Interface Functions 9

1.2.2 Functions Added as Methods 10

2. X-PUSH App 12

2.1 Register XPush Object 12

2.2 Display Real-time Messages in Grid 13

2.3 Reliable Messaging 16

2.3.1 Register Topic for Reliable Messaging 16

2.3.2 Receive Messages 18

2.3.3 Unregister Topic 19

3. Developing the Widget App 20

3.1 Creating a Simple Calendar Widget 20

3.2 Creating a Widget that Runs Immediately 23

3.3 Controlling the Widget in the Tray 25

4. Unit Test in the Windows OS NRE 30

4.1 Configuring the WinAppDriver Linkage 30

4.1.1 Installing the WinAppDriver 30

4.1.2 Installing the Development Tools 32
Installing Visual Studio 32
Installing Eclipse 33
4.2 Creating & Running a Unit Test 34
4.2.1 Creating & Deploying the Nexacro App 34
4.2.2 Creating a Unit Test Project 36
Creating a Unit Test Project in Visual Studio 36
Creating the Unit Test Project in Eclipse 41
4.2.3 Running the Unit Test 47
4.3 Checking the AccessibilityId Property Value 48
4.3.1 Checking the Component Property Value 48
4.3.2 Checking the Control & the Item Property Values 48
4.3.3 Checking the Property Value with Inspect 49
4.4 Running the Demo Unit Test by Feature 52
4.5 Supported Interface 53
4.5.1 Session 54
Creating a Session 54
Ending a Session 54
4.5.2 Timeout 54
Setting a Timeout 54
4.5.3 Frame 55
Checking the Mainframe Information 55
4.5.4 Window 55
Changing the Enabled Window 55
Ending the Enabled Window 55
Obtaining the Enabled Window Handle 55
Obtaining the Window Handle Array 56
Checking the Window Size Information 56
Changing the Window Size 56
Checking the Window Position Information 56
Changing the Window Position 57
Maximizing the Window 57
4.5.5 Element 57
Obtaining the Element 57
Obtaining the Element Array 58
Checking the Element id Value 58
Checking the Text Information Displayed on Element 58
Processing the Element Mouse Click Event 59
Deleting the Element Input Value 59
Passing the Keyboard Input Value to Element 60

Checking Whether the Element is Selected ... 61
Checking Whether the Element Can be Used ... 61
Checking Whether the Element Screen is Displayed ... 61
Checking the Element Size Information ... 62
Checking the Element Position Information (Based on the Parent Element) ... 62
Checking the Element Position Information (Based on the Screen) ... 62
Obtaining the Enabled Element ... 62
Comparing 2 Elements ... 63
Checking the UIA Interface Attribute Value of the Element ... 63
4.5.6 ScreenShot ... 63
Obtaining the Window & Element Screenshot ... 63
4.6 Related Link Information ... 64

Part II. How Nexacro Platform Works ... 66

5. Structures of Project and Files ... 67
5.1 Creating Project ... 67
5.1.1 Project Settings File (XPRJ) ... 68
5.1.2 Application Settings File (XADL) ... 69
5.1.3 Type Definition ... 69
5.1.4 Environment ... 71
5.1.5 AppVariables ... 71
5.2 Resources ... 72
5.2.1 initValue ... 72
5.2.2 XTHEME ... 72
5.2.3 ImageResource ... 73
5.2.4 UserFont ... 73
5.3 Others ... 74
5.3.1 Form (XFDL) ... 74
5.3.2 Script (XJS) ... 75
5.3.3 Style Settings File (XCSS) ... 75
6. Application Running Scenario ... 77
6.1 bootstrap ... 77
6.2 Application Loading ... 78
6.3 Form Loading ... 79
7. Nexacro Platform Script Language ... 80
7.1 Scope ... 80
7.1.1 Local ... 80
7.1.2 this ... 81
7.1.3 Global ... 82

7.1.4 Expr ... 85
7.1.5 lookup ... 86
7.2 Event Handler ... 87
7.2.1 Method ... 87
7.2.2 Object Type ... 88
7.3 Setting Property Values ... 89
7.3.1 General Properties ... 89
7.3.2 Dynamic Properties ... 89
7.4 Other Change Items ... 90
7.4.1 Nexacro Methods ... 90
7.4.2 Operational Mode Change ... 91
7.4.3 Restrictions on Object Names ... 91
7.4.4 Restrictions on Variable or Function Names ... 92
7.4.5 Restrictions on External Modules with Window Object ... 93
8. Frame Tree ... 94
8.1 Application ... 94
8.1.1 Script Example Screen ... 95
8.1.2 Accessing Form Object from Application ... 95
8.1.3 Accessing Parent Objects from Form ... 96
8.2 Form ... 96
8.2.1 Script Example Screen ... 98
8.2.2 Components / Objects / Bind ... 99
8.2.3 Container Components ... 99
8.2.4 Container Component Connected to Form ... 100
8.2.5 Parents ... 100

Part III. Supplements ... 101

9. Cache of App ... 102
9.1 Types of Caches ... 102
9.2 Cache Application Method ... 103
10. Dataset XML Format ... 105
10.1 Dataset XML Layout ... 105
10.1.1 XML Declaration ... 106
10.1.2 XML Example ... 106
10.2 Dataset Elements ... 107
10.2.1 Root ... 107
10.2.2 Parameters ... 107
10.2.3 Parameters > Parameter ... 108
ID ... 108

Type ... 109
10.2.4 Dataset ... 109
10.2.5 Dataset > ColumnInfo ... 110
10.2.6 Dataset > ColumnInfo > ConstColumn ... 111
10.2.7 Dataset > ColumnInfo > Column ... 112
10.2.8 Dataset > Rows ... 113
10.2.9 Dataset > Rows > Row ... 113
10.2.10 Dataset > Rows > Row > Col ... 114
10.2.11 Dataset > Rows > Row > OrgRow ... 115
10.2.12 Dataset > Rows > Row > OrgRow > Col ... 116
11. Dataset SSV Format ... 117
11.1 Dataset SSV layout ... 117
11.1.1 Stream Header ... 118
11.1.2 Variables ... 118
Variable ... 118
11.1.3 Datasets ... 119
Dataset ... 119
11.2 Dataset Format Details ... 119
11.2.1 Dataset Header ... 119
11.2.2 Const Column Infos ... 120
11.2.3 Column Infos ... 120
11.2.4 Records ... 121
11.2.5 Null Record ... 121
11.3 References ... 121
11.3.1 Type ... 121
11.3.2 Examples ... 122
When There is 1 Dataset ... 122
When There is 1 Dataset with a Const Column ... 122
When There are Multiple Datasets ... 123
When There is Only a Variable ... 123
When There exist both Variable & Dataset ... 123
12. Dataset JSON Format ... 125
12.1 Dataset JSON layout ... 125
12.1.1 version ... 125
12.1.2 Parameters ... 125
12.1.3 Datasets ... 126
Dataset ... 126
12.2 Dataset format details ... 127
12.2.1 id ... 127
12.2.2 ColumnInfo > ConstColumn ... 127

12.2.3 ColumnInfo > Column 128
12.2.4 Rows 128
12.3 Note 129
12.3.1 type 129
12.3.2 JSON example 130

Part I.

Advanced Features

1.

Developing Protocol Adaptor

Nexacro basically uses HTTP to handle requests and responses. The platform does not convert data itself during the process. Depending on environments, however, you may need to encrypt data while receiving and converting certain types of data.

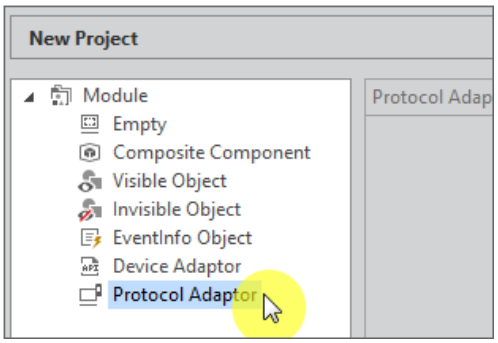
Protocol adapters allow you to encrypt data or change data structure with simple settings. In that way, they help you to achieve your intentions.

1.1 Creating Protocol Adaptor Module & Using It in the App

To use a protocol adapter, you need to write a module, register it in Nexacro Studio and add it as a protocol and service.

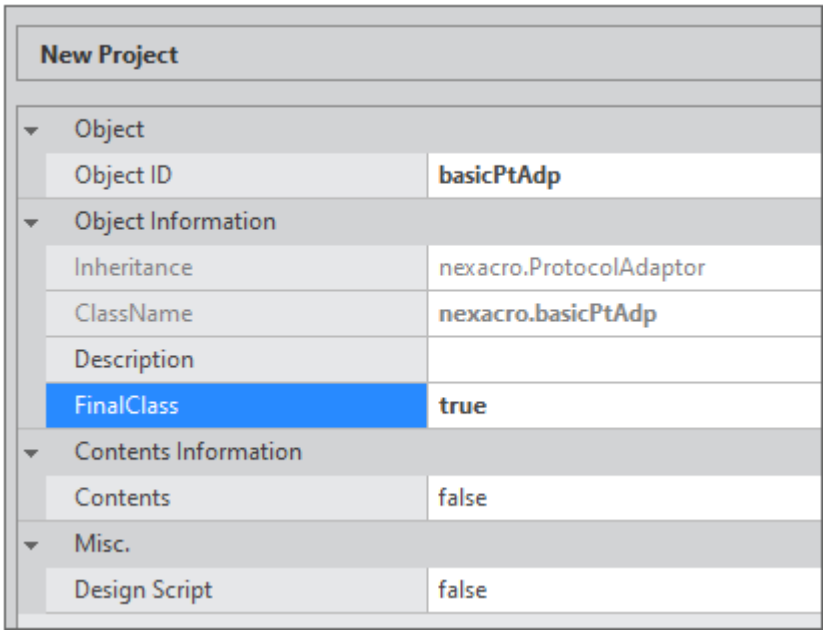
1.1.1 Creating Protocol Adaptor Module

1. Create a new project in the Nexacro Module Developer. Select [File > New > Project] from the menu.
2. When Project Wizard is opened, select the "Protocol Adaptor" from the Module item.

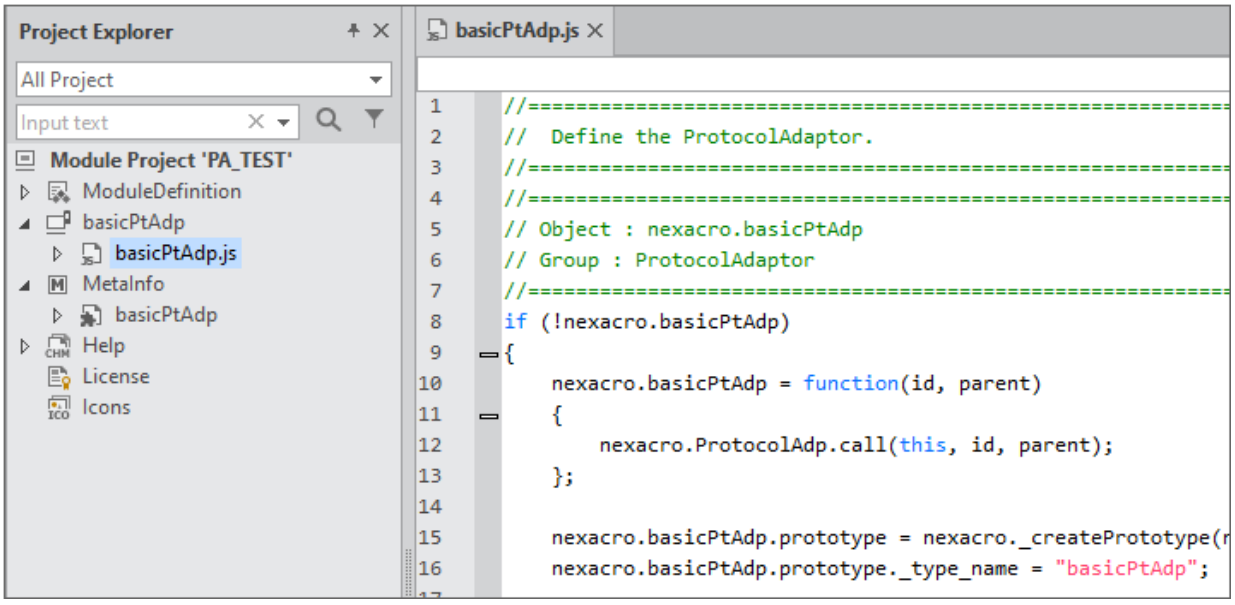


3 Enter the project ID and then click the [Next] button.

4 Enter the Object ID and then change the FinalClass item to true.



5 The project is created and the basicPtAdp.js file is displayed in the edit window.



1.1.2 Editing Protocol Adaptor Module

When you create a new protocol adaptor module, 4 functions are created by default.

Function	Description
initialize	The initialize function is called when the object is created. If an external module is required, then the module is created or initialized within the initialize function.
finalize	The finalize function is called when the object is deleted. If an external module is created or finalization is required, then the created module within the finalize function is deleted and processed so that it does not remain in the memory.
encrypt	The encrypt function is called before communication is executed. The data to be transmitted can be encrypted or converted into the desired form.
decrypt	The decrypt function is called after communication is finished. The data received in response can be decrypted or converted into the desired form.

initialize

```
nexacro.basicPtAdp.prototype.initialize = function()
{
    trace("basicPtAdp initialize");
};
```

In the case of the external plug-in module being used, initialize it using PluginElement as shown below.

```
nexacro.basicPtAdp.prototype.initialize = function()
{
    this._plugin = new nexacro.PluginElement();
    this._plugin.setElementClassId("Microsoft.XMLDOM");
    //this._plugin.setElementClassId("{7E9FDB80-5316-11D4-B02C-00C04F0CD404}");
    this._plugin.create();
};
```

finalize

```
nexacro.basicPtAdp.prototype.finalize = function()
{
    trace("basicPtAdp finalize");
};
```

The external module created through the initialize function is deleted.

```
nexacro.basicPtAdp.prototype.finalize = function()
{
    this._plugin.destroy();
    delete this._plugin;
}
```

encrypt

The data to be transmitted is converted and processed by being put into the return string variable.

```
nexacro.basicPtAdp.prototype.encrypt = function(strUrl, strData)
{
    trace("encrypt url=" + strUrl + ";data=" + strData);
    return strData;
};
```

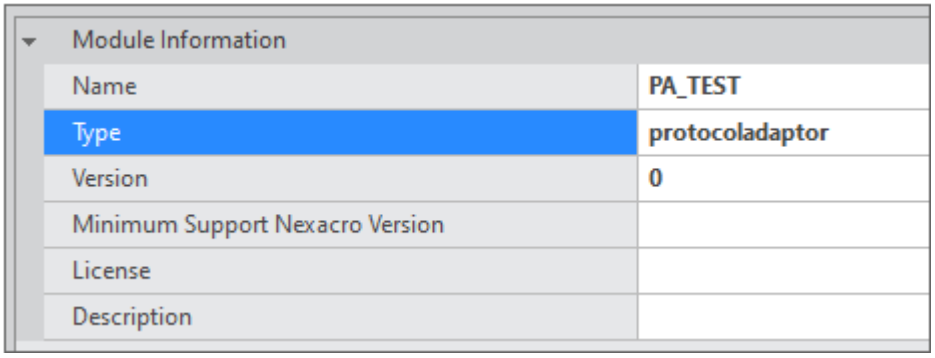
decrypt

The data received in response is converted and passed to the calling app.

```
nexacro.basicPtAdp.prototype.decrypt = function(strUrl, strData)
{
    trace("decrypt url=" + strUrl + ";data=" + strData);
    return strData;
}
```

1.1.3 Deploying the Protocol Adaptor Module

1. Select the Nexacro Module Developer menu [Deploy > Module Package] and then run the Deploy Wizard.
2. Change the Type item to "protocoladaptor" in the Module Information.



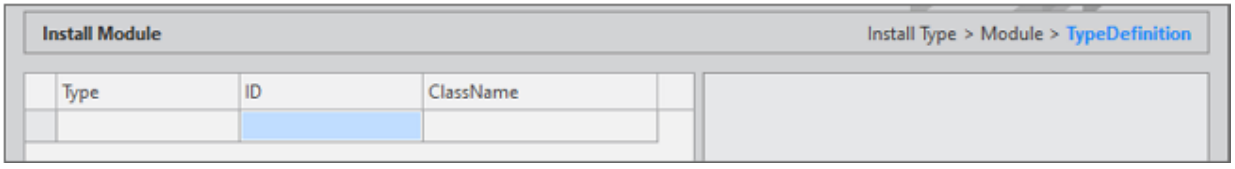
3 Click the [Deploy] button to deploy the module.

1.1.4 Installing the Module & Registering the Protocol Service

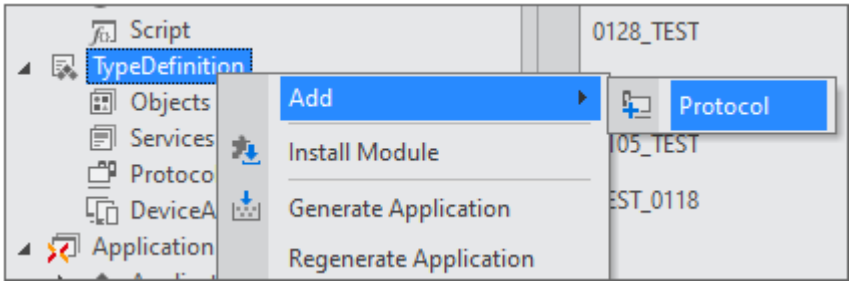
1 Select the Nexacro Studio menu [File > Install Module] and then run the Install Module Wizard.

2 Select and install the module installation file created in the Nexacro Module Developer.

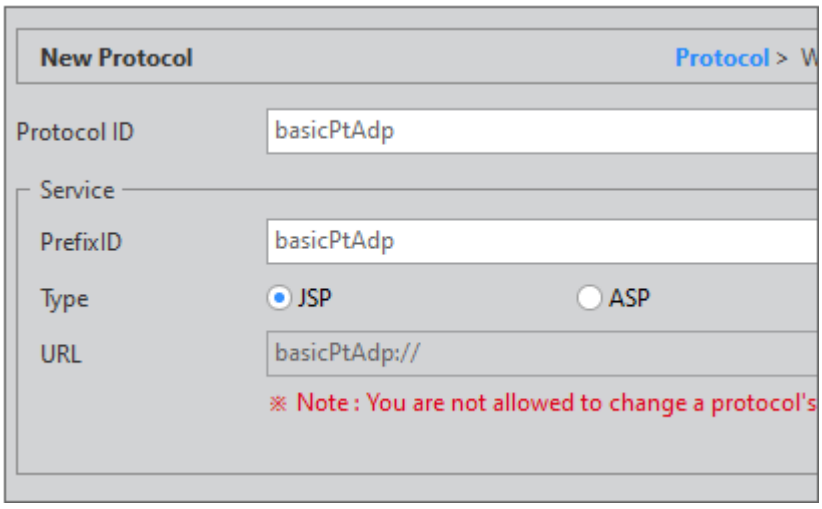
> Since the registration property value among the module meta-info properties is set to the default value ("deny"), the list of modules to be installed is not displayed.



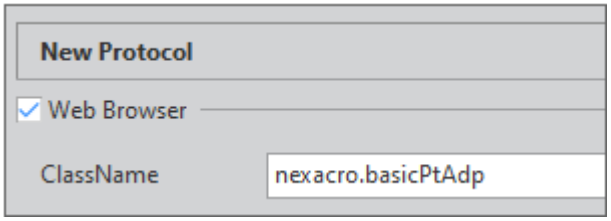
3 Select the TypeDefinition item in the Project Explorer and then select [Add > Protocol] from the context menu.



4 Set the Protocol ID and the Prefix ID in the New Protocol Wizard.



5 Click the [Next] button to check the Web Browser item and then enter the ClassName.



1.1.5 Calling Service through the Protocol Adaptor

When executing the transaction method, if the protocol adaptor URL format is applied to the strURL parameter as shown below, then the communication request is not directly passed to the server, but rather passes through the protocol adaptor.

```
this.Button00_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.transaction("srv00", "basicPtAdp://127.0.0.1:4098/test.xml", "",
"dsOut=Dataset00","value=a","fnCallback");
};

this.fnCallback = function(id, code, message)
{
    trace("Error["+code+"]:"+message);
}
```

When you click the button, you can see that the protocol adaptor operation is processed in the following order.

1. initialize
   There is no initialization in the example, and only the trace method is executed.
2. encrypt
   Check that the value passed to the strArgument parameter is processed.
3. decrypt
   Check the response data for the request.
4. CallbackFunc
   Check that the callback function is operating after normal processing.

```
basicPtAdp initialize

encrypt url=basicPtAdp://127.0.0.1:8080/test.xml;data=<?xml version="1.0"
encoding="UTF-8"?>
<Root xmlns="http://www.nexacroplatform.com/platform/dataset">
    <Parameters>
        <Parameter id="value">b</Parameter>
    </Parameters>
</Root>

decrypt url=basicPtAdp://127.0.0.1:8080/test.xml;data=<?xml version="1.0"
encoding="utf-8"?>
<Root xmlns="http://www.nexacroplatform.com/platform/dataset" ver="5000" >
     <Parameters>
          <Parameter id="ErrorCode" type="int">0</Parameter>
          <Parameter id="ErrorMsg" type="string">SUCC</Parameter>
     </Parameters>
     <Dataset id="customers">
          <ColumnInfo>
               <Column id="id" type="STRING" size="4"/>
...

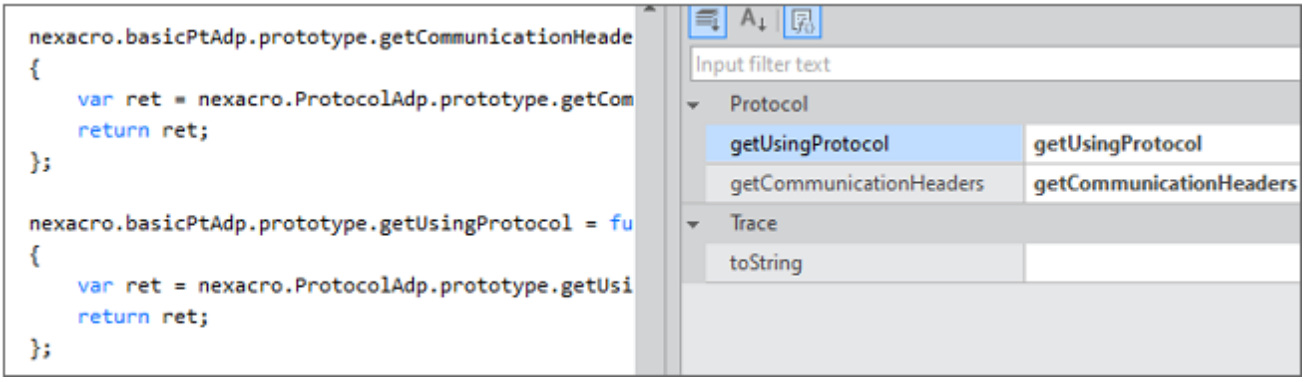
Error[0]:SAUCC
```

1.2 Additional Configurable Interface Functions

The protocol adaptor provides additional configurable interface functions.

1.2.1 Redefinable Interface Functions

The functions provided by the inherited nexacro.ProtocolAdp can be redefined. They can be added and used in the properties window.



Function	Description
getUsingProtocol	It sets the character string to be used as the protocol.
getCommunicationHeaders	It additionally sets the information in the cookie when communicating.

The getCommunicationHeaders function can be set as follows.

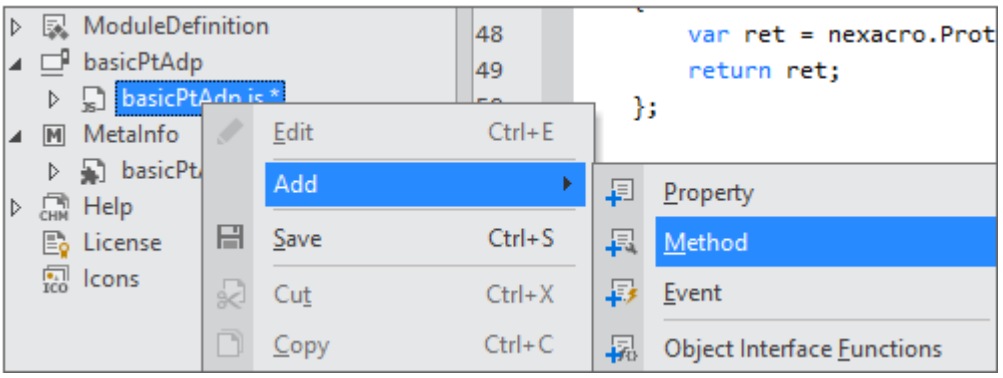
```
nexacro.basicPtAdp.prototype.getCommunicationHeaders = function (strUrl)
{
    var ret = nexacro.ProtocolAdp.prototype.getCommunicationHeaders.call(this,
strUrl);
    ret.push({id:"PHPSESSID", value:"298zf09hf012fh2"});
    ret.push({id:"csrftoken", value:"u32t4o3tb3gg43"});
    return ret;
};
```

You can see that additional cookie information has been set when communicating.



1.2.2 Functions Added as Methods

Select the object in the Project Explorer and then set it by adding the method from the context menu.



Function	Description
getHTTPHeader	It sets the HTTP header value. You can specify id, value of the header value you want.
version	To use the getHTTPHeader function, you must set the version function to a value greater than "1.1".

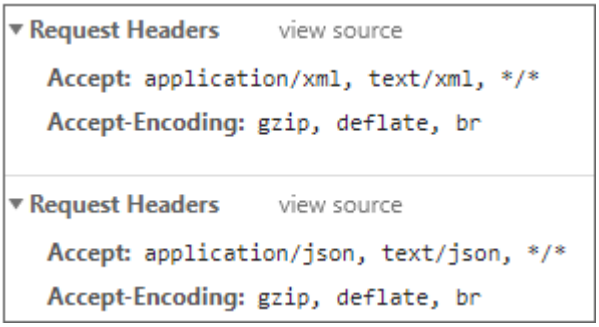
The getHTTPHeader, version function can be set as follows.

```
nexacro.basicPtAdp.prototype.initialize = function()
{
    this._httpheaders = [];
    this._httpheaders.push({ id: "Accept", value: "application/json, text/json, */*"});
};

nexacro.basicPtAdp.prototype.version = function ()
{
    return "1.1";
};

nexacro.basicPtAdp.prototype.getHTTPHeader = function ()
{
    return this._httpheaders;
};
```

You can see that the header Accept information has been changed when communicating.


Request Headers view source
Accept: application/xml, text/xml, */*
Accept-Encoding: gzip, deflate, br
Request Headers view source
Accept: application/json, text/json, */*
Accept-Encoding: gzip, deflate, br

2.

X-PUSH App

X-PUSH is a product that employs server push, a style of Internet-based communication where information or messages are delivered on a real-time basis. Nexacro provides an XPush object, which processes messages sent from an X-PUSH server. The object allows you to request and process messages easily just with simple settings. This chapter shows you how to register an XPush object and develop an application designed to process messages, all using Nexacro Studio.

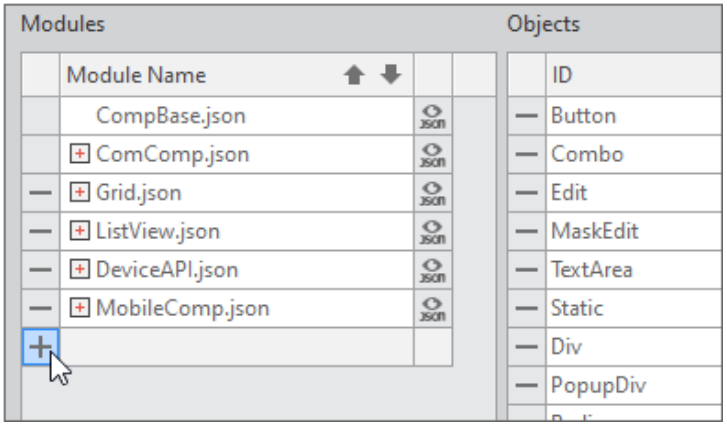
2.1 Register XPush Object

As the XPush is not a default object, you should register the object through Project Explorer of Nexacro Studio to make it available.

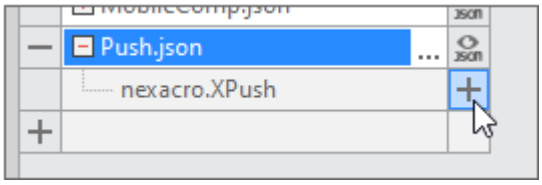
1. Choose the menu [File > New > Project] to create a new project.

2. Choose [TypeDefinition > Objects] in the Project Explorer to open a window that allows you to add a module.

3. Click the + button in Modules on the left side of the window. After the File Explorer is opened, select the *Push.json* file.



4 After finding that *Push.json* has been added to the module list, click the + button on the right side of *nexacro.XPush* to add XPush to the object list.



5 After finding that XPush has been added to the object list. click the OK button to close the window. The project will be reloaded automatically to apply the changes made by the addition of the XPush object.

2.2 Display Real-time Messages in Grid

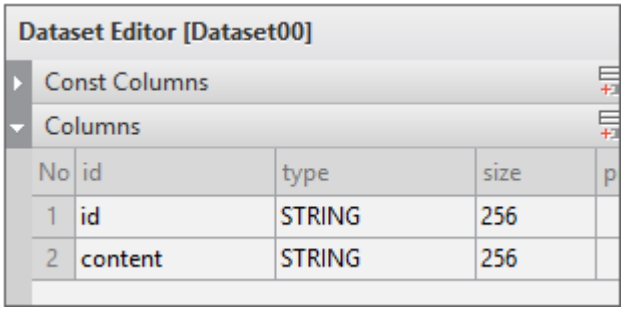
In this section, you will learn about how to add an XPush object, receive messages from an X-PUSH server on a real-time basis and display obtained messages through a Grid.

To receive real-time messages, the X-PUSH server must be in operation with the message provider service also running. An X-PUSH server sends messages while the *run_provider_demo_Push.bat* file is being executed, and the transmission continues until the end of the execution of the file.

1 Choose the menu [File > New > Project] to create a Form.

2 Add a Grid and Dataset to the Form.

3 Set the columns of the Dataset as presented below through the Dataset Editor and bind the Dataset to the Grid.

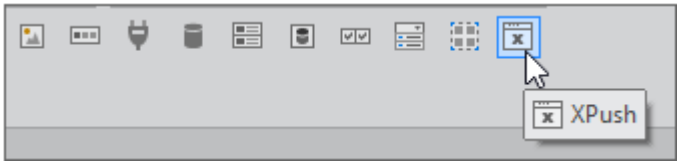




The columns of the Dataset must match the IDs defined by the message layout XML file that is set by the *layouturl* property.

```
...
<message type="OPDT">
    <field id="id" type="string" size="9"/>
    <field id="content" type="string" size="255"/>
</message>
...
```

4 Add an XPush object by dragging and dropping it from the component toolbar onto the Form design pane. Like a Dataset, the object will appear in the Invisible Object pane.



5 After selecting the added XPush object, set the *layouturl* and *iplist* properties in the Properties pane. To set *layouturl*, you can designate a message layout XML file by specifying the file's URL or designating its location in the local computer. To set *iplist*, specify the IP address and port number of the X-PUSH server.



You can select either the TCP connection (Nexacro Browser) or the HTTP connection (web browser) as a value of the *iplist* property.
When you select the TCP connection, you must specify 10081 as a port number.
When you select the HTTP connection, you must specify 10080 as a port number.
If you want to provide both services—NRE and WRE—simultaneously, you must specify the two port numbers together as a value of *iplist*.
`iplist="http://localhost:10080, localhost:10081"`

6 Add the *onload* event of the Form with the below handler, which is designed to connect to the X-PUSH server.

```
this.Form_Work_onload = function(obj:nexacro.Form,e:nexacro.LoadEventInfo)
{
    this.XPush00.connect("user00", "password");
};
```

7 Add the *onsuccess* event of the XPush object with the below handler, which prompts the following actions if the connection is successfully made: the *subscribe* method will be called to receive messages on a real-time basis. Then, the data of the received messages will be added to the Dataset. Eventually, the messages will be displayed through the Grid.

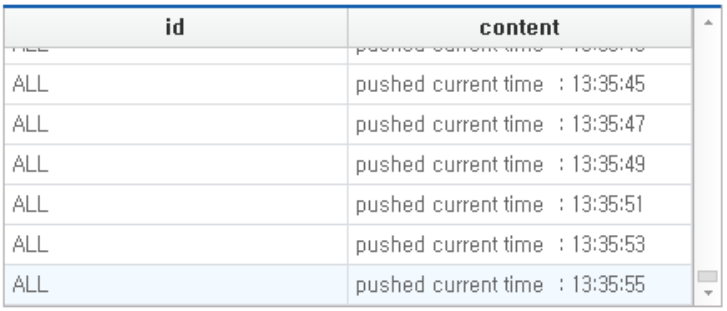
```
this.XPush00_onsuccess = function
(obj:nexacro.XPush,e:nexacro.XPushEventInfo)
{
    if(e.action == nexacro.XPushAction.AUTH) // 0
    {
        this.XPush00.subscribe("OPDT", "ALL", this, this.Dataset00,
"append", "fn_PushCallback");
    }
};

this.fn_PushCallback = function(Row,ChangeColumn,AllColumn,Type,ActionType)
{
    trace(Row,ChangeColumn,AllColumn,Type,ActionType);
}
```

8 Add the *onerror* event of the XPush object with the below handler, which is designed to check the reason for the connect failure.

```
this.XPush00_onerror = function
(obj:nexacro.XPush,e:nexacro.XPushErrorEventInfo)
{
    trace(e.errormsg, e.statuscode);
};
```

9 Upon launching the app, messages will be received after the X-PUSH server is connected within the *onload* event. You can see that the Grid will display messages in real time.



2.3 Reliable Messaging



The process of creating a Form and initializing an XPush object is the same in this section and in Display Real-time Messages in Grid. Therefore, this section begins its explanation with adding the *onload* event handler.



For reliable messaging, the X-PUSH server must be in operation with the message provider service also running. An X-PUSH server sends one message while the *run_provider_demo_Reli.bat* file is being executed.

2.3.1 Register Topic for Reliable Messaging

Reliable messaging is the concept of making certain guarantees about the successful transmission of the messages. Real-time messaging does not check whether users have received the messages after the transmission. This is like radio broadcasting, where messages keep being sent unilaterally and listeners just keep receiving them. However, reliable messaging is similar to an audio message aimed at a certain recipient. Even if users miss messages at the point of transmission, reliable messaging guarantees that they will check the messages later.

For reliable messaging, there should be a predetermined arrangement about who receives what type of messages. You can register such information in a database by calling the *registerTopic* method.

1. Add the *onload* event of the Form with the below handler, which is designed to the X-PUSH server.

```
this.Form_Work_onload = function(obj:nexacro.Form,e:nexacro.LoadEventInfo)
{
    this.XPush00.connect("user00", "password");
}
```

2. Add the *onsuccess* event of the XPush object with the below handler, which prompts the following actions if the connection is successfully made: the *subscribe* method will be called to receive messages on a real-time basis. Then, the data of the received messages will be added to the Dataset. Eventually, the messages will be displayed through the Grid.

```
this.XPush00_onsuccess = function
(obj:nexacro.XPush,e:nexacro.XPushEventInfo)
{
    if(e.action == nexacro.XPushAction.AUTH) // 0
    {
        this.XPush00.subscribe("NOTI", "ALL", this, this.Dataset00,
"append", "fn_PushCallback");
    }
};
```

3 Add a Button component to the Form. Then, add the *onclick* event with the below handler, which is designed to add a topic under the user ID specified when the *connect* method was called. You should register a topic only once.

```
this.Button00_onclick = function
(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.XPush00.registerTopic("NOTI", "ALL");
};
```

4 Add the *onerror* event of the XPush object with the below handler, which is designed to check the reason for a failure to register a topic: e.g. there has been a registered topic already.

```
this.XPush00_onerror = function
(obj:nexacro.XPush,e:nexacro.XPushErrorEventInfo)
{
    trace(e.errormsg, e.statuscode);
};
```

5 Once a topic is registered, reliable messaging begins. You can check the registered topic in the T_TOPIC table of the database.

SELECT * FROM T_TOPIC;

ID	USER_ID	TOPIC_TYPE	TOPIC_ID	REGISTER_DATE	ACTIVE
2	user00	NOTI	ALL	20181011160747	Y

(1 row, 2 ms)

2.3.2 Receive Messages

If messages are sent while the app is not in operation, those messages will be saved to the T_USER_MESSAGE table of the database. if the value of the MESSAGE_STATE column is 0, it means that the messages have not been reached the user.

SELECT * FROM T_USER_MESSAGE;

ID	USER_ID	MESSAGE_ID	MESSAGE_STATE	CHECK_DATE
4	user00	387c[redacted]277523	0	*null*

(1 row, 2 ms)

1. Add a Button component to the Form. Then, add the *onclick* event with the below handler, which is designed to check whether there are unreceived messages under the user ID specified when the connect method was called. If found, the server will send those messages, causing the value of the MESSAGE_STATE column to change to 1.

```
this.Button01_onclick = function
(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.XPush00.requestMessage("NOTI", "ALL");
};
```

2. As the *requestMessage* method does not define a separate callback, it uses the callback defined by the *subscribe* method instead. Once you receive messages through the *requestMessage* method, you can send the response to the reception by calling the *sendResponse* method. After the server receives the response, the value of the MESSAGE_STATE column will turn to 2.

```
this.fn_PushCallback = function
(Row,ChangeColumn,AllColumn,Type,ActionType,MessageId)
{
    trace(Row,ChangeColumn,AllColumn,Type,ActionType,MessageId);
    if(ActionType=="RELI")
        this.XPush00.sendResponse(MessageId);
}
```

2.3.3 Unregister Topic

If you do not need to receive messages anymore, you can call the *unregisterTopic* method to cancel the registered topic.

1. Add a Button component to the Form. Then, add the *onclick* event with the below handler, which is designed to unregister a topic under the user ID specified when the *connect* method was called.

```
this.Button02_onclick = function
(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.XPush00.unregisterTopic("NOTI", "ALL");
};
```

2. Once you unregister the topic, the reliable messaging will also stop. You will see that the value of the ACTIVE column has been changed to "N" in the T_TOPIC table of the database, the table containing the information on registered topics.

SELECT * FROM T_TOPIC;

ID	USER_ID	TOPIC_TYPE	TOPIC_ID	REGISTER_DATE	ACTIVE
2	user00	NOTI	ALL	20181012112527	N

(1 row, 9 ms)

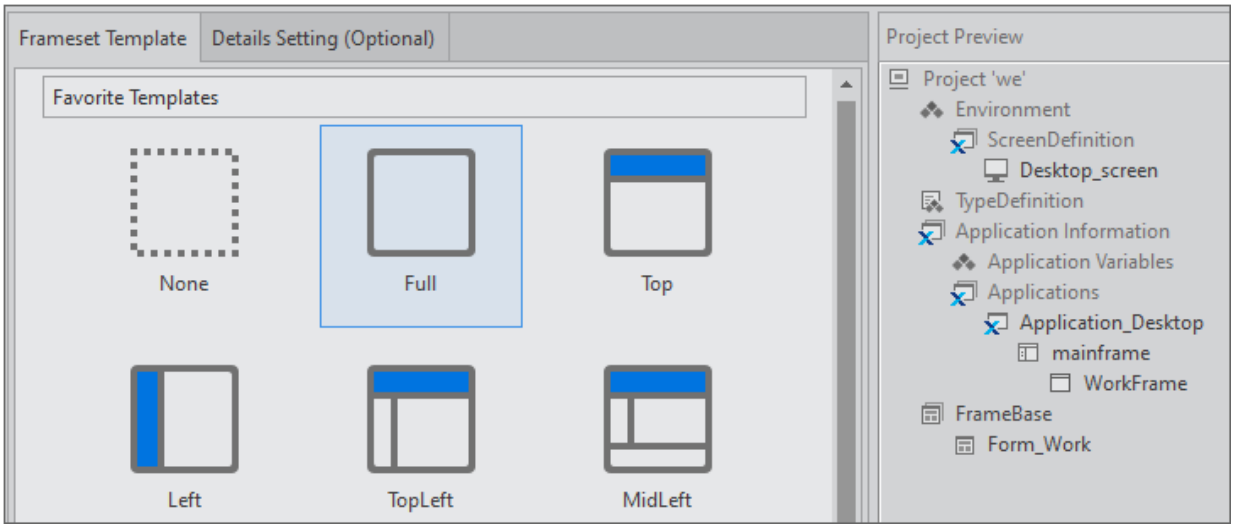
3.

Developing the Widget App

Widgets are small UI tools that are always displayed on the screen for monitoring real-time information, such as clock or CPU status. In the Nexacro, widget UI can be implemented by opening a new window with a transparent layer.

3.1 Creating a Simple Calendar Widget

1. Create a new project. Select the Full type from Frameset Template.



2. Add the Button component to the Form_Work form and create the onclick event function as shown below.

   The layered property is a property that sets the background window to be transparent when creating ChildFrame. By setting this property value to true, you can display a background image of the desired shape on the screen.

```
this.Button00_onclick = function
(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    nexacro.open("NewWidget",
        "FrameBase::WidgetMain1.xfdl",
        this.getOwnerFrame(), "",
        "showontaskbar=false showtitlebar=false showstatusbar=false
topmost=true layered=true border='0px none'",
        0, 0);
};
```

The WidgetMain1.xfdl form is a form that will be displayed in the widget form. As only necessary UI is exposed in the widget form, the taskbar, title bar, and status bar are processed to be invisible in the OpenStyle parameter setting. Then, set the topmost property value to true to ensure that it is not covered by other programs.

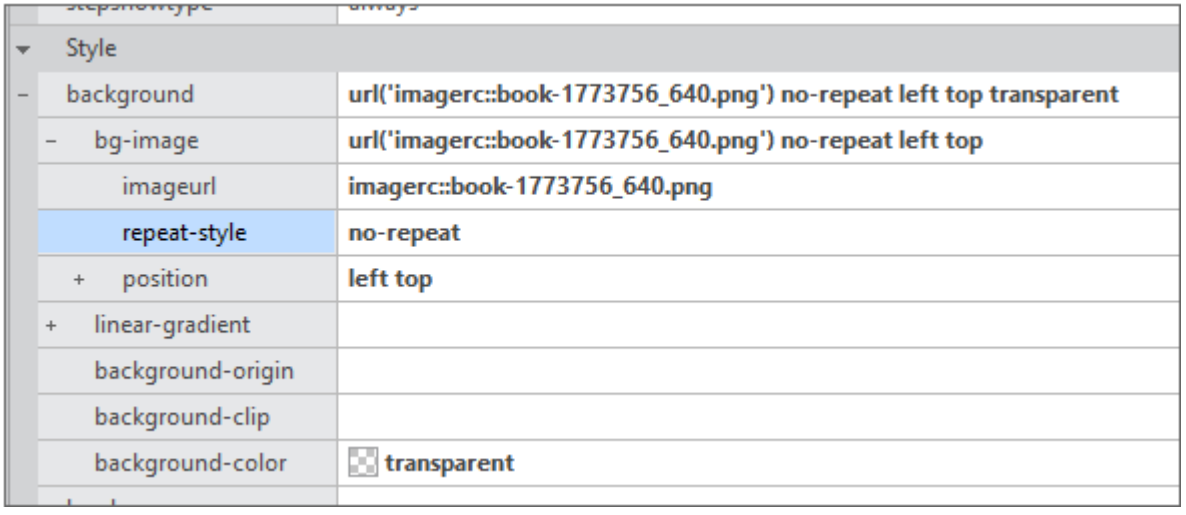
3. Create a new Form object with the name of WidgetMain1.

4. On the ResourceExplorer tab, select ImageResource, and select "Import" from the Context menu.

   Import a transparent image file to be used as the background image. For example, if it is a circular shape, only the circle is displayed with the background area being displayed transparently. In the example, the image below was used.

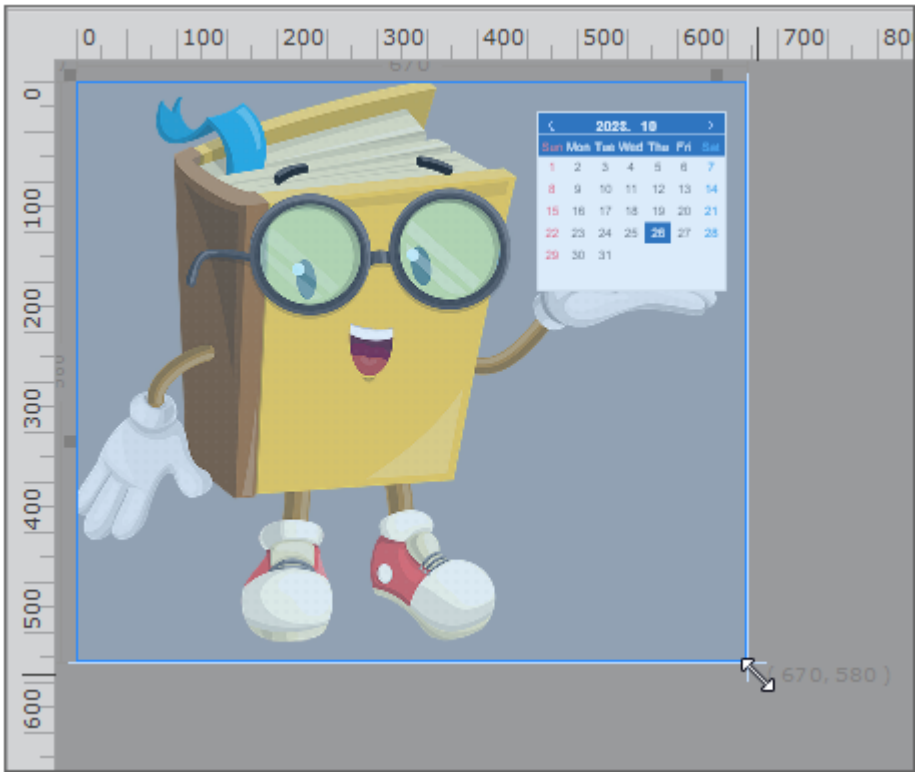
   > `https://pixabay.com/en/book-character-glasses-show-1773756/`

5. Select the transparent image file as the imageurl property value in the background property of the Form object and then set the background-color property value to transparent.



6 Resize the Form object to fit the transparent image file and then place the Calendar component in the desired position.

Set the type in the Calendar component to "monthonly."



7 Select NRE as the execution option from the toolbar and click the Launch icon or enter the shortcut key (Ctrl + F5) to run the app.

> When running Launch, the widget function cannot be used in web browsers other than NRE.

8 When you click the button in the app, the widget UI created with WidgetMain1 is displayed on the screen.

9 You can drag the non-transparent background image area with the mouse to move it to the desired position.

> If you click the transparent background image area, then you cannot change the position of the widget by dragging it with the mouse as the focus moves to the window shown at the back.
> If you click the individual component, then you cannot change the position of the widget by dragging it with the mouse as the individual component needs to operate.



3.2 Creating a Widget that Runs Immediately

In the previous example, the widget UI appeared when the main form was executed and the button was clicked. In this example, we will create a widget that runs immediately (or appears to do so) without user input.

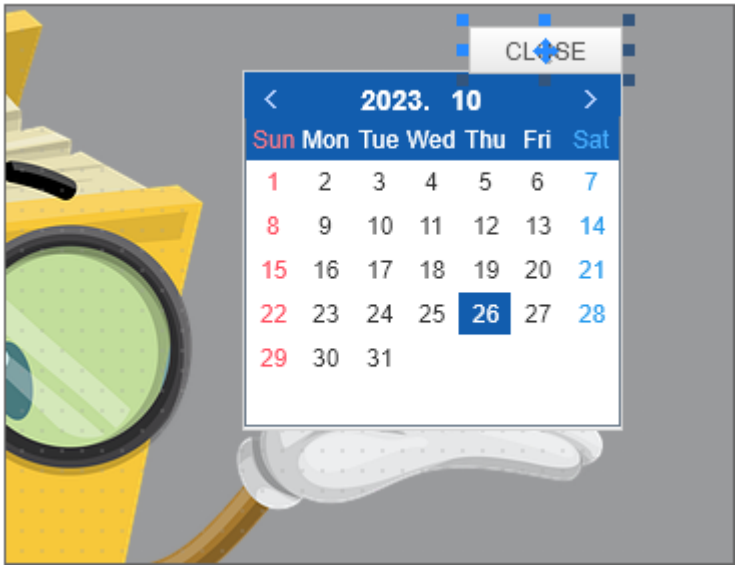
1. Create the oninit event function in the Form_Work form that was used in the previous example as shown below.

```
this.Form_Work_oninit = function(obj:nexacro.Form,e:nexacro.EventInfo)
{
    nexacro.getApplication().mainframe.visible = false;
    nexacro.open("NewWidget",
        "FrameBase::WidgetMain1.xfdl",
        this.getOwnerFrame(), "",
        "showontaskbar=false showtitlebar=false showstatusbar=false
topmost=true layered=true  border='0px none'",
```

```
        0, 0);
};
```

2 Add the Button component for closing the widget to the widget form (WidgetMain1).

In the previous example, when the main form is closed, the widget is closed with it but as the title bar is not displayed on the widget, a separate close button needs to be created.



3 Add the Button component and then create the onclick event function as shown below.

```
this.Button00_onclick = function
(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.close();
};
```

4 Select NRE as the execution option from the toolbar and then click the Launch icon or enter the shortcut key (Ctrl + F5) to run the app.

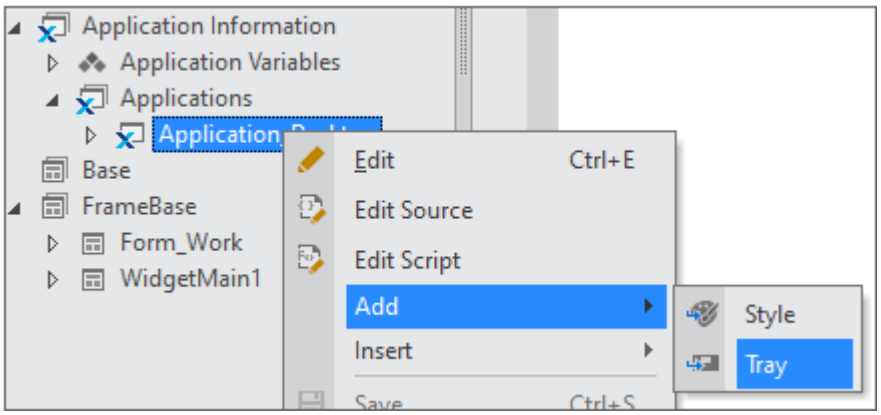
As the app runs, the open method is executed while making the mainframe invisible, resulting in only the widget UI being displayed on the screen. Click the [CLOSE] button to close the widget UI.



3.3 Controlling the Widget in the Tray

Having the widget displayed on the screen may interfere with any other tasks you are performing. In that case, you can hide the widget for a while and then make it visible again. You can register the tray in the app and control widgets from the tray.

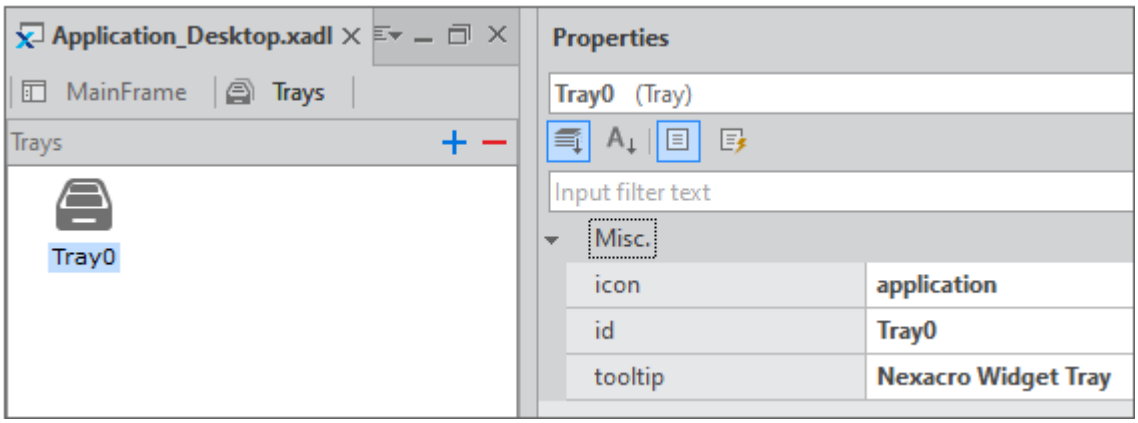
1. Select the App in the Project Explorer and select the [Add > Tray] item from the context menu.



2. Select the tray icon and then set the icon, tooltip property values in the property window.

```
icon: "application"
tooltip: Nexacro Widget Tray
```

For the icon property value, you can use the system icon provided by default or set the URL value. In the example,

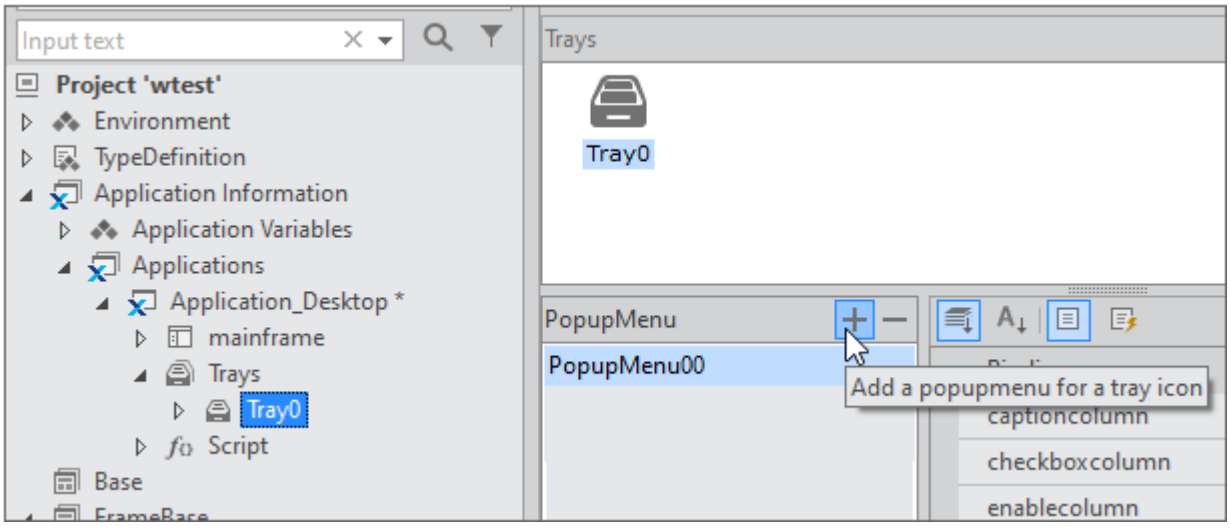


3 Add the onrbuttonup event function as shown below.

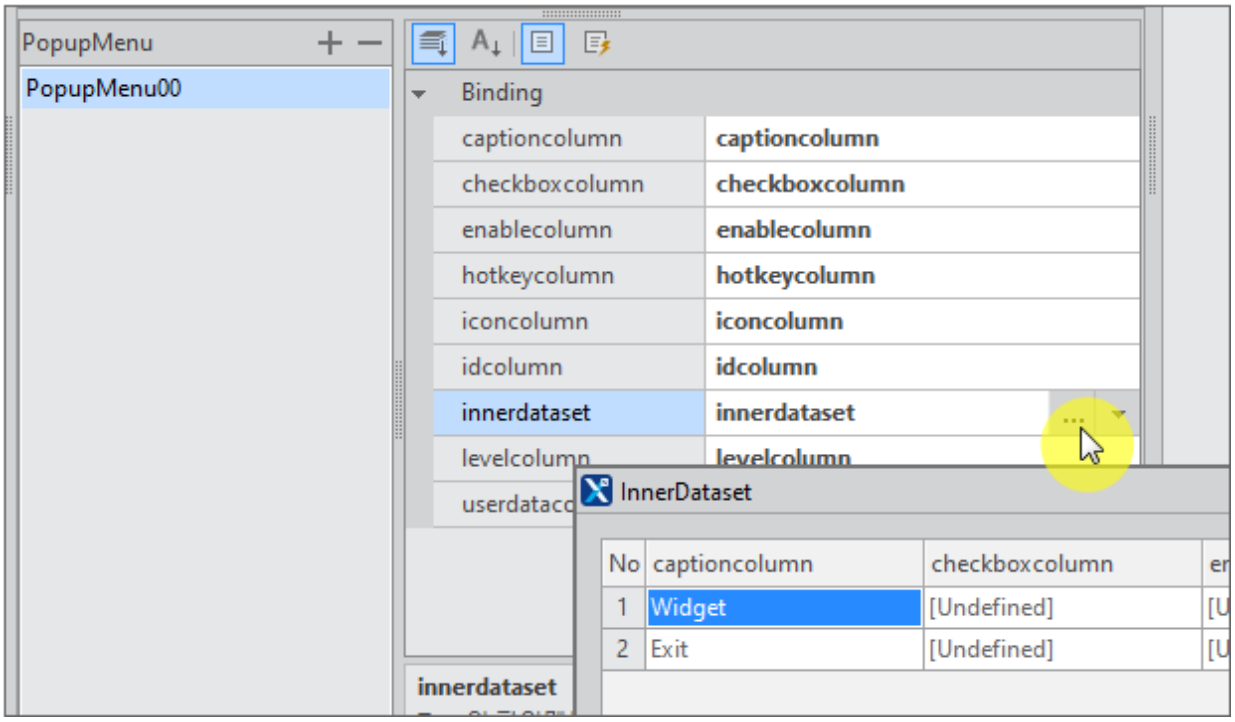
"PopupMenu00" is the PopupMenu component id value to be created in the next step. It makes the menu appear when right-clicking on the tray.

```
this.Tray0_onrbuttonup = function
(obj:nexacro.Tray,e:nexacro.MouseEventInfo)
{
    var pdivWigetMenu = obj.items[obj.findItem("PopupMenu00")];
    pdivWigetMenu.trackPopup()
};
```

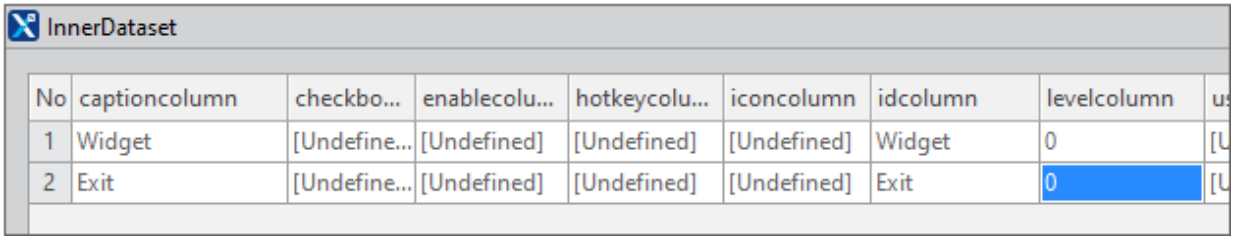
4 Click the add button in the PopupMenu window underneath the tray icon.



5 Select the PopupMenu00 item to set innerdataset in the property window to the right.



Specify the captioncolumn, idcolumn items as "Widget", "Exit" and the levelcolumn item as 0.



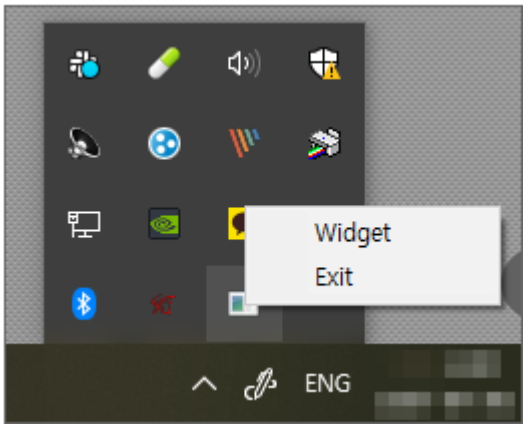
6 By selecting the Event icon from the properties window, you can add event functions. Add the onmenuclick event function as below.

This code specifies the operation when the menu item is selected by right-clicking with the mouse cursor on the tray. When the "Widget" item is selected, the widget is displayed if the widget is closed (invisible), and only the log is recorded if it is already displayed. When the "Exit" item is selected, the app will be closed.

```
this.Tray0_PopupMenu00_onmenuclick = function
(obj:nexacro.TrayPopupMenu,e:nexacro.MenuClickEventInfo)
{
    if(e.id == "Widget")
    {
        var widgetFrame = nexacro.getPopupFrames()["NewWidget"];
        if(widgetFrame.visible)
```

```
        {
            trace(widgetFrame);
        }
        else
        {
            widgetFrame.visible = true;
        }

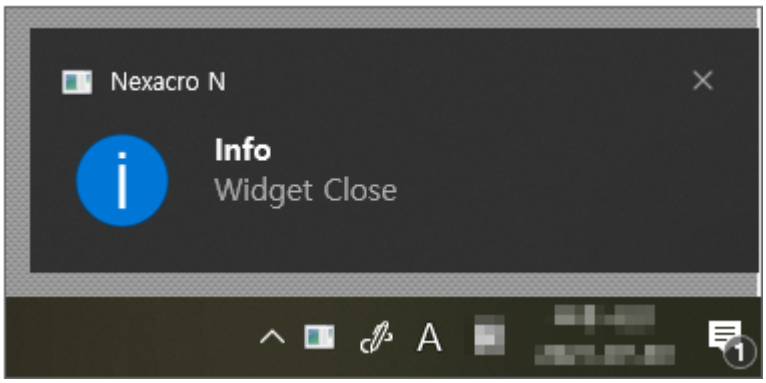
    }
    else if(e.id == "Exit")
    {
        nexacro.getApplication().exit();
    }
};
```



7 Modify the event script that occurs when the button is clicked on the widget form (WidgetMain1).

When selecting the menu in the tray, do not create a new widget screen every time, but rather only modify the visible property value to change whether to display or not. Then, add the code to display a message in the tray area when the widget is closed.

```
this.Button00_onclick = function
(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    //this.close();
    this.parent.visible = false;
    nexacro.getApplication().trays["Tray0"].showBalloonTip("information",
"Info", "Widget Close");
};
```



8 Select NRE as the execution option from the toolbar and then click the Launch icon or enter the shortcut key (Ctrl + F5) to check if the tray is displayed normally in the lower right corner of the window screen, and check that the menu is displayed when right-clicked.

4.

Unit Test in the Windows OS NRE

The WinAppDriver linkage function uses the UIAutomation interface provided by the Nexacro to read or write properties of apps running in NRE and to support methodological execution by specifying parameters. You can write unit tests and test using UI automation testing tools, such as Selenium.

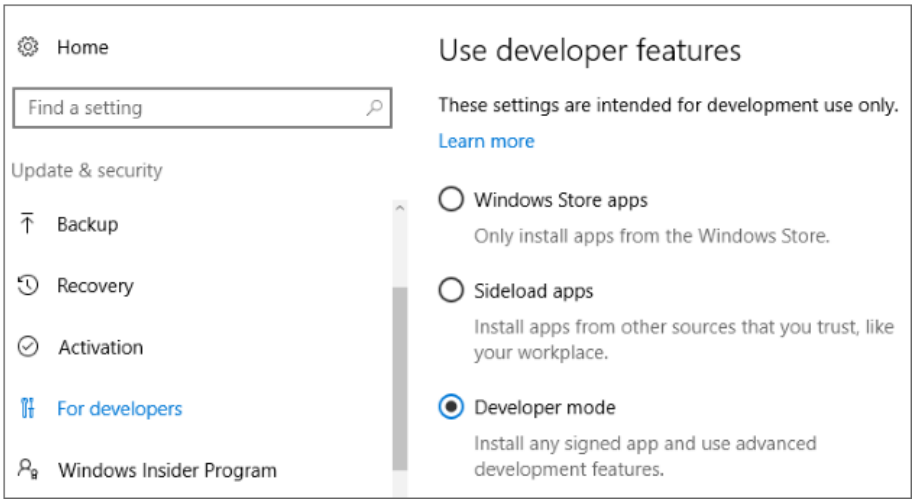
In this chapter, we will look at the steps to install the WinAppDriver, create basic C# and Java projects in Visual Studio and Eclipse, and connect them to the Nexacro apps.

> WinAppDriver operates in Windows 10 or later environments.
> https://github.com/Microsoft/WinAppDriver#system-requirements

4.1 Configuring the WinAppDriver Linkage

4.1.1 Installing the WinAppDriver

1. Enable "Developer Mode" in the Windows operating system [Settings > Use Developer Features].



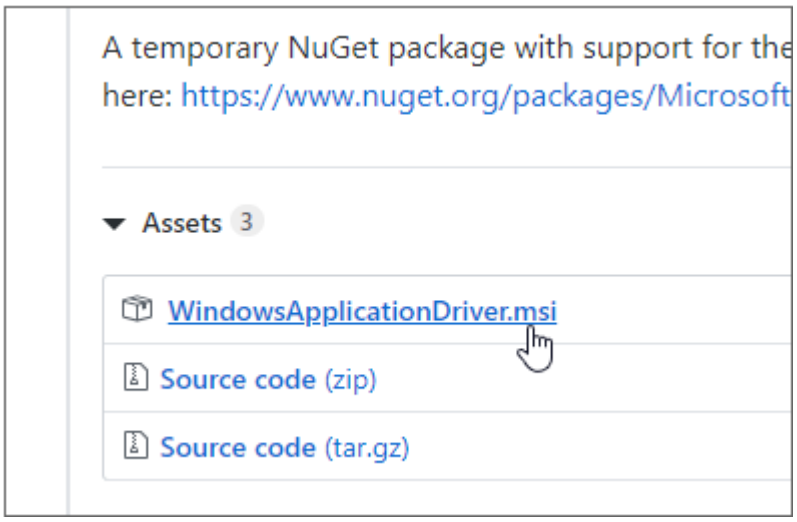


If you are enabling developer mode for the first time, the operating system configuration may take some time. Please allow some time to enable the mode.

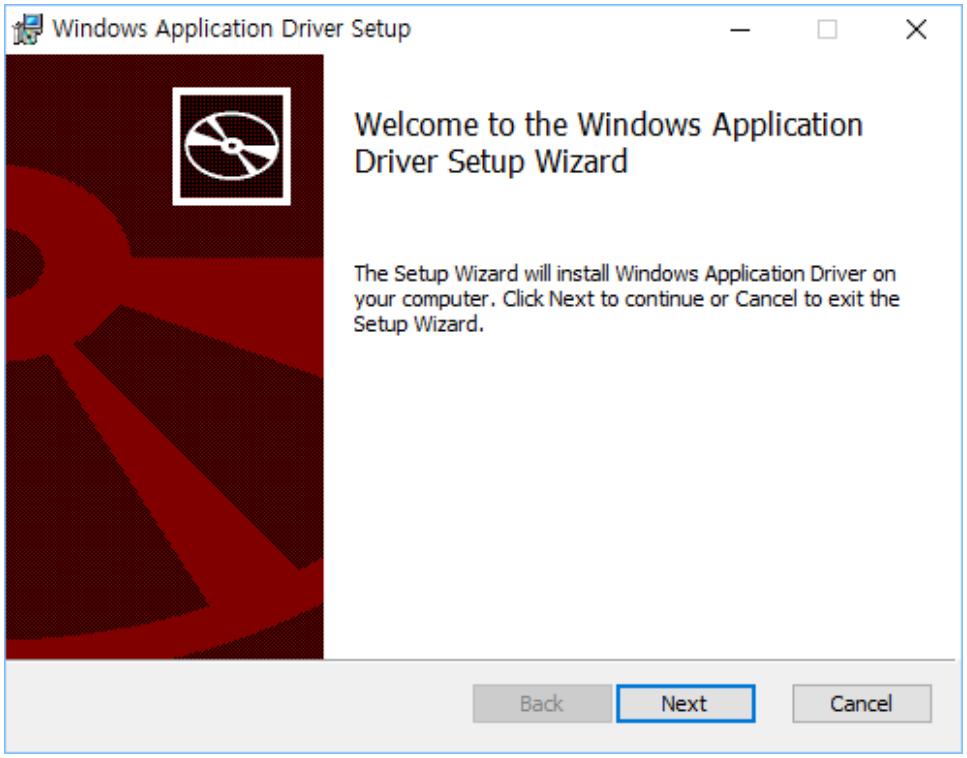
2 Download the WinAppDriver installation file (WindowsApplicationDriver.msi).



https://github.com/Microsoft/WinAppDriver
https://github.com/Microsoft/WinAppDriver/releases

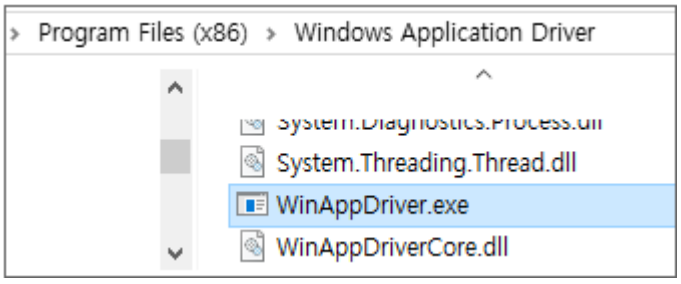


3 Run the downloaded installation file (WindowsApplicationDriver.msi) to install the WinAppDriver.

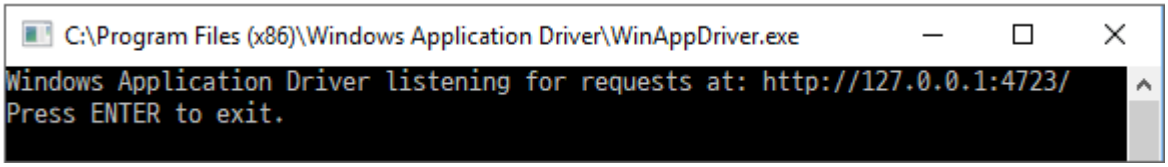


4 Run the WinAppDriver with administrator privileges from the installation path.

> The general installation path is as follows.
> C:\Program Files (x86)\Windows Application Driver\



5 Check if it runs normally.



> If you want to change the ip, port settings, then execute the following.
> > WinAppDriver.exe [ip] [port]

4.1.2 Installing the Development Tools

Unit test projects can be created using the C# or Java languages.

Installing Visual Studio

In the case of using the C# language and Visual Studio not being installed, you can install the development tool provided for free.

1 Download the Visual Studio community installation file (vs_community__xxxx.xxxx.exe).

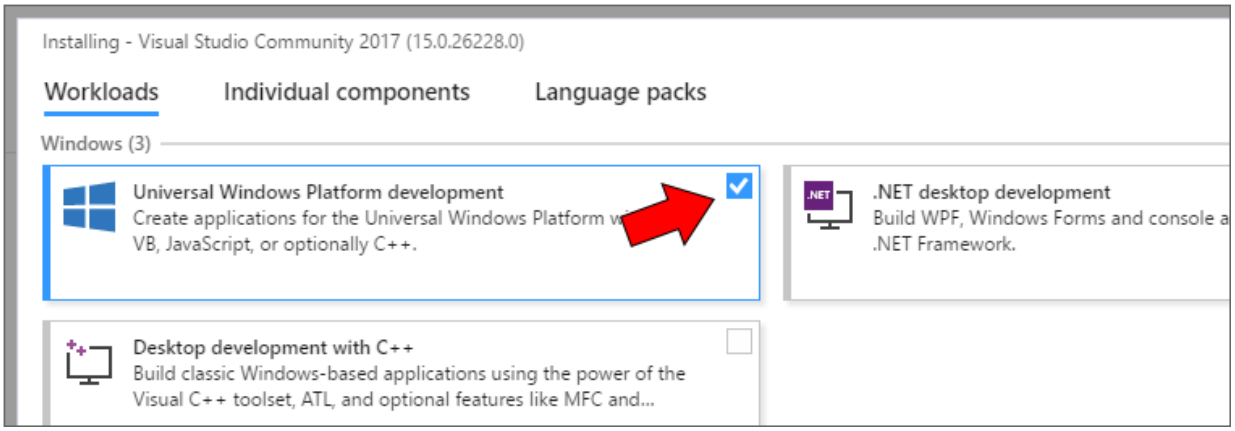
> Visual Studio Community
> https://visualstudio.microsoft.com/vs/community/

2 Run the downloaded installation file (vs_community__xxxx.xxxx.exe) to install Visual Studio.



It may take some time to download the required package during installation. Please allow some time before proceeding with the installation.

3 Select the "Universal Windows Platform development" from the Workloads item and then proceed with the installation.





The development environment can also be added following installation.



A window to log in to your Microsoft account is displayed, but you can still test without logging in.

Installing Eclipse

In the case of using the Java language and Eclipse not being installed, you can install the development tool provided for free.

1 Download the Eclipse installation file (eclipse-inst-win64.exe).



eclipse.org
https://www.eclipse.org/downloads/

2 Run the downloaded installation file (eclipse-inst-win64.exe) to install Eclipse.

3 Select the "Java Developers" item from the development type selection step.



4.2 Creating & Running a Unit Test

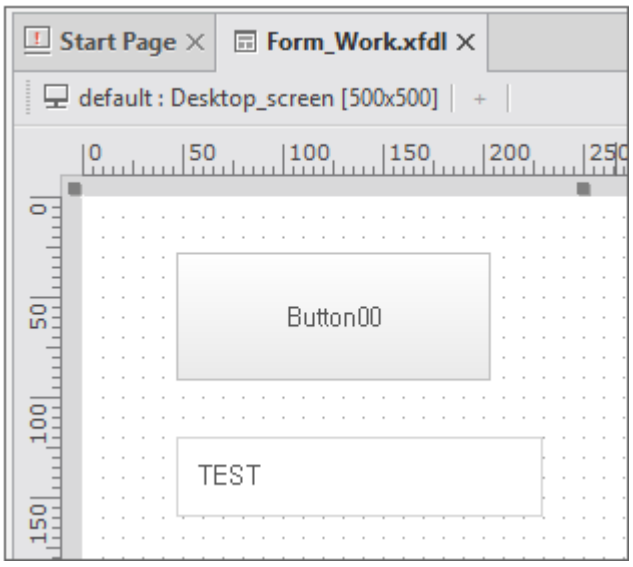
4.2.1 Creating & Deploying the Nexacro App

Create and deploy a simple Nexacro app.

You can skip this step if you are going to use the Nexacro app that has already been installed.

1. Run Nexacro Studio.
2. Create a project and a simple form screen.

   Place only the Button component and the Edit component on the screen.



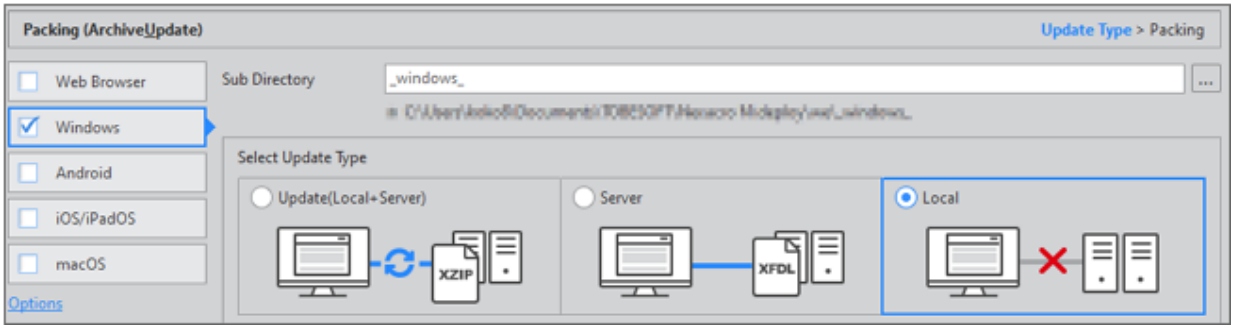
3 Write the onclick event function of the Button component.

When the Button component onclick event occurs, change the value property value in the Edit component.

```
this.Button00_onclick = function
(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.Edit00.value = "BUTTON CLICK";
};
```

4 Select the menu [Deploy > Packing (Archive&Update)] item and then create an archive file to be deployed.

Set the Update Type to "Local" so that it can run without a server connection.



5 Click the [Packing] button to create the archive file and then click the [Build App] button.

6 Click the [Build] button in the Build App window to create the installation file.

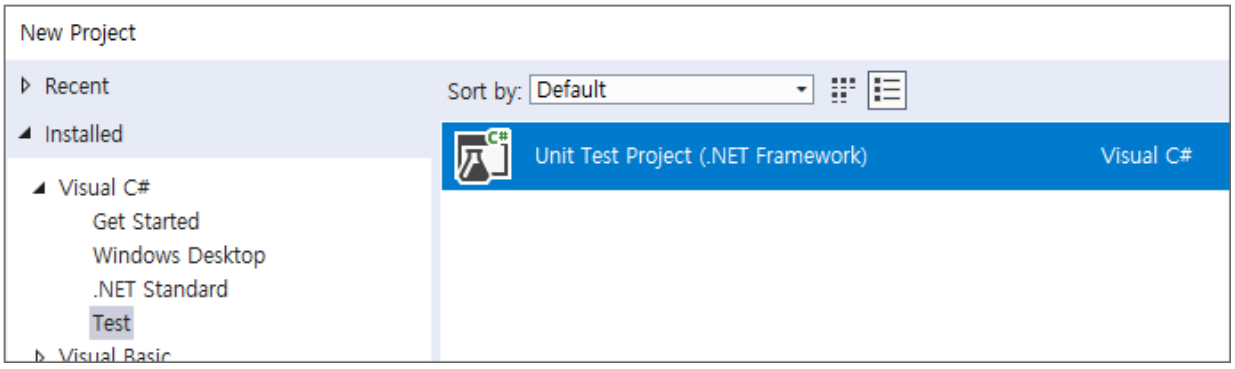
7 Run the created installation file to install the Nexacro app.

4.2.2 Creating a Unit Test Project

Creating a Unit Test Project in Visual Studio

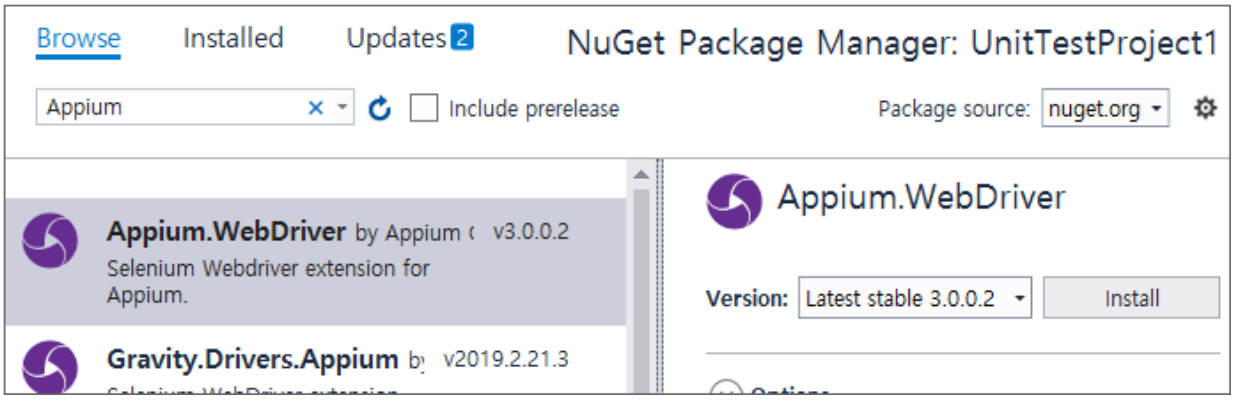
1 Run Visual Studio.

2 Select the "Unit Test Project" from the New Project to create a new project.



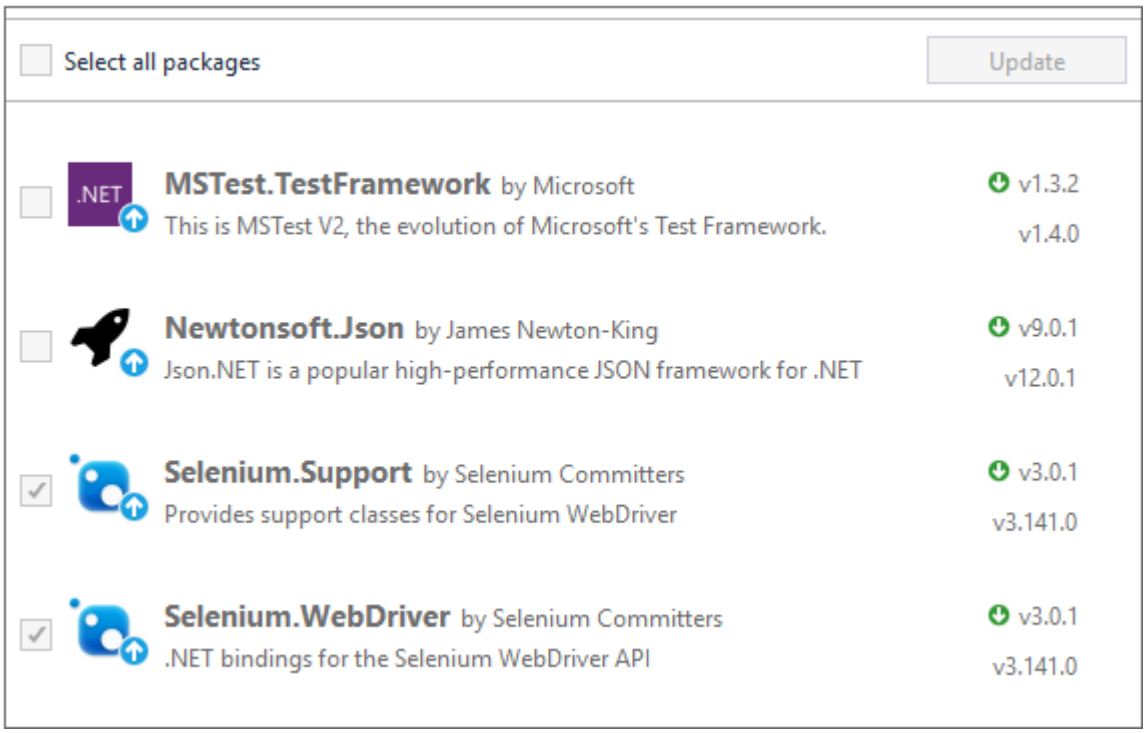
3 Install the Appium WebDriver library.

Select the menu [Project > Manage NuGet Package] and then enter "Appium" in the search window to check the Appium WebDriver library in the search results. Click the [Install] button to install the library.

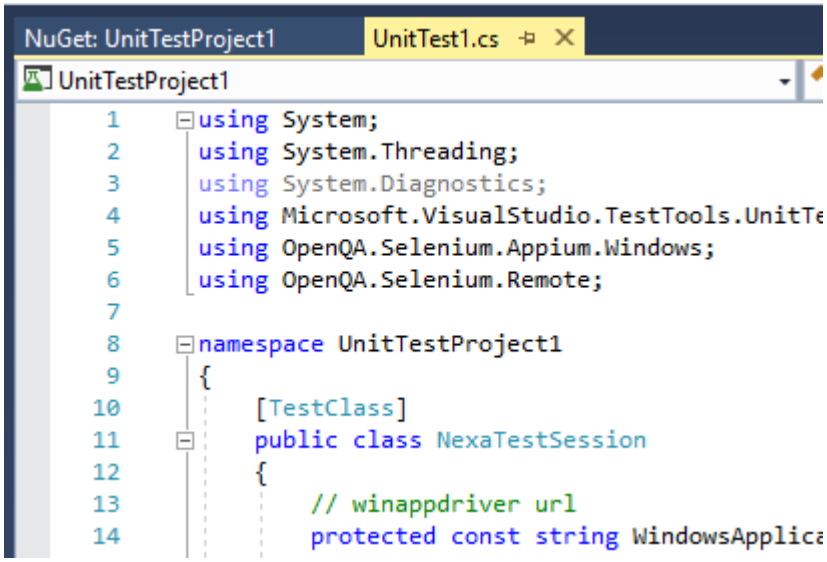


4 Check the Selenium related package in the Updates tab.

> If the latest version is installed, then the code may not work. If you have the latest version, please conduct the minor upgrade to version 3.0.1.
> https://github.com/Microsoft/WinAppDriver/issues/497



5 Modify the created C# source code file as follows.



Modify the settings related to the Nexacro app through the code below. Modify the pathway where the app is installed, the path to the nexacro.exe executable file, and the nexacro.exe execution option to match the actual installation path.

	value
WindowsApplicationDriverUrl	`http://127.0.0.1:4723/`
NexacroAppDr	`C:\\Program Files\\TOBESOFT\\Nexacro N\\21\\app0201`
NexacroAppId	`C:\\Program Files\\TOBESOFT\\Nexacro`

	value
	N\\21\\app0201\\nexacro.exe
NexacroAppAg	-k 'UnitTest' -s 'C:\\Program Files\\TOBESOFT\\Nexacro N\\21\\app0201\\app0201\\start.json'

```
using System;
using System.Threading;
using System.Diagnostics;
using Microsoft.VisualStudio.TestTools.UnitTesting;
using OpenQA.Selenium.Appium.Windows;
using OpenQA.Selenium.Remote;

namespace UnitTestProject1
{
    [TestClass]
    public class NexaTestSession
    {
        // winappdriver url
        protected const string WindowsApplicationDriverUrl =
"http://127.0.0.1:4723/";
        // app path
        private const string NexacroAppDr = "C:\\Program
Files\\TOBESOFT\\Nexacro N\\21\\app0201";
        // nexacro.exe path
        private const string NexacroAppId = "C:\\Program
Files\\TOBESOFT\\Nexacro N\\21\\app0201\\nexacro.exe";
        // nexacro.exe argument
        private const string NexacroAppAg = "-k 'UnitTest' -s 'C:\\Program
Files\\TOBESOFT\\Nexacro N\\21\\app0201\\app0201\\start.json'";
        // search root
        private const string FindNexacro = "Root";
        // OS/Device
        private const string TestOs = "Windows";
        private const string TestDevice = "WindowsPC";

        // Session Object : WindowsDriver
        protected static WindowsDriver<WindowsElement> session;

        // WinAppDriver Session Create
        [ClassInitialize]
        public static void Setup(TestContext context)
```

```
        {
            // Launch a new instance of nexacro application or find nexacro
application
        }

        // WinAppDriver Session Delete
        public static void TearDown()
        {
            // Close the application and delete the session
            if (session != null)
            {
                // Close Session – Close Application
                session.Close();
                // Quit Session
                session.Quit();
                // Delete Session
                session = null;
            }
        }

        [TestMethod]
        public void Test_Session()
        {
            Debug.WriteLine("session:" + session);
            Debug.WriteLine("session.SessionId:" + session.SessionId);
        }
    }
}
```

6 Run the Nexacro app and add code to create the session.

Modify the Setup method in the code above as follows.

```
// WinAppDriver Session Create
[ClassInitialize]
public static void Setup(TestContext context)
{
    // Launch a new instance of nexacro application or find nexacro
application
    if (session == null)
    {
```

```
        // Create a new session to launch nexacro application
        DesiredCapabilities appCapabilities = new DesiredCapabilities();
        TimeSpan time = new TimeSpan(0, 0, 10);

        // Launch Nexacro
        appCapabilities.SetCapability("app", NexacroAppId);
        appCapabilities.SetCapability("appArguments", NexacroAppAg);
        appCapabilities.SetCapability("appWorkingDir", NexacroAppDr);
        appCapabilities.SetCapability("platformName", TestOs);
        appCapabilities.SetCapability("deviceName", TestDevice);

        // Request Remote WinAppDriver Service
    session = new WindowsDriver<WindowsElement>(new
Uri(WindowsApplicationDriverUrl), appCapabilities, time);

        // Set implicit timeout to 1.5 seconds to make element search to
retry every 500 ms for at most three times
        session.Manage().Timeouts().ImplicitlyWait(TimeSpan.FromSeconds(0.5
));
        Thread.Sleep(TimeSpan.FromSeconds(1.0));
    }
}
```

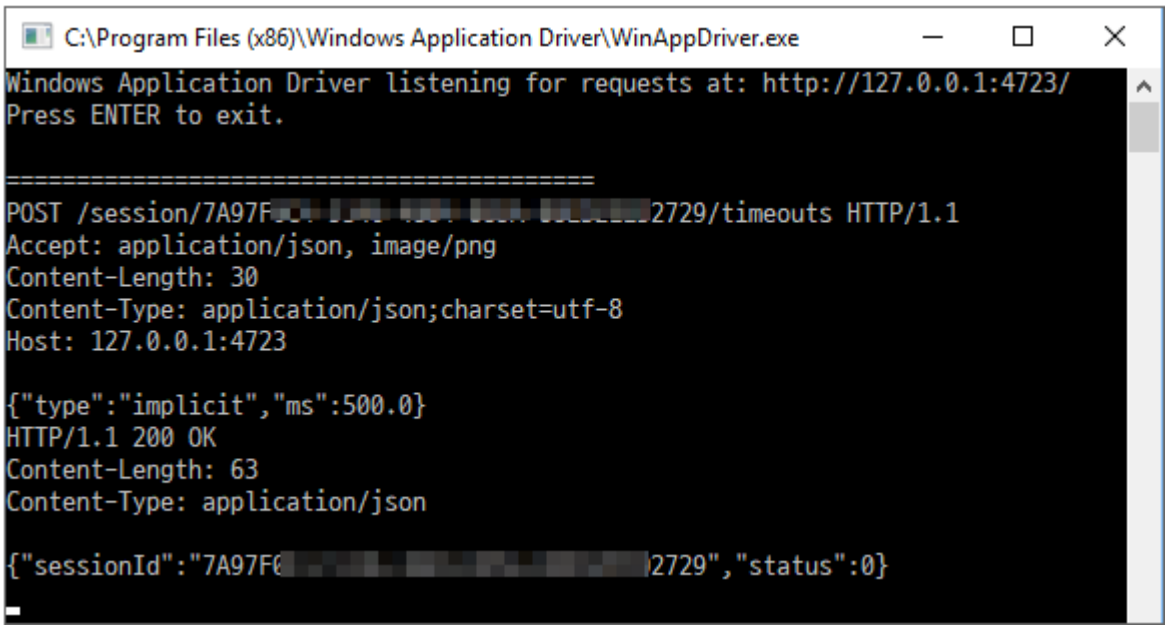


A timeout error may occur depending on the TimeSpan setting value. In that case, change the setting value as follows and proceed with the test.

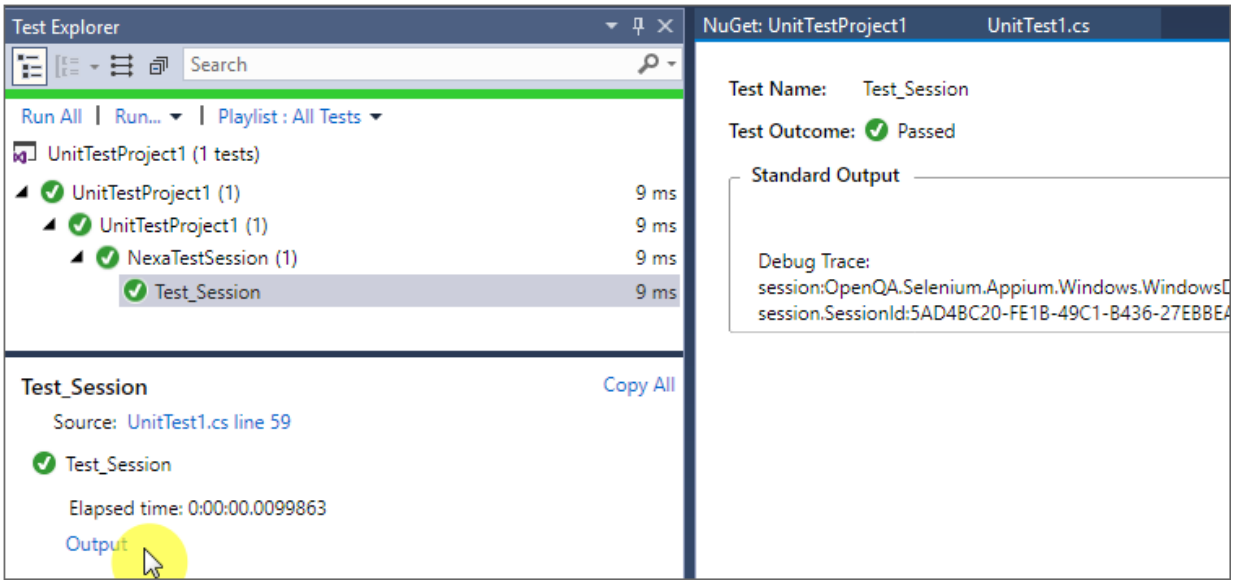
```
TimeSpan time = new TimeSpan(0, 0, 30);
```

7 Select the menu [Test > Run > All Tests] item to proceed with the test. You can see that the installed app is running.

You can check the access information and session being created on the console screen where the WinAppDriver was executed.



8 You can check the successful test information in the Visual Studio test explorer.



Creating the Unit Test Project in Eclipse

1 Run Eclipse.

2 Select the menu [File > New > Java Project] and create a new project.

3 Download the required library and unzip the compressed file (.zip).

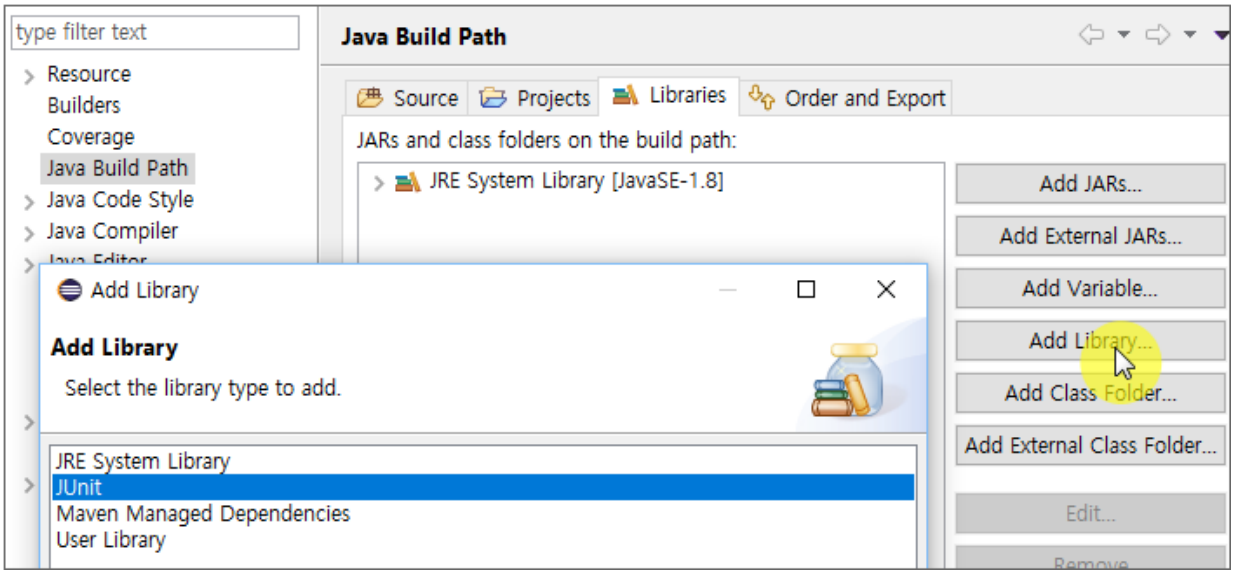
Library	Download Path
Appium Client Libraries	http://appium.io/downloads.html

Library	Download Path
	java-client-7.0.0.jar
Selenium Client	https://www.seleniumhq.org/download/ selenium-java-3.141.59.zip
Apache Commons Lang	http://commons.apache.org/proper/commons-lang/download_lang.cgi commons-lang3-3.8.1-bin.zip

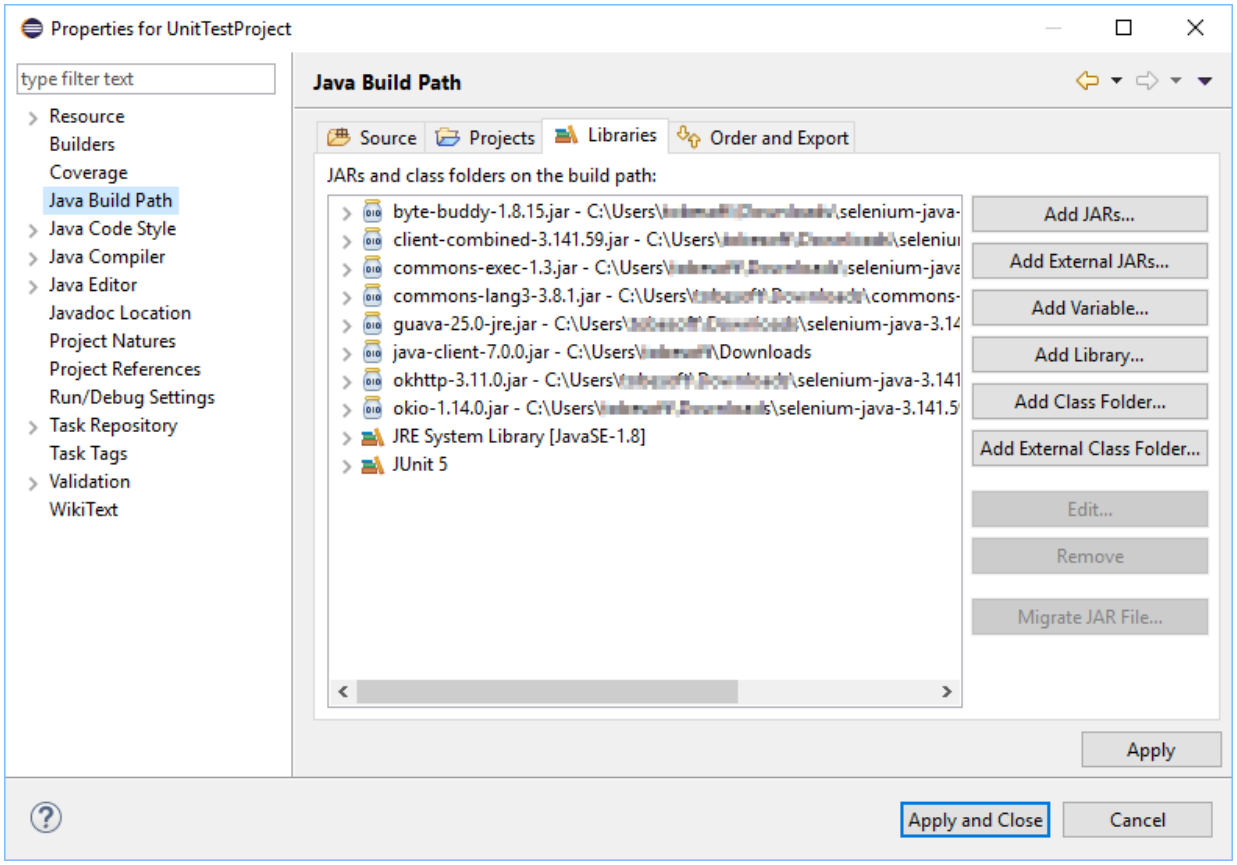


Download and install the Apache Commons Lang library when the following error occurs during execution.
It may not be necessary, depending on the development environment.
`java.lang.NoClassDefFoundError: org/apache/commons/lang3/StringUtils`

4 Add the JUnit library. Select the menu [Project > Properties > Java Build Path > Libraries] tab and then click the [Add Library] button.



5 Add the Appium, Selenium libraries. Click the [Add External JARs] button and add the JAR file from the downloaded library file.



6 Click the [Apply and Close] button to apply the changes.

7 Select the menu [File > New > Class] and then create a new class file. Only enter the Name item and do not modify other options.

8 Modify the created Java source code file as follows.

```
UnitTest.java
1  import org.junit.*;
2  import org.openqa.selenium.remote.DesiredCapab
3  import org.openqa.selenium.remote.RemoteWebEle
4  import io.appium.java_client.windows.WindowsDr
5  import java.util.concurrent.TimeUnit;
6  import java.net.URL;
7
8  public class UnitTest {
9
10     // winappdriver url
11     private String WindowsApplicationDriverUrl
12     // app path
13     private String NexacroAppDr = "C:\\Program
14     // nexacro.exe path
15     private String NexacroAppId = "C:\\Program
```

Modify the settings related to the Nexacro app in the code below. Modify the pathway where the app is installed, the pathway to the nexacro.exe executable file, and the nexacro.exe execution option to

match the actual installation pathway.

	value
WindowsApplicationDriverUrl	`http://127.0.0.1:4723/`
NexacroAppDr	`C:\\Program Files (x86)\\TOBESOFT\\Nexacro N\\21\\app0201`
NexacroAppId	`C:\\Program Files (x86)\\TOBESOFT\\Nexacro N\\21\\app0201\\nexacro.exe`
NexacroAppAg	`-k 'UnitTest' -s 'C:\\Program Files (x86)\\TOBESOFT\\Nexacro N\\21\\app0201\\app0201\\start.json'`

```
import org.junit.*;
import org.openqa.selenium.remote.DesiredCapabilities;
import org.openqa.selenium.remote.RemoteWebElement;
import io.appium.java_client.windows.WindowsDriver;
import java.util.concurrent.TimeUnit;
import java.net.URL;

public class UnitTest {

    // winappdriver url
    private String WindowsApplicationDriverUrl = "http://127.0.0.1:4723/";
    // app path
    private String NexacroAppDr = "C:\\Program Files
(x86)\\TOBESOFT\\Nexacro N\\21\\app0201";
    // nexacro.exe path
    private String NexacroAppId = "C:\\Program Files
(x86)\\TOBESOFT\\Nexacro N\\21\\app0201\\nexacro.exe";
    // nexacro.exe argument
    private String NexacroAppAg = "-k 'UnitTest' -s 'C:\\Program Files
(x86)\\TOBESOFT\\Nexacro N\\21\\app0201\\app0201\\start.json'";
    // search root
    // private String FindNexacro = "Root";

    // OS/Device
    private String TestOs = "Windows";
    private String TestDevice = "WindowsPC";

    // Session Object : WindowsDriver
    private static WindowsDriver<RemoteWebElement> session = null;
```

```
    @Before
    public void setUp() throws Exception {
        // Launch a new instance of nexacro application or find nexacro
application
    }

    @After
    public void tearDown() throws Exception {
        if (session != null) {
            // Nexacro17Session.quit();
        }
    }

    @Test
    public void Test_01_Session()
    {
        System.out.println("session : " + session);
        System.out.println("session.SessionId : " +
session.getSessionId());
    }
}
```

9 Run the Nexacro app and then add code to create the session.

Modify the Setup method in the code above as follows.

```
@Before
public void setUp() throws Exception {
    // Launch a new instance of nexacro application or find nexacro
application
    try {
        // Create a new session to launch nexacro application
        DesiredCapabilities appCapabilities = new DesiredCapabilities();

        appCapabilities.setCapability("app", NexacroAppId);
        appCapabilities.setCapability("appArguments", NexacroAppAg);
        appCapabilities.setCapability("appWorkingDir", NexacroAppDr);
        appCapabilities.setCapability("platformName", TestOs);
        appCapabilities.setCapability("deviceName", TestDevice);

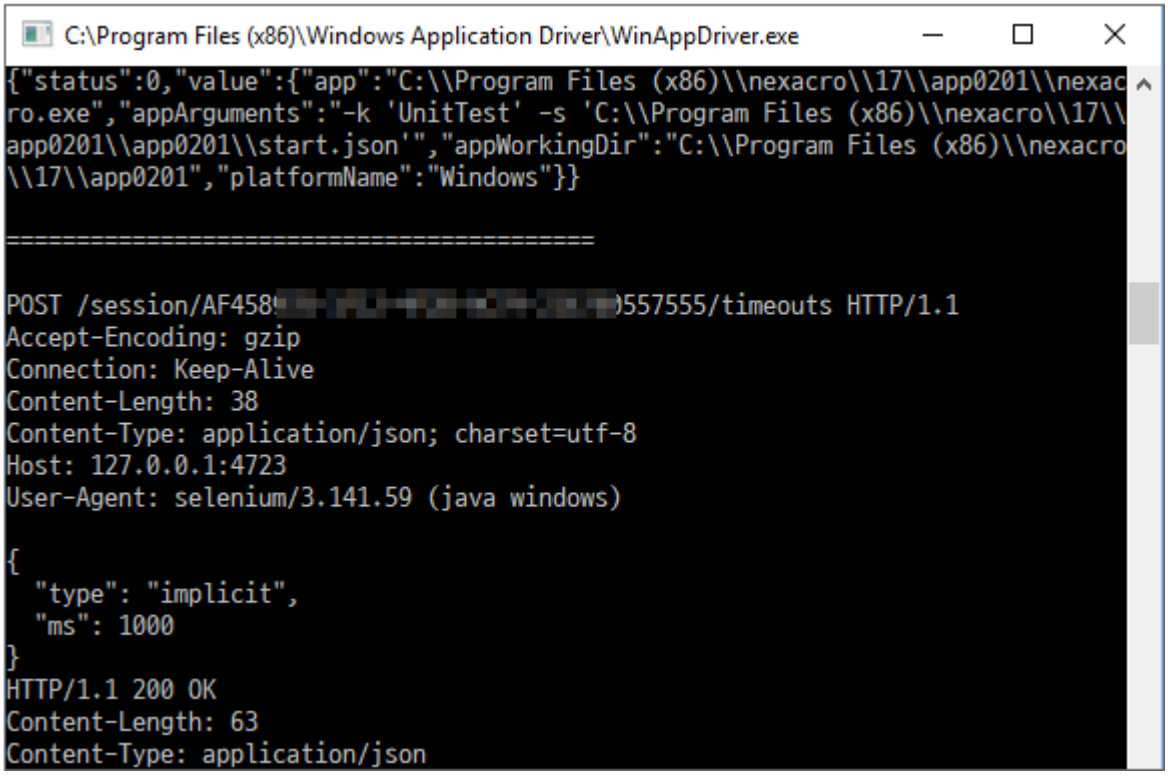
        // Request Remote WinAppDriver Service
```

```
        session = new WindowsDriver<RemoteWebElement>(new
URL(WindowsApplicationDriverUrl), appCapabilities);
        session.manage().timeouts().implicitlyWait(1, TimeUnit.SECONDS);

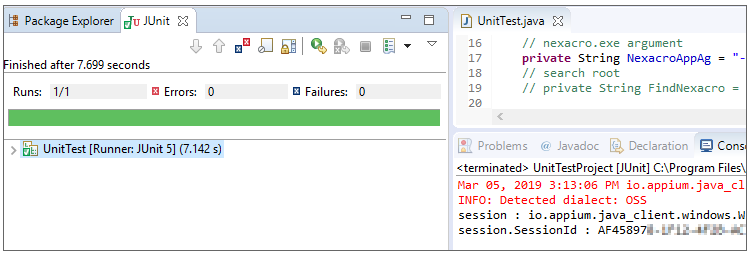
    } catch (Exception e) {
        e.printStackTrace();
    } finally {
    }
}
```

10 Select the menu [Run > Run As > JUnit Test] item to proceed with the test. You can see that the installed app is running.

You can check the access information and session being created on the console screen where the WinAppDriver was executed.



11 You can check the successful test information in the JUnit panel.



4.2.3 Running the Unit Test

1. Add the code to run the unit test as follows.

   Find the Button component in the current session and then process the mouse click command.

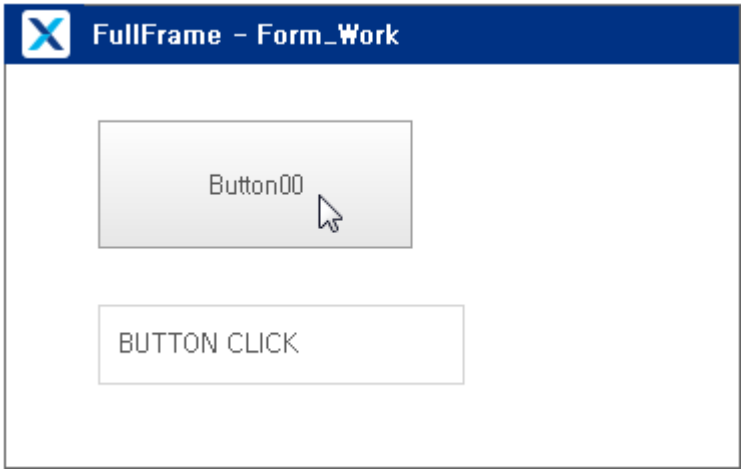
   C#

```
[TestMethod]
public void Test_Button()
{
    WindowsElement element = session.FindElementByAccessibilityId(
"mainframe.WorkFrame.form.Button00");
    Assert.IsNotNull(element);
    Thread.Sleep(TimeSpan.FromSeconds(0.1));
    element.Click();
}
```

   Java

```
@Test
public void Test_Button() throws Exception {
    WebElement element = session.findElementByAccessibilityId(
"mainframe.WorkFrame.form.Button00");
    Assert.assertNotNull(element);
    Thread.sleep(1000);
    element.click();
}
```

2. When the test is run, the installed app is executed. You can see that the mouse cursor is placed on the Button component and the click command is executed.



4.3 Checking the AccessibilityId Property Value

To access the component or control, the FindElementByAccessibilityId method is used, and you need to check the AccessibilityId property value at this time.

4.3.1 Checking the Component Property Value

The method to access the component itself placed on the form is as follows.

> `[MainFrame id property value].[ChildFrame id property value].form.[Component id property value]`

To get the element of the Button component with the id property value being "Button00", write the following code.

C#

```
WindowsElement element = session.FindElementByAccessibilityId(
"mainframe.ChildFrame00.form.Button00");
```

Components placed in the Div component can be accessed as follows.

C#

```
WindowsElement element = session.FindElementByAccessibilityId(
"mainframe.ChildFrame00.form.Div00.form.CheckBox00");
```

4.3.2 Checking the Control & the Item Property Values

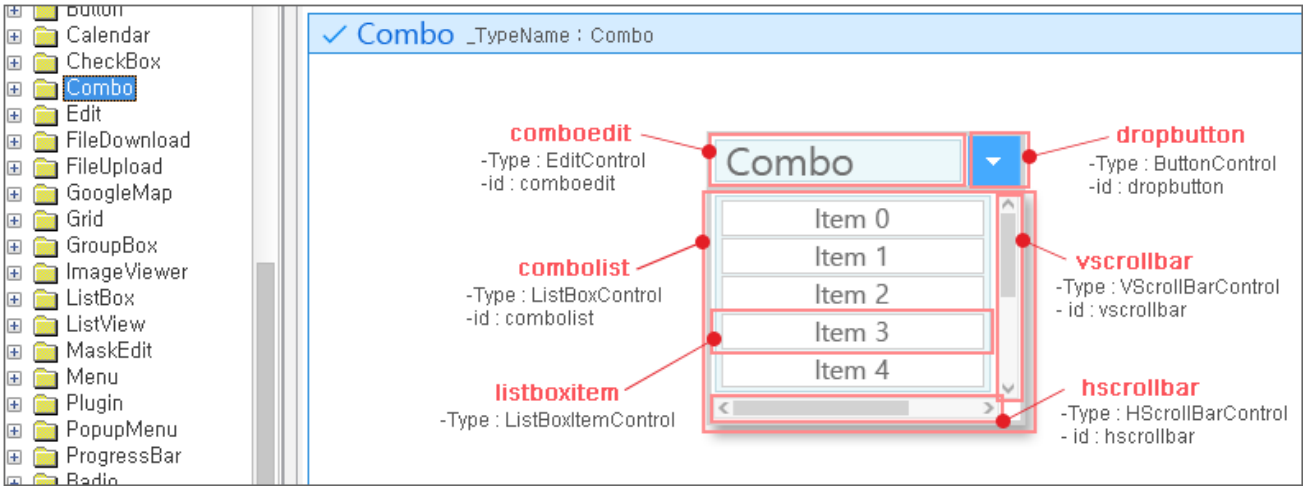
For example, the dropbutton of the Combo component placed in the Div component can generate the element as below.

C#

```
WindowsElement element = session.FindElementByAccessibilityId(
"mainframe.ChildFrame00.form.Div00.form.Combo00.dropbutton");
```

To proceed with the unit test, you need to expand the dropdown list of the Combo component and then select the item. To expand the dropdown list, you need to access the ButtonControl in the Combo component and then check the property value of the control.

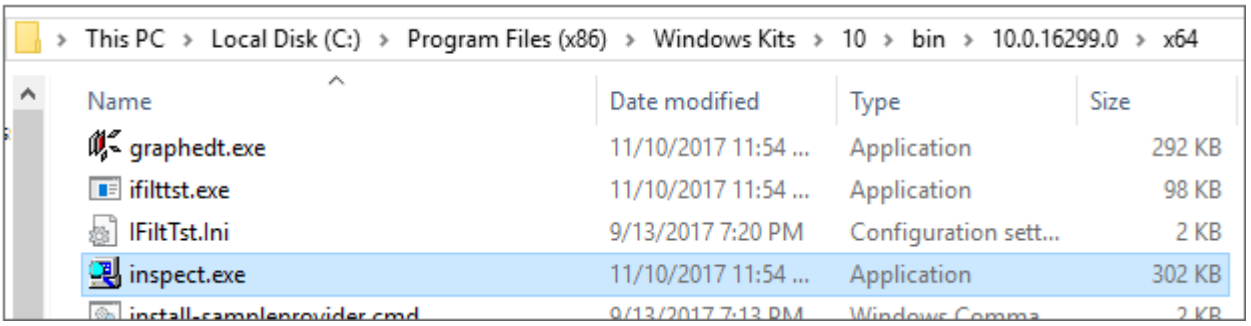
The control property value can be checked in the component overview by running Help (shortcut key F1) in the Nexacro Studio. You can check that the id property value of the button clicked to expand the dropdown list in the Combo component is "dropbutton".



4.3.3 Checking the Property Value with Inspect

> Property values (sub-controls) that are not exposed in Help may be changed depending on the product version.
> When the property value is changed, the written test may not work.

However, it does not tell you how to access Item 3 after expanding the dropdown list. In this case, you can use the tool called "Inspect". "Inspect" can be found in the installation pathway if Windows SDK is installed.



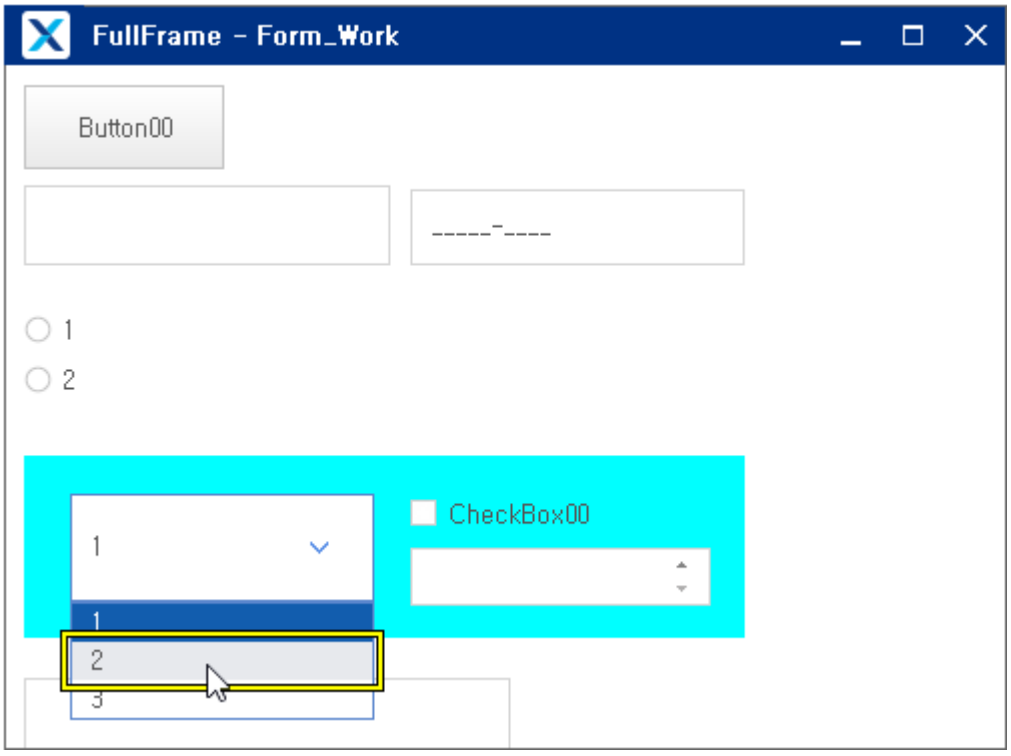
> Inspect
> https://docs.microsoft.com/en-us/windows/desktop/winauto/inspect-objects

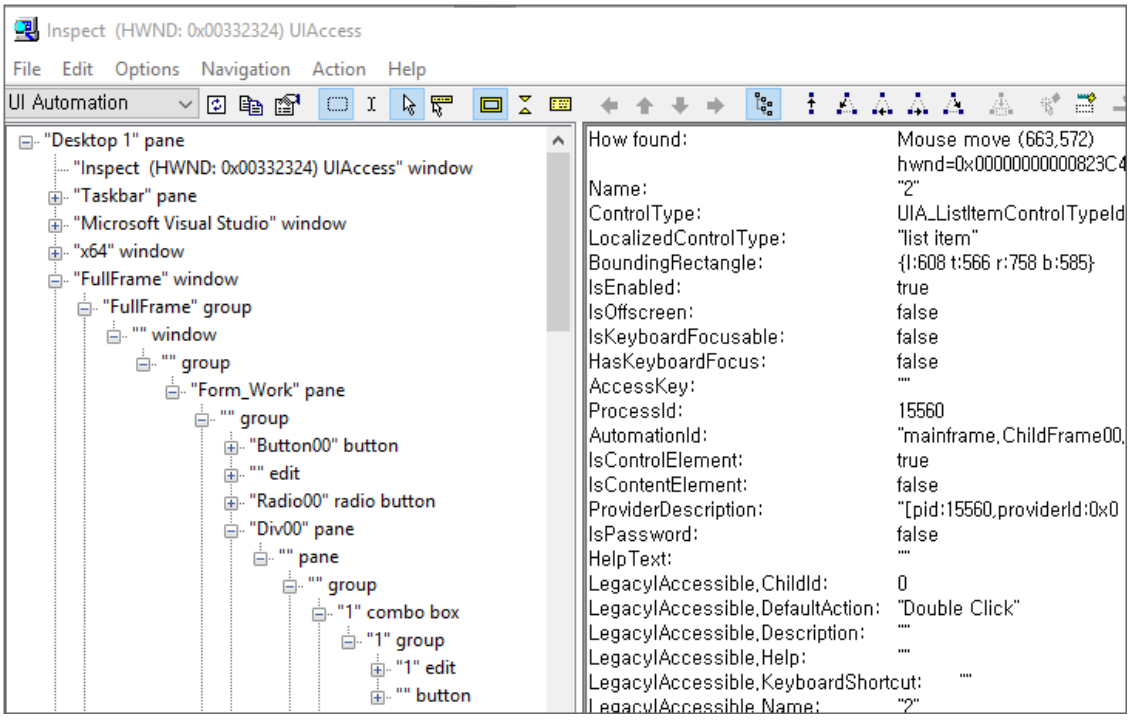
1. Run the inspect.exe file in the Windows SDK installation pathway.

2 Run the Nexacro and then move the mouse cursor to the desired location.

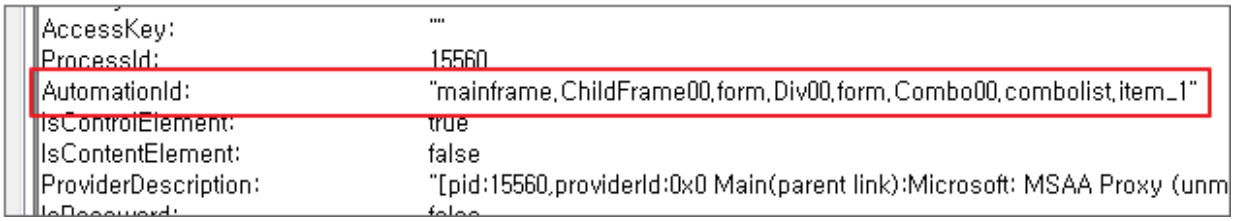
3 Click the dropdown button of the Combo component to expand the dropdown list.

4 Place the mouse cursor on the desired item area. As shown in the figure below, a borderline is displayed in the target area recognized by Inspect, and the related information is displayed in the Inspect window.





Various information is displayed, and the item to be used as the AccessibilityId property value among them is "AutomationId".

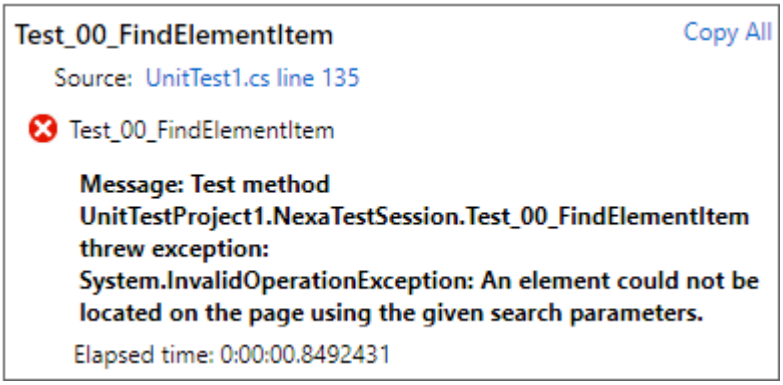


When selecting the 2nd item from the dropdown list, you can generate the element as shown below.

C#

```
WindowsElement element = session.FindElementByAccessibilityId(
"mainframe.ChildFrame00.form.Div00.form.Combo00.combolist.item_1");
```

The element corresponding to the item can be accessed after the dropdown list is expanded. If you try to access the item before expanding the list, then it will be processed as a test failure.



In the test code, find the dropbutton element first, process the mouse click event, and then access the desired item when the dropdown list is expanded.

C#

```
[TestMethod]
public void Test_00_FindElementItem()
{
    WindowsElement element = session.FindElementByAccessibilityId(
"mainframe.ChildFrame00.form.Div00.form.Combo00.dropbutton");
    Assert.IsNotNull(element);
    element.Click();
    Thread.Sleep(TimeSpan.FromSeconds(0.1));
    WindowsElement elementItem = session.FindElementByAccessibilityId(
"mainframe.ChildFrame00.form.Div00.form.Combo00.combolist.item_1");
```

```
    Assert.IsNotNull(elementItem);
    Debug.WriteLine("elementItem.Text:" + elementItem.Text);
}
```

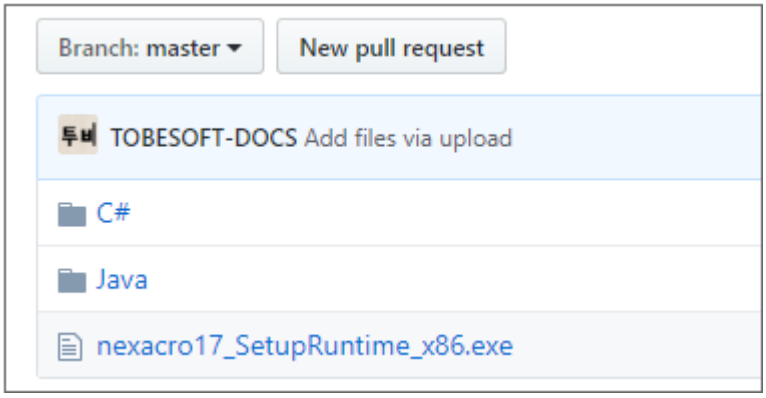
4.4 Running the Demo Unit Test by Feature

You can download the Nexacro app installation file and the Java and C# sample files to run demos by feature.

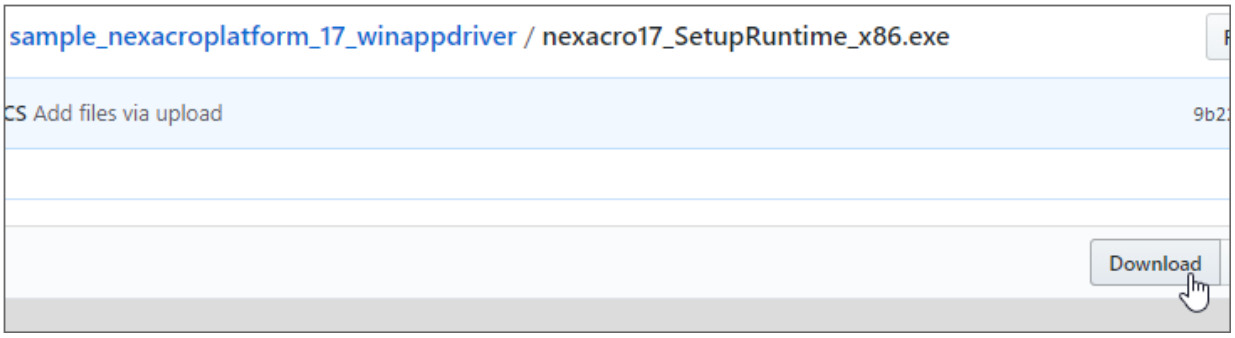
1. Access the address below in a web browser.

   > https://github.com/TOBESOFT-DOCS/sample_nexacroplatform_17_winappdriver

   You can see 2 folders and 1 installation file.



2. Download the nexacro17_SetupRuntime_x86.exe file. Click the file name with the mouse to see the [Download] button.

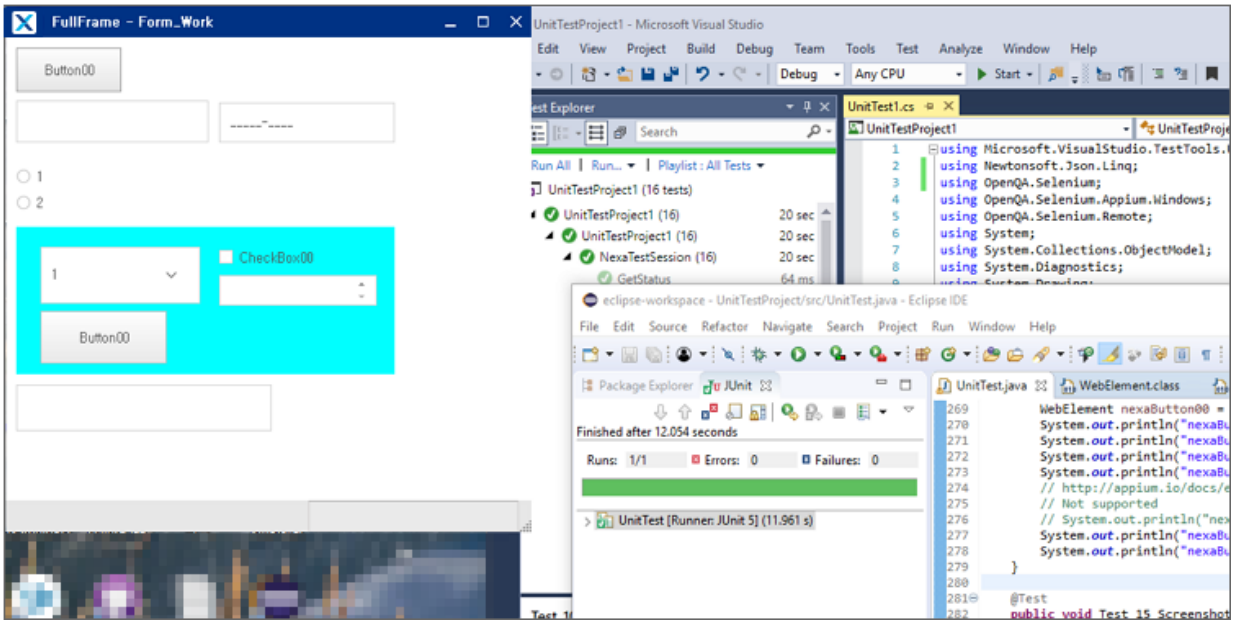


3. Install the app by running the downloaded installation file. The app will be installed in the pathway below.

```
C:\Program Files (x86)\nexacro\17\app0201
```

4. Paste the C# and Java samples into the project created in the Unit Test Project.

5. Run the unit test and check if the unit test is processing normally.



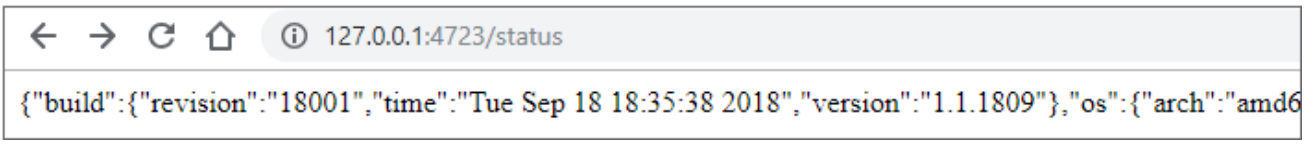
4.5 Supported Interface

> Not all SeleniumRC and Appium interfaces are supported.
> Features that cannot be supported in the Nexacro or are not supported in the WinAppDriver are excluded.

> The items below can be used when the client does not support a separate interface and they are called directly from the web browser or are called using HttpWebResponse, etc.

HTTP	WebDriver	WinAppDriver
GET	/status	/status
GET		/sessions
GET	/session/{session id}/window	/session/:sessionId/window
GET	/session/{session id}/window/handles	/session/:sessionId/window/handles

HTTP	WebDriver	WinAppDriver
POST	/session/{session id}/window/maximize	/session/:sessionId/window/maximize



4.5.1 Session

Creating a Session

Create the session with the winappdriver information and then run the app.

C#	`session = new WindowsDriver(new Uri(WinAppDriverUrl), appCapabilities);`
Java	`session = new WindowsDriver(new URL(WindowsApplicationDriverUrl), appCapabilities);`
Comm	`POST /session { …capabilities json data… }`

Ending a Session

End the session and then close the app.

C#	`session.Quit();`
Java	`session.quit();`
Comm	`DELETE /session/:session_id`

Some of the interfaces provided by the WinAppDriver are not supported in the Nexacro.

4.5.2 Timeout

Setting a Timeout

Set the timeout value to be applied during the session operation.

C#	`session.Manage().Timeouts().ImplicitlyWait(TimeSpan.FromSeconds(0.5));`
Java	`session.manage().timeouts().implicitlyWait(1, TimeUnit.SECONDS);`

Comm	`POST /session/:session_id/timeouts {"type":"implicit","ms":500.0}`

4.5.3 Frame

Checking the Mainframe Information

Return the titletext property value of the MainFrame object.

C#	`session.Title;`
Java	`session.getTitle();`
Comm	`GET /session/:sessionId/title`

4.5.4 Window

Changing the Enabled Window

Change the current window. Windowhandle can be obtained from session.WindowHandles. For the initial execution, the MainFrame object is set as the current window.

C#	`session.SwitchTo().Window(session.WindowHandles[0]);`
Java	`Set<String> handles = session.getWindowHandles();` `Set handles = session.getWindowHandles();`
Comm	`POST /session/:sessionId/window {"name":"0x000A12BC"}`

Ending the Enabled Window

End the current window.

C#	`session.Close();`
Java	`session.close();`
Comm	`DELETE /session/:sessionId/window`

Obtaining the Enabled Window Handle

Return the current window handle object.

C#	`handle = session.CurrentWindowHandle;`

Java	`Handle = session.getWindowHandle();`
Comm	`GET /session/:sessionId/window_handle`

Obtaining the Window Handle Array

Return all window handle objects as an array.

C#	`handlearray = session.WindowHandles;`
Java	`Set handles = session.getWindowHandles();`
Comm	`GET /session/:sessionId/window_handles`

Checking the Window Size Information

Return the window size in the form of the Size object.

C#	`session.Manage().Window.Size;`
Java	`session.getWindowHandles().size()`
Comm	`GET /session/:sessionId/window/:windowHandle/size`

Changing the Window Size

Change the window size.

C#	`session.Manage().Window.Size = new Size(width,height);`
Java	`session.manage().window().setSize(new Dimension(width, height));`
Comm	`POST /session/:sessionId/window/:windowHandle/size`

Checking the Window Position Information

Return the window position in the form of the Point object.

C#	`point = session.Manage().Window.Position;`
Java	`point = session.manage().window().getPosition()`
Comm	`GET /session/:sessionId/window/:windowHandle/position`

Changing the Window Position

Change the window coordinates.

C#	`session.Manage().Window.Position = new Point(left,top);`
Java	`session.manage().window().setPosition(new Point(left, top))`
Comm	`POST /session/:sessionId/window/:windowHandle/position`

Maximizing the Window

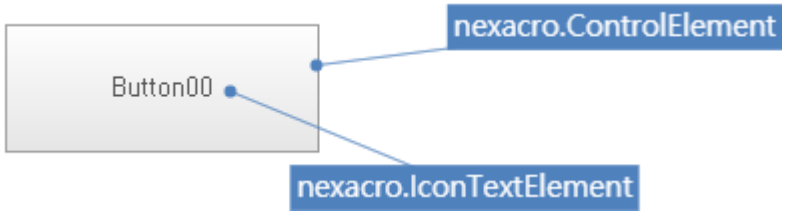
Maximize the window size.

C#	`session.Manage().Window.Maxmize();`
Java	`session.manage().window().maximize();`
Comm	`POST /session/:sessionId/window/:windowHandle/maximize`

4.5.5 Element

Element is the smallest unit of the WinAppDriver operation. In the Nexacro, nexacro Element, which is the element of components and controls, is processed as Element.

ControlElement surrounds the component and Elements, such as ContainerElement, InputElement, IconTextElement, TextElement, are located inside. In the case of the Button component, it is configured as follows.



Obtaining the Element

Return the ControlElement object of the component and control.

C#	`element = session.FindElementByAccessibilityId("mainframe.ChildFrame00.form.Button00");`
Java	`element = session.findElementByAccessibilityId("mainframe.ChildFrame00.form.Button00");`
Comm	`POST /session/:sessionId/element`

	{ "using":"accessibilityid", "value":"mainframe.ChildFrame00.form.Button00" }



In the case of the objParentFrame parameter value set to null when executing the open method in the Nexacro, it is accessed based on the ChildFrame ID.

C#

```
// When specifying parent null: nexacro.open("openframe", url, null, ...)
nexaButton00 = session.FindElementByAccessibilityId("openframe.form.Button00");

// When specifying parent frame: nexacro.open("openframe", url, ownerframe,
...)
nexaButton00 = session.FindElementByAccessibilityId(
"mainframe.ChildFrame00.openframe.form.Button00");
```

Obtaining the Element Array

Return the ControlElement object list of component and control as an array value.

C#	element = session.FindElementsByAccessibilityId("mainframe.ChildFrame00.form.Button00");
Java	element = session.findElementsByAccessibilityId("mainframe.ChildFrame00.form.Button00");
Comm	POST /session/:sessionId/elements { "using":"accessibilityid", "value":"mainframe.ChildFrame00.form.Button00" }

Checking the Element id Value

Return the id value of the WinAppDriver Element.

C#	id = element.Id;
Java	id = element.getTagName()
Comm	GET /session/:sessionId/element/:id/name

Checking the Text Information Displayed on Element

Return the text value of Element.

C#	text = element.Text;
Java	text = element.getText();
Comm	GET /session/:sessionId/element/:id/text



The property value specified in the Nexacro prioritizes processing the text displayed on the screen. The priorities are as follows.

`displaytext` > `text` > `titletext` > `value`

Processing the Element Mouse Click Event

The mouse click event occurs at the center of the Element.

C#	`element.Click();`
Java	`element.click();`
Comm	`POST /session/:sessionId/element/:id/click`



Since the mouse click event occurs at the center of the element, the caret position does not move to the front in the case of the MaskEdit component. In this case, you can enter the CTRL+"A" keys to select the entire input area and then enter the text string.

C#

```
[TestMethod]
public void Test_10_MaskEditInput()
{
    WindowsElement nexaMaskEdit00 = session.FindElementByAccessibilityId(
"mainframe.ChildFrame00.form.MaskEdit00");
    Assert.IsNotNull(nexaMaskEdit00);
    Thread.Sleep(TimeSpan.FromSeconds(0.1));

    nexaMaskEdit00.Clear();
    nexaMaskEdit00.Click();
    nexaMaskEdit00.SendKeys(Keys.Control + "a");
    Thread.Sleep(TimeSpan.FromSeconds(0.1));
    nexaMaskEdit00.SendKeys("abcde1234"+Keys.Enter);
    Assert.AreEqual("abcde-1234", nexaMaskEdit00.Text);
}
```

Deleting the Element Input Value

Delete the input value of the component with the text editing area.

C#	`element.Clear();`

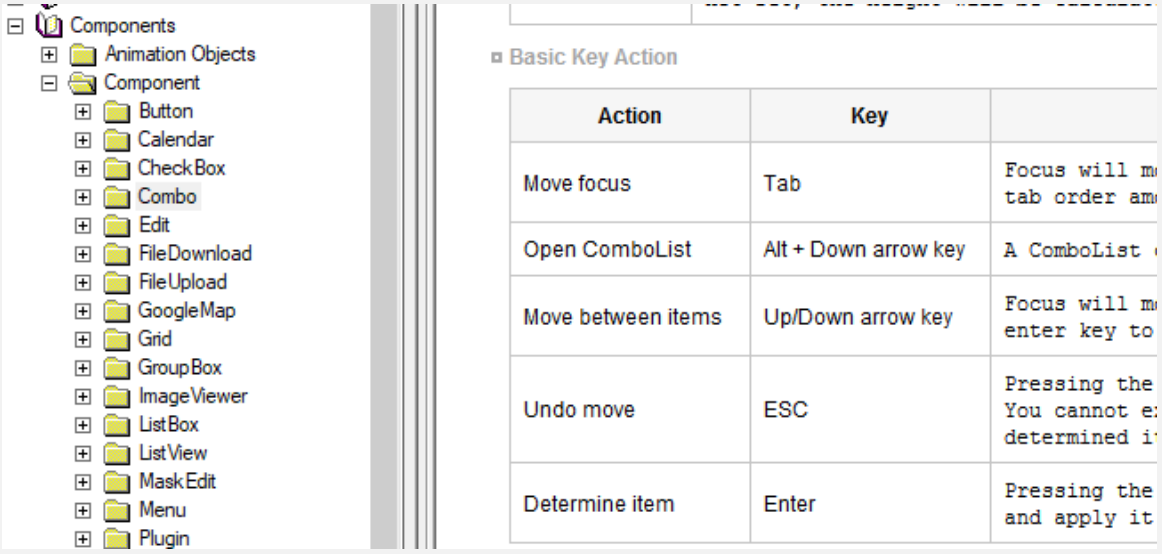
Java	`element.clear();`
Comm	`POST /session/:sessionId/element/:id/clear`

Passing the Keyboard Input Value to Element

Process the keyboard input value.

C#	`element.SendKeys(KeyString)`
Java	`element.sendKeys(KeyString)`
Comm	`POST /session/:sessionId/element/:id/value`

Input values are processed differently depending on the component.
Please refer to the "Basic Key Action" item of the component in Product Help.



When passing keyboard input values, the app must be enabled, and if 2 or more windows are displayed, then the target window must be enabled.

The keyboard input value must be passed after processing the mouse click event to the target element to operate normally.

If 2 or more displays are being used, then the keyboard input values may not be passed normally. Either use only one display, or restart the WinAppDriver.

Checking Whether the Element is Selected

Return the Selected status value of the Element.

C#	`isSelected = element.Selected;`
Java	`isSelected = element.isSelected();`
Comm	`GET /session/:sessionId/element/:id/selected`



Components and controls supported in the Nexacro are as follows.
CheckBox, RadioButton, TabButton, ListBoxItem, MenuItem, TreeCell, GridCell

Checking Whether the Element Can be Used

Return the Enabled status value of the Element.

C#	`isEnabled = element.Enabled;`
Java	`isEnabled = element.isEnabled();`
Comm	`GET /session/:sessionId/element/:id/enabled`

Checking Whether the Element Screen is Displayed

Return whether the Element is displayed.

C#	`isDisplayed = element.Displayed;`
Java	`isDisplayed = element.isDisplayed();`
Comm	`GET /session/:sessionId/element/:id/displayed`



It is not returning the visible property value of the component, but rather it is returning whether it is displayed on the actual screen.

element.Displayed	Input Operation
true	visible = true When all or part of the component is shown in the screen area
false	visible = false When visible = true, but if the scroll bar of the parent container component is not moved, located in an invisible area on the screen



If the visible property value of the component is set to false when starting the app, then the corresponding Element cannot be found. This is because the Element is created only when the visible property value is true in the Nexacro.
The Element can be processed by setting the visible property value to true when loading the app and then changing the visible property value of the component to false within the onload event function.

Checking the Element Size Information

Return the Element size in the form of the Size object.

C#	`size = element.Size;`
Java	`size = element.getSize();`
Comm	`GET /session/:sessionId/element/:id/size`

Checking the Element Position Information (Based on the Parent Element)

Return the Element position in the form of the Point object based on the parent Element.

C#	`size = element.Location;`
Java	`size = element.getLocation();`
Comm	`GET /session/:sessionId/element/:id/location`

Checking the Element Position Information (Based on the Screen)

Return the Element position in the form of the Point object based on the Screen.

C#	`size = element.LocationOnScreenOnceScrolledIntoView;`
Java	Not supported
Comm	`GET /session/:sessionId/element/:id/location_in_view`

Obtaining the Enabled Element

Return the currently enabled Element.

C#	`element = session.SwitchTo().ActiveElement();`
Java	`element = session.switchTo().activeElement();`

Comm	`GET /session/:sessionId/element/active`

Comparing 2 Elements

Compare Elements to see if they are the same Element.

C#	`isEqual = element.Equals(element2);`
Java	`isEqual = element.equals(element2);`
Comm	`GET /session/:sessionId/element/:id/equals`

Checking the UIA Interface Attribute Value of the Element

Provide the attribute value of the UIA Interface of the Element.

C#	`element.GetAttribute("AutomationId");`
Java	`element.getAttribute("AutomationId");`
Comm	`GET /session/:sessionId/element/:id/attribute/:name`



Automation Element Property Identifiers
https://docs.microsoft.com/en-us/windows/desktop/WinAuto/uiauto-automation-element-propids

4.5.6 ScreenShot

Obtaining the Window & Element Screenshot

Return the Screen window area or the Element as a Screenshot object.

C#	`Screenshot sessionshot = session.GetScreenshot();`
Java	`Screenshot sessionshot = session.getScreenshotAs();`
Comm	`GET /session/:sessionId/screenshot` `GET /session/:sessionId/element/:id/screenshot`



You can use the SaveAsFile method, AsBase64EncodedString property, to convert to an image file or to Base64 format.



The window must be exposed at the top to obtain a normal screenshot.



Java code can only access repositories for which you have privileges.

Java

```
FileHandler.copy(sessionshot, new File(System.getProperty("user.home")+
"\\Downloads\\SessionScreenShot.png"));
```

4.6 Related Link Information

WebDriver Interface

```
http://www.w3.org/TR/webdriver1/
https://www.seleniumhq.org/projects/webdriver/
https://developer.mozilla.org/en-US/docs/Web/WebDriver
```

AppDriver Interface

```
https://github.com/Microsoft/WinAppDriver
https://appium.io/docs/en/drivers/windows/
https://developer.microsoft.com/en-us/microsoft-edge/tools/webdriver/
```

UIA Interface

```
https://docs.microsoft.com/en-us/windows/desktop/WinAuto/entry-uiauto-win32
https://docs.microsoft.com/en-us/windows/desktop/winauto/testing-tools
https://en.wikipedia.org/wiki/Microsoft_UI_Automation
```

MSAA Interface

```
https://docs.microsoft.com/en-us/windows/desktop/WinAuto/microsoft-active-accessibility
https://docs.microsoft.com/en-us/windows/desktop/winauto/testing-tools
https://en.wikipedia.org/wiki/Microsoft_Active_Accessibility
```

Part II.

How Nexacro Platform Works

5.

Structures of Project and Files

If you want to develop a Form running on the Nexacro, you must create a project first. Then, you need to create files or link existing data to prepare the constituents necessary for the project, including components, screens, environments and themes (or styles). The form can be executed in the form of either an app or a Form itself.

This chapter explains what types of files are created in the working folder as a result of creating a project or form and what role each file performs. In addition, this chapter touches upon the files that are created as a result of generation.

5.1 Creating Project

A project contains basic information on settings for developing a Form. Creating a new project accompanies the creation of basic files—including settings files—that comprise the project.

The below table describes such files.

Created file or folder	Extension	Description
Nexacro Project	XPRJ	Project settings file [project name]+extension
Application Info	XADL	Application settings file It contains the information on the screens that are set in the process of creating the project. It also includes the information on basic themes. The below combination is the format of a default .xadl file. [project name]+extension
Type Definition	XML	Settings file that contains information on modules, components, services, protocol and updates The name of the default file is typedefinition.xml.
Environment	XML	This file contains the information on screens, variables, cookies and HTTP headers. The name of the default file is environment.xml.

Created file or folder	Extension	Description
AppVariables	XML	This file contains the information on Datasets and variables that can be used globally by any application in a project. The default file will be created even if you have not input any data. The name of the default file is appvariables.xml.
resource	Folder	This folder manages resources including themes, initial values, StringResource, images and fonts. This folder can contain the following sub-folders: [_font_], [_images_], [_initvalue_], [_stringrc_], [_theme_] and [_xcss_].
extlib	Folder	Manage the related resources when installing the module.
Base	Folder	Service folder A form type service is basically created when you create a project. The PrefixID of such a service is Base. The PrefixID or folder name of a created service can be changed or deleted.



A service folder for frames may be added depending on what type of frame structure you set when you create a project.
If the application consists of multiple frames, for example, a service folder for frames will be created automatically.

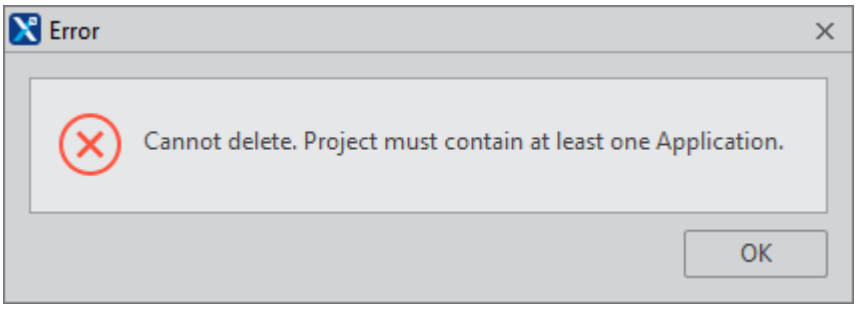
5.1.1 Project Settings File (XPRJ)

This file is created automatically when you create a project. The information on application settings can be added or deleted.

```
<?xml version="1.0" encoding="utf-8"?>
<Project version="2.1" nexacrosdk="Latest Version" sdkversion="">
  <EnvironmentDefinition url="environment.xml"/>
  <TypeDefinition url="typedefinition.xml"/>
  <AppVariables url="appvariables.xml"/>
  <AppInfos>
        <AppInfo url="[file name].xadl"/>
        <AppInfo url="[file name].xadl"/>
  </AppInfos>
</Project>
```

5.1.2 Application Settings File (XADL)

An App Info file is created automatically when you create a project, and you can add more files. A project must include at least one App Info file. Therefore, if you attempt to delete a sole App Info file, Nexacro Studio will not remove it while opening the below alert message.




```
<?xml version="1.0" encoding="utf-8"?>
<ADL version="2.0">
  <Application id="[id]" screenid="[screen id]">
    <Layout>
      <MainFrame>
        <ChildFrame/>
      </MainFrame>
    </Layout>
  </Application>
  <Script type="xscript5.1"><![CDATA[…]]></Script>
</ADL>
```

You can add a script to an .xadl file. An App Info file manages a script that affects the entire scope of the application, such as an event for the application or a certain frame.

5.1.3 Type Definition

This settings file contains information on modules, components, services, protocol and updates. You can add, change or delete information for each item. In the build process, Nexacro Studio may automatically edit the information on protocols and deployment options.

	Feature	Description
Modules	Module information	Contains information on modules developed through JavaScript. - You can add self-developed modules - It can be registered as a js or json file. If there is a dependency relationship

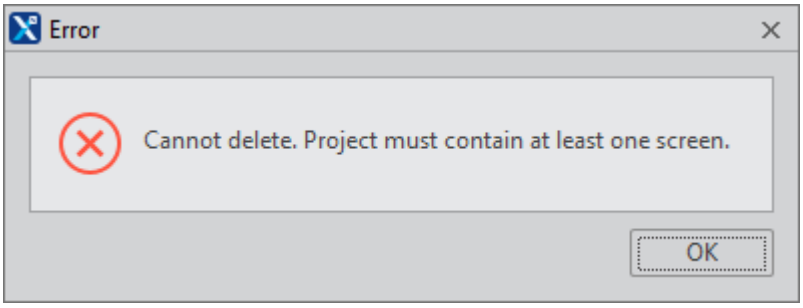
	Feature	Description
		between files, register the files in order.
Components	Object information	Contains information on objects needed to run applications - Designated objects are included in distribution - Each object is assigned based on a module
Services	Service group	Service groups are defined internally according to the project size - Each service group can set the cache level individually
Protocols	Protocol information	You can register/delete protocols and edit them for each device type.
Update	Deployment options	Information related to app deployment (Runtime version) is assigned

```
<?xml version="1.0" encoding="utf-8"?>
<TypeDefinition version="3.0">
  <Modules>
    <Module url="CompBase.json"/>
    <Module url="ComComp.json"/>
    <Module url="Grid.json"/>
    <Module url="ListView.json"/>
    <Module url="DeviceAPI.json"/>
    <Module url="MobileComp.json"/>
  </Modules>
  <Components>
    <Component type="JavaScript" id="Button" classname="nexacro.Button"/>
    <Component type="JavaScript" id="Combo" classname="nexacro.Combo"/>
    <Component type="JavaScript" id="Edit" classname="nexacro.Edit"/>
    ...
  </Components>
  <Services>
    <Service prefixid="theme" type="resource" url="./_resource_/_theme_/"/>
    <Service prefixid="initvalue" type="resource" url=
"./_resource_/_initvalue_/"/>
    <Service prefixid="xcssrc" type="resource" url="./_resource_/_xcss_/"/>
    <Service prefixid="imagerc" type="resource" url="./_resource_/_images_/"/>
    <Service prefixid="font" type="resource" url="./_resource_/_font_/"/>
    <Service prefixid="stringrc" type="resource" url="./_resource_/_stringrc_/"
/>
    <Service prefixid="extlib" type="resource" url="./_extlib_/"/>
    <Service prefixid="Base" type="form" cachelevel="session" url="./Base/"/>
  </Services>
  <Protocols/>
  <DeviceAdaptors/>
  <Update/>
```

```
</TypeDefinition>
```

5.1.4 Environment

This settings file contains the information on screens, variables, cookies and HTTP headers. A project must include one screen at least. If you try to delete a sole screen, the below alert will appear and Nexacro Studio will not remove it.




```
<?xml version="1.0" encoding="utf-8"?>
<ENV version="2.1">
  <Environment themeid="theme::blue" datatyperule="2.0">
    <ScreenDefinition>
      <Screen id="[Screen id]" type="[type property value]"/>
    </ScreenDefinition>
    <Variables/>
    <Cookies/>
  </Environment>
</ENV>
```

5.1.5 AppVariables

This settings file contains the information on Datasets and variables that can be used globally by any application in a project.

```
<?xml version="1.0" encoding="utf-8"?>
<AppVariables version="2.0">
  <Datasets>
    <Dataset id="[Dataset id]">
      <ColumnInfo/>
      <Rows/>
```

```
    </Dataset>
  </Datasets>
  <Variables>
    <Variable id="[Variable id]" initval="[init value]"/>
  </Variables>
</AppVariables>
```

5.2 Resources

5.2.1 initValue

When you create a project, an initValue folder is created rather than an initValue file. Later on, you can add an XML file to save initial settings values. After creating a file, select an object and then specify initial values for the object. Although the file format is XML. the file extension is *.xiv*.

```
<?xml version="1.0" encoding="utf-8"?>
<GlobalPropInitvalueDefinition>
    <nexacro.Button>
        <initvalueid id="[initvalueid ]" ...  />
    </nexacro.Button>
    <nexacro.Edit>
        <initvalueid id="[initvalueid ]" ... />
    </nexacro.Edit>
</GlobalPropInitvalueDefinition>
```

5.2.2 XTHEME

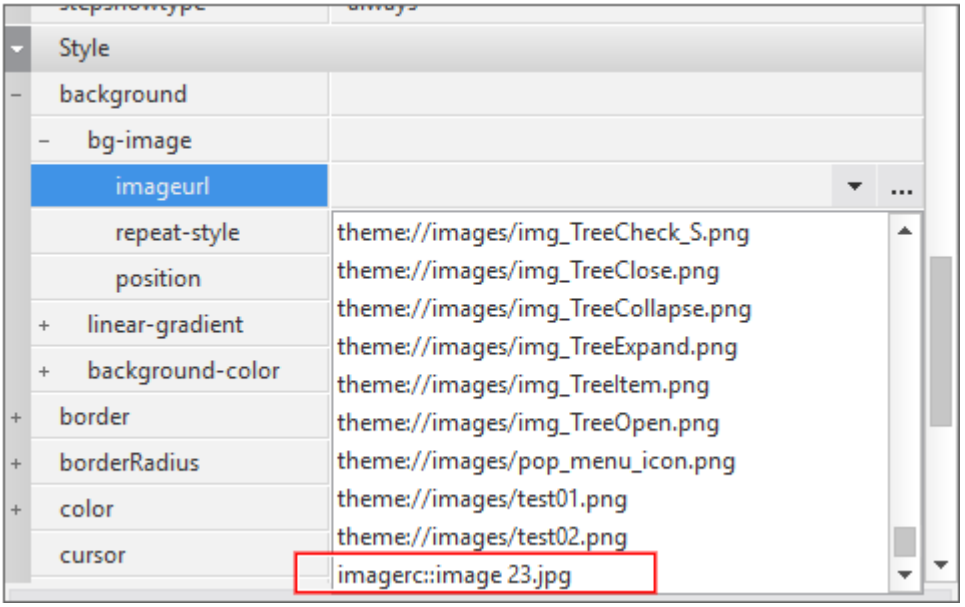
If you see the Resource Explorer of Nexacro Studio immediately after creating a project, you will find two roots of themes: one is **Nexacro Theme** and the other is **Theme**. The Nexacro Theme root includes basic theme files including *default.xtheme* while the Theme root is empty at its inception. You can add new themes only to the Theme root, not Nexacro Theme. You can copy existing themes into or create new themes in the Theme root.

Basically a theme is a combination of a style file (*theme.xcss*) and a group of image files.

The theme files included in the Nexacro Theme root are stored in a separate cache space. You cannot edit those basic theme files.

5.2.3 ImageResource

This segment manages image files that can be globally used within a project. The added image files will be exposed in the Properties pane when you need to specify an image file as the value of a certain property in the process of developing a Form.



5.2.4 UserFont

This segment manage font files, which are largely divided into a conventional type (*.woff*, *.ttf*, *.off*) and UserFont type. UserFont files are saved with the extension *.xfont*, and their codes are written as described below.

```
@font-face {
    font-family : 'NanumGothic';
    font-style : normal;
    font-weight : 400;
    src : local("NanumGothic"),
          url(NanumGothic.ttf) format('truetype'),
          url(NanumGothic.eot),
```

```
            url(NanumGothic.eot?#iefix) format('embedded-opentype'),
            url(NanumGothic.wpff2) format('woff2'),
            url(NanumGothic.woff) format('woff'),
}
```

5.3 Others

5.3.1 Form (XFDL)

A form is the definition of a screen that users actually handle to do their jobs. A form contains program codes to conduct various invisible tasks such as data transaction with a server or calculations.

Element	Description
Component Deployment	Includes the information to deploy a visible object component Sets up component properties and events
Object Set-up	Sets up properties and events of an invisible object, such as a Dataset and etc
Script	Writes the event function of all of the objects, including the Form, as a script Writes the required functions, in addition to the event function, as a script
BindItem	Sets up the bindItem object, which connects a Dataset with a Form, a component and an invisible object

```
<?xml version="1.0" encoding="utf-8"?>
<FDL version="2.1">
    <Form>
        <Layouts>
            <Layout>
                <Button/>
            </Layout>
        </Layouts>
        <Script type="xscript5.1"><![CDATA[]]></Script>
        <Objects>
            <Dataset/>
        </Objects>
        <Bind>
            <BindItem/>
        </Bind>
    </Form>
```

```
</FDL>
```



A form cannot designate an XCSS file, which can apply only to the application level.

5.3.2 Script (XJS)

You can write a script in a Form file if the script is necessary only to the Form. If a script applies a whole application, however, it should be written separately. Then, you should program each form to refer to the script.

You can make a Form refer to a separate script by writing a code as written below.

```
include "lib::comLib.xjs"
```

If a script file is too long, you can divide it into several portions. In such a case where multiple `.xjs` files comprise one script file, you can include a specific reference rather than one big inclusive reference when writing source codes. For example, You can divide the foregoing example comLib.xjs into the below script files. Afterwards, you can add new features just by modifying comLib.xjs without changing the whole form file, even if the number of reference files increases.

```
include "lib::comConstants.xjs";
include "lib::comService.xjs";
include "lib::comForm.xjs";
include "lib::comGrid.xjs";
include "lib::comUtil.xjs";
include "lib::comDataset.xjs";
```

5.3.3 Style Settings File (XCSS)

Although CSS has its official standards, the ways it applies to each web browser are different from each other. Therefore, in the process of developing web apps, you need to specify several CSS properties that are necessary only to specific browsers. However, Nexacro makes it easy: once you write a single source based on the CSS standards, the platform will automatically create CSS codes appropriate to each browser during the process of building an application.

```
<?xml version="1.0" encoding="utf-8"?>
<XCSS type="xcss1.0"><![CDATA[.Button
{
    background : #c0504d;
    -nexa-text-align : left;
}
]]></XCSS>
```

The above example XCSS codes include both a CSS standard property and a Nexacro-only property. In this case, the platform will produce CSS codes like the below one as a result of building an application.

```
.Button
{
    background        :        #c0504d;
}
.Button .nexacontentsbox
{
    text-align        :        left;
}
```



An XCSS file is formatted in XML. Moreover, while an XCSS file complies with the CSS standards, it does not mean that the file supports all the standard CSS properties.



An XCSS file can apply only to the application level.
If you want to style a whole project, you need to use a theme file. Meanwhile, apply an XCSS file if you want to style an application.

6.

Application Running Scenario

This section describes the process of launching a Nexacro app.

6.1 bootstrap

Bootstrap refers to the process of verifying related information and fetching necessary resources to load an application.

	WRE	NRE
❶	index.html	start.json start_android.json start_ios.json
❷	get bootstrap info	
❸		update engine
❹		update resource (data)
❺	load framework files	
❻	load component modules	
❼	load Application	



In general parlance, bootstrapping usually refers to a self-starting process that is supposed to proceed without external input. In computer technology the term (usually shortened to booting) usually refers to the process of loading the basic software into the memory of a computer.
https://en.wikipedia.org/wiki/Bootstrapping
In relation to Nexacro, bootstrap means a preparatory process for launching an app.

6.2 Application Loading

The process of loading the form connected to Application to run the app is as follows.

	Loading sequence	Event occurence
❶	Loading the environment file	
❷	Selecting a screen (when there are more than one screen)	
❸	Loading the information on the screen	
	- Checking Apps that use the ID of the selected screen as a value of the *screenid* property.	
	- Creating Application objects	
❹	Processing the environment file (TypeDefinition, properties, Variables)	
		Environment.onload
❺	Loading the theme file (css, *map.js* file)	
❻	Loading the Initvalue file	
❼	Loading the Application (**.xadl.js* file)	
	- Setting the properties of the Application	
		Application.onloadingappvariables
	- Creating and initializing a MainFrame	
		MainFrame.onactivate
	- Creating and initializing Frames	
		ChildFrame.onactivate
		Application.onload
❽	Loading Forms (**.xfdl.js* files)	
		Form.oninit
		Form.onload
		Form.onactivate
		Application.onloadforms



Because of the loading sequence, you cannot use objects that are not loaded yet.
For example, if you access AppVariables within the event handler of the *Environment.onload* event, "undefined" will be returned.

```
this.Environment_onload = function
(obj:nexacro.Environment,e:nexacro.LoadEventInfo)
{
    trace(nexacro.getApplication().all['Variable0']); //undefined
}
```

6.3 Form Loading

The below sequence is the elaboration of the process between *12. Loading Forms* and *20. Event occurrence: `Form.onload`*, as in the above Application Loading.

1. Loading a Form
2. Executing the Form
3. Downloading and executing sub-files
4. Initializing the Form
5. Setting style properties of the Form
6. Setting non-style properties of the Form
7. Creating objects, components and BindItems
8. Performing scripts that are processed inside the engine
9. Registering event handlers
10. Calling the methods of the objects and components: `Object.createComponent`
11. **Event occurrence**: `Form.oninit`
12. Method calling: `Form.on_created`
13. Calling the methods of the objects and components: `Object.on_created`
14. Loading images and processing transactions
15. **Event occurrence**: `Form.onload`



A subfile refers to a file that is connected through the include directive. (eg. a script file)



The objects and components included in a Form are created according to the specified Z-order.



The loading times for Div and Tab components may vary according to network speeds if those components are designed to load Forms connected through the *url* property.

7.

Nexacro Platform Script Language

You will apply basic JavaScript syntax when developing a Nexacro-based app. Nexacro offers its own APIs to provide additional features that cannot be achieved with JavaScript. Scripts written in Nexacro can be executed in web browsers and mobile devices directly after they undergo the build process.

This chapter explains the syntactic elements and other noteworthy points that you need to know when writing a script in Nexacro.

7.1 Scope

Scope explains issues related to the range of accessible variables. Apps used in business often manipulate a number of screens simultaneously while one variable can be referred to by multiple screens. Therefore, the accurate use of scope lets you exactly access the resources that you want.

7.1.1 Local

If you write the below code on the first line of a Form script, the variable will be given local scope. If this is standard JavaScript code, the scope of the variable will be global. However, Nexacro works differently.

```
var value0 = 0;
```

You can find the syntactic differences between standard JavaScript and Nexacro by seeing the below code, which is generated as a result of the build process and will apply to web browsers. The above code will be included in a automatically-created function, making the scope of the variable local.

```
this.registerScript("TEST.xfdl", function() {
    var value0 = 0;
});
```



If you want to get a global variable in a Form script, do not use the keyword `var` when you declare a variable. However, there is a possibility that a global variable can affect the entire app. Therefore, you are recommended to define appropriate scope.

You do not need to define scope when you declare a variable within a function. You also do not need to define scope for a value delivered as a parameter.

```
this.Button00_onclick = function(obj:nexacro.Button, e:nexacro.ClickEventInfo)
{
     obj.text = "button";
     var a = 3;
     trace(a); // 3
}
```

7.1.2 this

When designating the effective scope for items used in Application or the application form, you always need to use "this". You should designate a proper level of scope when referring to properties and Forms, as well as variables.

Variables within the Form are declared as follows:

```
this.formvalue = 4;
```

You need to designate the scope even when you declare a function. Moreover, you also need to designate the scope when you reference a variable in a function and the variable has been declared within the relevant Form.

```
this.formvalue = 4;

this.test = function()
{
     this.formvalue = 3;
     this.parent.value = 3;
     this.parent.parent.value = 3;
}
```

As for some methods like *trace* and *alert*, you can opt for either of the following types according to designated

scope: one follows the basic JavaScript syntax while the other is additionally provided by Nexacro. That is to say, calling the *alert* method will vary markedly depending on whether you do not define scope or you designate scope by specifying *this*.

```
this.Button00_onclick = function(obj:nexacro.Button, e:nexacro.ClickEventInfo)
{
    alert(e.button); // JavaScript
    this.alert(e.button); // Form.alert()
    nexacro.getApplication.alert(e.button); // Application.alert()
}
```

When declaring a function, you need to designate the scope so that it can be included as a member of a Form or an application.

```
this.Button00_onclick = function(obj:nexacro.Button, e:nexacro.ClickEventInfo)
{
...
}
```

When using the `eval` function to access a Form, `this` should be used as well.

```
eval("this.formfunc()");
```



As presented below, you can declare a function without designating scope. However, such a function will be used globally and may become unsupported later. Therefore, this way of function declaration is not recommended.

```
function Button00_onclick(obj:nexacro.Button, e:nexacro.ClickEventInfo)
{
...
}
```

7.1.3 Global

If you do not define scope of a variable or function in a script written in Nexacro, the variable or function will treated as a global member. The below code is an example.

```
globalvar = 2;
var globalvar2 = 3;
```

Even variables used in a function are treated as global when their scopes are not designated. However, variables defined by using the keyword `var` cannot be used outside a function.

```
this.test = function()
{
    trace(globalvar); //2
    trace(globalvar2); //3

    value = 4;
    var localvar = 5;
}


this.test = function()
{
    trace(value); // 4
    trace(localvar); // ReferenceError: localvar is not defined
}
```



When a Form includes a script file from outside, it will treat the script file as a function. Therefore, you should be careful when using the keyword `var`.

Please see the below two sample codes. Code 2 (*includecode.xjs.js*) shows how the variables defined in the *includecode.xjs* file are processed after generation. (The generated code may differ according to the version of the platform.)

includecode.xjs

```
bbb = "bbb";
var ccc = "ccc";
```

includecode.xjs.js

```
(function()
{
    return function(path)
    {
        var obj;
```

```
        // User Script
        this.registerScript(path, function() {
            bbb = "bbb";
            var ccc = "ccc";
        });

        this.loadIncludeScript(path);
        obj = null;
    };
}
)();
```

When you declare or call a function, you need to designate the associated scope. `Test()` and `this.test()` in the script below calls two different functions respectively. If you call `test()` without designating the scope, the function declared in global will be called.

```
this.Button00_onclick = function(obj:nexacro.Button, e:nexacro.ClickEventInfo)
{
     test();
     this.test();
}
```

The functions designated without the scope shown as below will be treated as global.

```
Button00_onclick = function(obj:nexacro.Button, e:nexacro.ClickEventInfo)
{
...
}

function Button00_onclick(obj:nexacro.Button, e:nexacro.ClickEventInfo)
{
...
}
```



You need to designate the scope of a method like *transaction()*, of which there are two variants: one belongs to the Application object and the other to the Form object.

```
//application
nexacro.getApplication().exit("");
```

```
nexacro.getApplication().transaction("");

//Form
this.close("");
this.go("");
this.transaction("");
```



If you want to use a certain variable in the scope of an application, the best practice is to register it in AppVariables instead of declaring it as a global variable.

```
nexacro.getApplication().addVariable("aaa", "TEST");
trace(nexacro.getApplication().aaa); // TEST
```

7.1.4 Expr

The *expr* script used in a Grid component can be applied without designating the scope as it is processed based on an internally bound Dataset within Nexacro. However, you should designate the scope to access an unbound Dataset directly.

To use `expr` in a bound Dataset shown, you do not need to use `this` as shown here.

```
<Cell text="expr:Column00"/>
<Cell text="expr:Column00.min"/>
<Cell text="expr:currow"/>
<Cell text="expr:rowposition"/>
<Cell text="expr:getSumNF('col0')"/>
```

```
Dataset.filterstr = "Column00=='test'";
Dataset.filter("Column00=='test'");
```

If you want to designate a Grid object, cell object or bound Dataset component as scope defined within an expr script, you need to use the below variables or selectors.

```
Grid: comp
Dataset: dataset
Cell: this
```

```
<Cell text="expr:this.col"/> <!-- cell -->
<Cell text="expr:dataset.rowcount"/> <!-- binded dataset -->
<Cell text="expr:comp.currentcell"/> <!-- grid -->
<Cell text="expr:dataset.parent.func1()"/> <!-- form -->
<Cell text="expr:comp.parent.func1()"/> <!-- form -->
```



There are no specific indicators for Forms, so you can use `comp.parent` or `dataset.parent` if necessary.



If you use `this` instead of `comp.parent.func()` when you access a function in a Form, it will be processed as an error. In an `expr` script, `this` means `cell`.
`<Cell text="expr:this.func01()"/>`

If a component has `expr` and `text` as its property at the same time, the `getDisplayText` property, which returns the text displayed on the screen, can be used.

```
this.Button00.text = "text";
this.Button00.expr = "1+1";

trace(this.Button00.text); // "text"
trace(this.Button00.expr); // "1+1"
trace(this.Button00.getDisplayText()); // "2"
```

7.1.5 lookup

The `lookup` method is designed to be used when it is difficult to access with the designated scope. For example, you can look up objects or functions with ID names or function names.

The `lookup` method is used in the actual code as follows.

```
Form.lookup( strid );
nexacro.getApplication().lookup( strid );
[Component].addEventHandlerLookup( eventid, funcid, target );
```

Here are additional examples.

```
// it returns an object relevant to objectid after lookup in this
var obj = this.lookup("objectid");

// it returns a function relevant to fn_onclick after lookup in this
this.lookup("fn_onclick")();

// Event handler handles a function relevant to fn_onclick after lookup in this
btn00.addEventHandlerLookup( "onclick", "fn_onclick", this );
```

7.2 Event Handler

Event handlers to deal with events can be added in the Properties window when using Nexacro Studio, or they can be added within scripts by using methods provided for each component and object as described in this section.

7.2.1 Method

Methods to add/set/delete event handlers are provided in order to handle events within scripts:

```
addEventHandler( eventid, funcobj, target )
addEventHandlerLookup( eventid, funcstr, target )
setEventHandler( eventid, funcobj, target )
setEventHandlerLookup( eventid, functstr, target )
removeEventHandler( eventid, funcobj, target )
removeEventHandlerLookup( eventid, funcstr, target )
```

Use appropriate methods to process events according to the scope of functions.

```
this.btn00.setEventHandler("onclick", this.fn_onclick, this);
this.dataset00.addEventHandlerLookup("onrowposchange", "fn_onchange", this);
```



It is recommended to use the *addEventHandlerLookup*, *setEventHandlerLookup* and *removeEventHandlerLookup* methods only when they are really necessary since they are likely to affect the performance of an app.

When you create an event through the Properties pane of Nexacro Studio, the event will be written in the form of a string as presented below.

```
<Button id="Button00" ... onclick="Button00_onclick"/>
```

The build process will convert the above code into the below JavaScript code by adding the selector *this*.

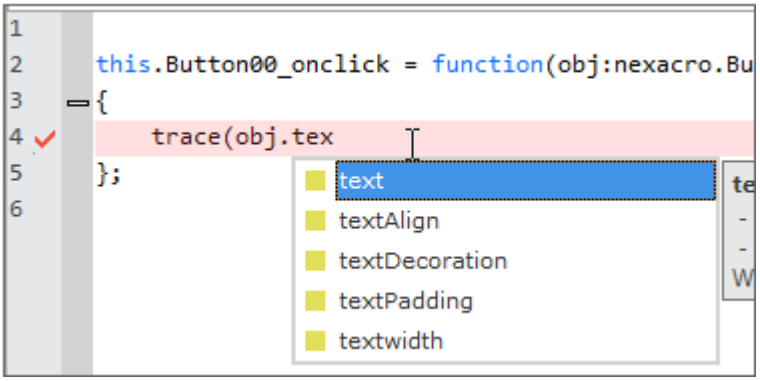
```
this.Button00.addEventHandler("onclick", this.Button00_onclick, this);
```

7.2.2 Object Type

You can set the object type when you set arguments to the event handler functions. Type is provided internally on Nexacro Studio for development convenience, which is not the JavaScript's standard grammar.

```
this.Button00_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    trace(obj.text);
}
```

Based on the input type value, Nexacro Studio supports the Intellisense function.



When you convert the relevant code to JavaScript code, delete the type value.

```
this.Button00_onclick = function(obj,  e)
{
    trace(obj.text);
}
```

Even though you do not designate the object type, it does not have any effect on executing the app.

The object type is not supported in other functions except the event handler function, where it can have an effect on the app's action.

7.3 Setting Property Values

7.3.1 General Properties

You can apply the text property to a button component as follows.

```
this.Button00.text = "text";
trace(this.Button00.text);
```

You can use the form of the set_ function used in previous versions as is.
You don't need to modify any scripts you've already created when you change the SDK version applied to the project.
(O) this.Button00.set_text("text");
(O) this.Button00.text = "text";

However, the set_ function may not be supported with respect to the new features.

To set the property value of an object that you've added yourself, use as follows:
<Button myprop="333">
Button00.myprop = "333";
Button00.mytext = "text";

7.3.2 Dynamic Properties

You can import property values using the existing method, but in case of objects whose property values change dynamically, a special get method is provided. For four properties in system objects - CursorX, CursorY, ScreenWidth, and ScreenHeight - you should use these additional methods:

```
nexacro.System.getCursorX();
nexacro.System.getCursorY();
nexacro.System.getScreenWidth();
nexacro.System.getScreenHeight();
```

Depending on component types, some properties which are not used cannot be adjusted. For example, if the calendar component type is not spin, you cannot adjust the properties below.

```
this.Calendar.spindownbutton
this.Calendar.spinupbutton
```



In the calendar component, the *spindownbutton* property refers the control property, and the property itself is output as a character string *[object ButtonControl]*.

7.4 Other Change Items

7.4.1 Nexacro Methods

Although Nexacro used to support methods not supported by ECMAScript by modifying them, it no longer does. Instead, such functions are now provided as Nexacro methods.

For example, the `floor`, `ceil` and `round` methods that supported two parameters in the math object are no longer available. If you want to use any of these methods, you must use the corresponding Nexacro methods.

```
//Math.floor( v, digit );
nexacro.floor( v, digit );

//Math.ceil( v, digit );
nexacro.ceil( v, digit );

//Math.round( v, digit );
nexacro.round( v, digit );
```

Some of object names used as reserved words in JavaScript have been changed.

```
//new Image();
new nexacro.Image();
```

Other component class names in *TypeDefinition* (not duplicated with reserved words) have been changed, such as *nexacro.Button*. However, you can use the button in the same way you did before.

```
this.button00 = new Button();
or
this.button00 = new nexacro.Button();
```



Some of the properties and objects have been changed as they collide with reserved words.

```
Component.class ?Component.cssclass
Export.export() ?Export.exportData()
VirtualFile.delete() ?VirtualFile.remove()
```

7.4.2 Operational Mode Change

Some of the items previously not supported by JavaScript, or working differently, have been changed.

<> Comparison operator not supported

<> Comparison operator is no longer supported. If you want to compare other values, use the operator "!=".

Character string processing method change within the switch statement

In the previous version, case "2" and case 2 were processed in the same way within the switch statement. In the present version, these are treated as different values.

Regular expression /g option applying method change

If you apply “replace” to the regular expression instead of the /g option, only one item will be changed. You need to apply the /g option to change entire items.

7.4.3 Restrictions on Object Names

You cannot create a ChildName identical to names of methods/properties or the container members. For example, you cannot designate a button component's ID as "text" since a Form has the text property. The button cannot be created in the following case.

```
<Form text="formtext">
     <Layouts>
          <Layout>
               <Button id='text'>
```

For invisible objects such as Datasets you cannot designate their IDs as property names, such as length, since they are treated as array members. In the following example, the columns cannot be created.

```
<Objects>
     <Dataset id="Dataset00">
          <ColumnInfo>
               <Column id="length" type="STRING" size="256"/>
          </ColumnInfo>
```

7.4.4 Restrictions on Variable or Function Names



If a variable or function includes an underbar (_) at the beginning its name, it can collide with the variables or functions registered in the Nexacro libraries. The collision can result in the failure to load the application or cause the app to behave differently from your intention.

For example, the below script will fail to be loaded, and thus nothing will appear on the screen. The reason is that a variable is declared with the name *_is_loading*, which equals the name of a variable in the Nexacro libraries.

```
this._is_loading = true;
this.Button00_onclick = function(obj:Button,  e:nexacro.ClickEventInfo)
{
    trace(this._is_loading);
}
```



Even if a variable or function does not include an underbar (_) in its name, it can collide with the variables or functions registered in the Nexacro libraries.

For example, the below script will fail to create a component, and thus nothing will appear on the screen. The reason is that a function is declared with the name *createComponent*, which equals the name of a function in the Nexacro libraries.

```
this.createComponent = function()
{
    trace('createComponent ');
}
```

7.4.5 Restrictions on External Modules with Window Object



You are sometimes required to register external libraries or JavaScript files as modules. If those modules include a window object, however, loading the relevant project may lead to an error in Nexacro Studio.

Nexacro Studio analyzes and processes the code of the modules when loading a project. If Studio finds a window object in this process, an error will occur because the development tool does not support the object. To avoid such an error, you need to add a branch at the top of the code block as presented below, causing Studio not to process a window object.

```
if( system.navigatorname != "nexacro DesignMode")
{
    if (typeof JSON !== 'object') {
        JSON = {};
    }

    (function () {
...
        })  (window)
    };
```

8.

Frame Tree

While you can make a simple single-form screen in Nexacro, you can also develop an type structure that can perform multiple associated tasks. Such a structure is so complicated that you need to learn how its constitutes interact with each other.

This chapter helps you understand such complicated structures by presenting several examples.

8.1 Application

```
// application -> mainframe
this.mainframe
this.id
this.all[n]
this.all['id']

// mainframe -> application
this.mainframe.parent

// mainframe -> childframe
this.mainframe.frame
this.mainframe.all[n]
this.mainframe.all['id']
this.mainframe.id

// childframe -> mainframe
this.mainframe.frame.parent
this.mainframe.frame.getOwnerFrame()

// childframe -> form
```

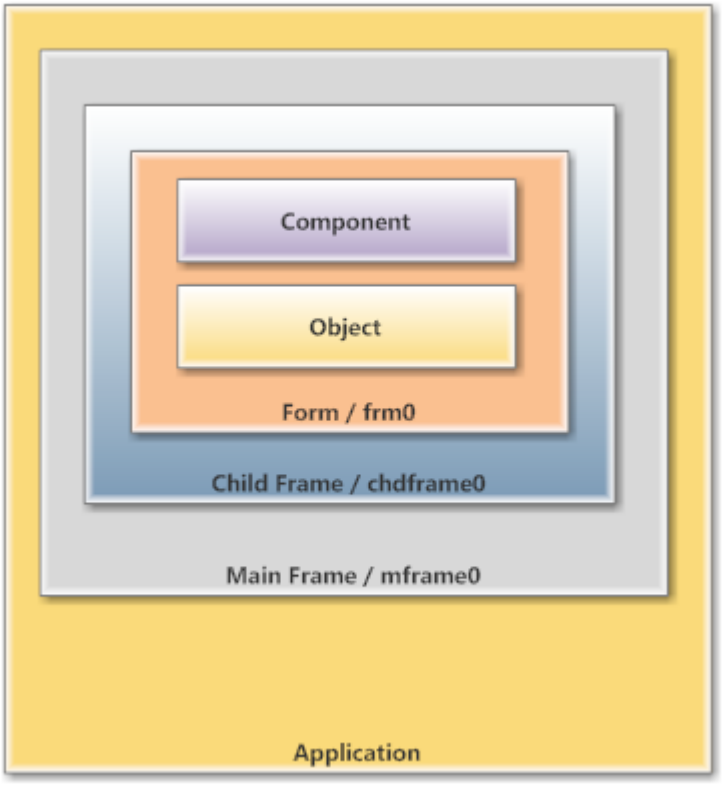
```
this.mainframe.frame.form

// form -> childframe
this.parent
this.getOwnerFrame()
nexacro.getApplication().mainframe.frame
```



The component/bind/invisible object in the Form does not have either `.getOwnerFrame()` nor `.getOwnerForm()`.

8.1.1 Script Example Screen



8.1.2 Accessing Form Object from Application

```
// Approach to frm0
this.mainframe.frame.form.name == "frm0"
this.mframe0.frame.form.name == "frm0"
this.mframe0.chdframe0.form.name == "frm0"
```

```
this.all[0].all[0].form.name == "frm0"
this.all["mframe0"].all["chdframe0"].form.name == "frm0"
```

8.1.3 Accessing Parent Objects from Form

```
// approach to application at frm0
this.parent.name == "chdframe0"
this.parent.parent.name == "mframe0"
this.getOwnerFrame().getOwnerFrame().name == "mframe0"
nexacro.getApplication().mainframe.name == "mframe0"
```

8.2 Form

A form has *one on N* structure, which implies that a single independent form interacts with its affiliated objects, binds and components. The below script codes are syntax examples of how to access the objects, binds and components in a Form.

```
// Form -> Object
this.all[n]
this.all['id']
this.id
this.objects[n]
this.objects['id']

// Form -> Bind
this.all[n]
this.all['id']
this.id
this.binds[n]
this.binds['id']

// Form -> Component
this.all[n]
this.all['id']
this.id
```

```
this.components[n]
this.components['id']

// Object, Bind, Component -> Bind
this.all[n].parent
```

You can connect a Form to a container component like a Div, PopupDiv and Tabpage through the `url` property. In this case, a one on one structure will apply between a container component and its connected form. The below script codes are syntax examples of how to access the objects, binds and components in the Form.

```
// Container Component -> Object
this.Div.form.all[n]
this.Div.form.all['id']
this.Div.form.id
this.Div.form.objects[n]
this.Div.form.objects['id']

// Container Component -> Bind
this.Div.form.all[n]
this.Div.form.all['id']
this.Div.form.id
this.Div.form.binds[n]
this.Div.form.binds['id']

// Container Component -> Component
this.Div.form.all[n]
this.Div.form.all['id']
this.Div.form.id
this.Div.form.components[n]
this.Div.form.components['id']

// Object, Bind, Component -> Container Component
this.Div.form.all[n].parent.parent
```

The `all`, `objects`, `binds`, and `components` properties are provided in the form of `nexacro.Collection`. That is to say, you can access each property with an index or ID in the Collection object.



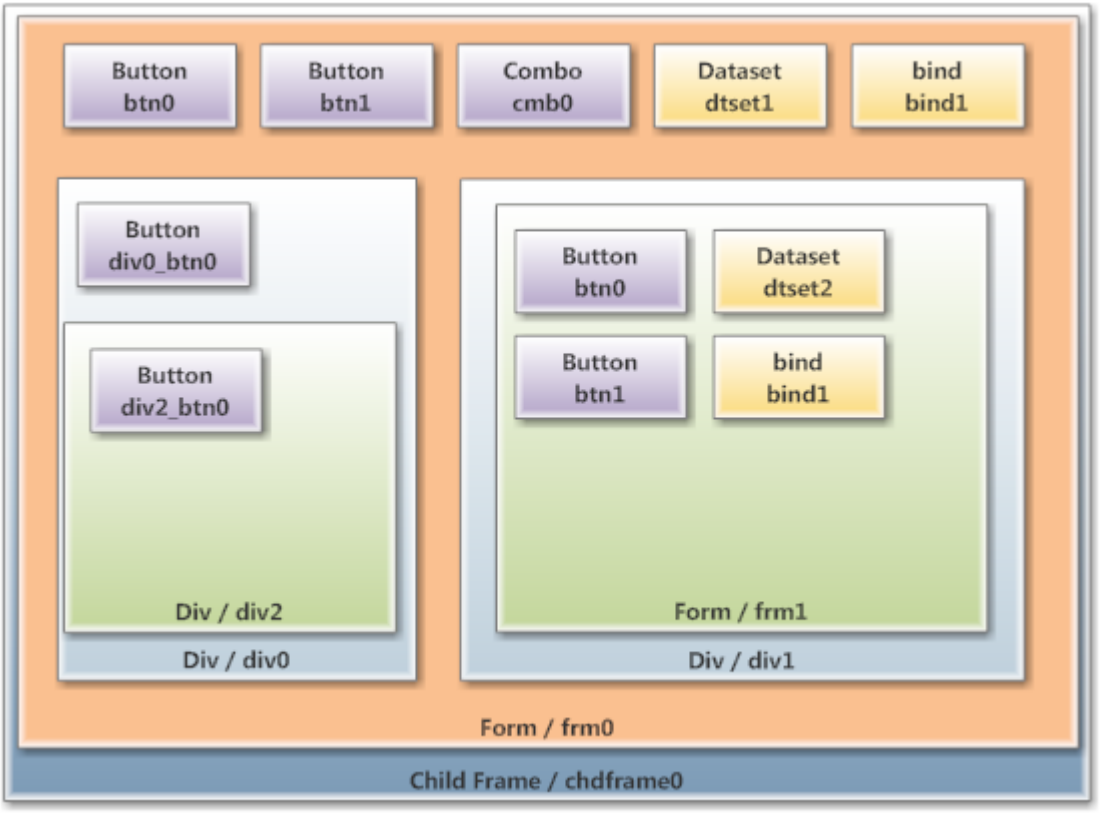
When you use the `all` property, the indexes within the Collection object are determined in the order described below.

1. Object

2. Component
3. Bind

Among the same types, the order of indexes follows the sequence of the source codes.

8.2.1 Script Example Screen



Index order is assumed to be in the order of dtset1, btn0, btn1, cmb0, div0, div1 and bind1. In general Forms, the index order begins with Dataset, followed by remaining components as they are assigned and bind objects are at the end.

In a Form on the same level, the ID cannot be duplicated. In other words, since btn0 and div0_btn0 and div2_btn are not the same levels, they may be designated as the same ID.

8.2.2 Components / Objects / Bind

```
this.name == "frm0"
```

```
// Access to btn0
this.all["btn0"].name == "btn0"
this.all[1].name == "btn0"
this.btn0.name == "btn0"
this.components[0].name == "btn0"
this.components["btn0"].name == "btn0"
```

```
// Access to dtset1
this.all["dtset1"].name == "dtset1"
this.all[0].name == "dtset1"
this.dtset1.name == "dtset1"
this.objects[0].name == "dtset1"
this.objects["dtset1"].name == "dtset1"
```

```
// Number of components / invisible objects / binds
this.all.length == 7
this.components.length == 5 //btn0, btn1, comb0, div0, div1
this.binds.length == 1
```

8.2.3 Container Components

A container component without a connection to a Form can dynamically have invisible objects / bind.

```
// Access to div0_btn0 inside div0
this.div0.form.div0_btn0.name == "div0_btn0"
this.div0.form.all[0].name == "div0_btn0"
this.components[3].form.components[0].name == "div0_btn0"
```

```
// Access to div2_btn0 inside div2
this.div0.form.div2.form.div2_btn0.name == "div2_btn0"
this.all["div0"].form.all["div2"].form.all["div2_btn0"].name == "div2_btn0"
this.components["div0"].form.components["div2"].form.components["div2_btn0"
```

```
].name == "div2_btn0"
```

8.2.4 Container Component Connected to Form

```
// (in case of the script in frm1) Access to btn0 in frm1
this.parent.name == "div1"
this.btn0.name == "btn0"
this.all["btn0"].name == "btn0"
```

```
// (in case of the script in frm0) Access to btn0 in frm1
this.name == "frm0"
this.div1.form.btn0.name == "btn0"
```

8.2.5 Parents

```
this.div1.form.dtset2.parent.parent.name == "form" // it is not frm0
this.div1.form.dtset2.parent.name == "div1"
```

Part III.

Supplements

9.

Cache of App

To use the network effectively, rather than downloading the developed application program from the distribution server every time, cache technique is used.

9.1 Types of Caches

Types of the supported caches are as follows.

Type of Cache	Description
none	The cache feature is not used.
dynamic	The file is reloaded only when it is updated on the server. If the file is not updated on the server, the local cache file is reused.
session	When the app is running, the file is fetched from the server and only the local cache file is used until the app is closed. (If the file on the server is the same as the local cache file during app execution, the cached file is reused.)
static	Once a file is downloaded from the server, the local cache file is used even when the app is restarted. (However, if the version information of the service group specified in the Type Definition changes, the file is reloaded.)



Notes for WRE Execution

- If the property value is set to static, cache files are saved in local storage. However, due to the limitations of web browser functionality, local storage cannot be used. Therefore, when used in a web browser, the property value operates as session even if it is set to static.
- When the web browser is restarted or refreshed, all memory-based cache managed by the

product is cleared. As a result, regardless of whether the property value is set to session, dynamic, or static, the behavior depends on the browser.

- When used in a web browser, it is normal for the browser not to use the cache if the property value is set to none. - When using Firefox, there may be an issue where the browser cache is used, which is a bug in Firefox.
  https://bugzilla.mozilla.org/show_bug.cgi?id=1129500



Notes for Execution in Nexacro Studio

- When running NRE using the [Launch Project] or [Quick View] menu in Nexacro Studio, the Service-cachelevel value set in typedefinition is ignored and operates as none.
  (The configured value of the cachelevel property is not changed.)

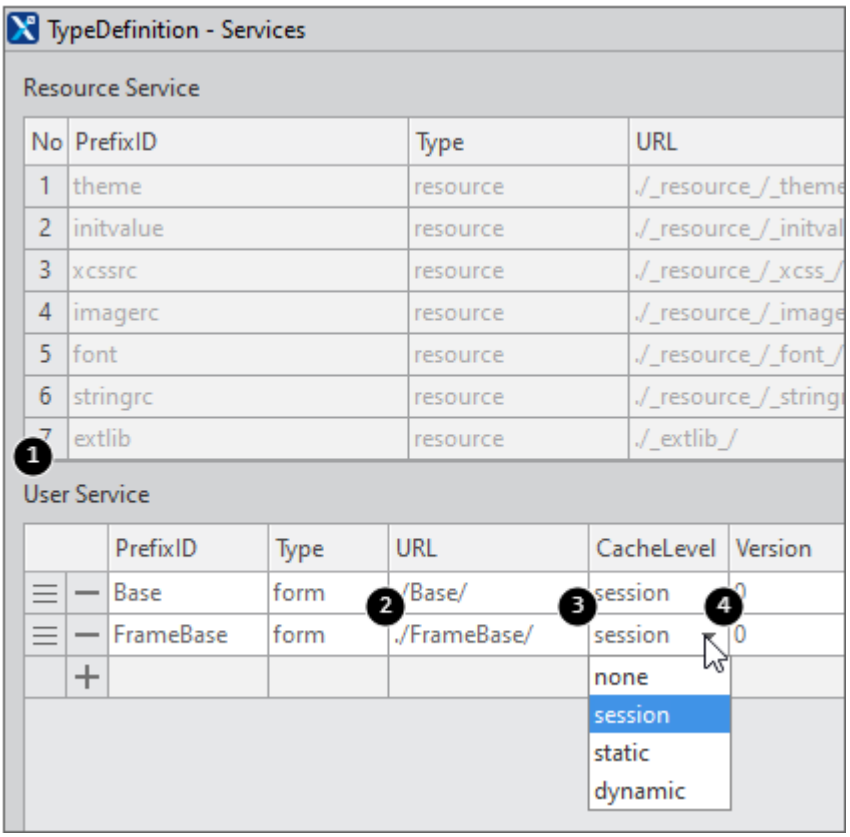


Differences in cachelevel application depending on the component or object.

- When processing images in Nexacro, the behavior is designed to follow the cache settings of the browser used, considering image rendering-related features (e.g., size, stretch) and performance reasons.
  (This behavior is independent of the cachelevel set in the project service group.)
- When importing or exporting Excel files using ExcelImportObject or ExcelExportObject, the behavior always operates as none regardless of the specified cachelevel property value.

9.2 Cache Application Method

The cache setting can be modified using our tool and will be saved in the TypeDefinition file, as shown in the below picture presenting the TypeDefinition editor.



Callout	Description
❶	Cache is only applied to the service group registered here
❷	You can see here that a cache is applied here to all files in the directory called “./Base/”
❸	Select the type of a cache
❹	When the cache type is set to "static," you can download files again by re-setting the version of the service group.

```
nexacro.getEnvironment().services["Base"].cachelevel = "none";
```

10.

Dataset XML Format

The Nexacro provides X-API as a server module. Using this Nexacro API, NRE interprets the created Dataset format and displays apps on the UI screen.

This chapter explains the Dataset format created by X-API. XML files abiding by this format can be acknowledged as applicable data on the Nexacro Runtime. It is possible to develop server programs without using X-API.

10.1 Dataset XML Layout

XML Declaration					
Root					
	Cache				
	Parameters				
		Parameter (repeat)			
	Dataset (repeat)				
		ColumnInfo			
			ConstColumn (repeat)		
			Column (repeat)		
		Rows			
			Row (repeat)		
				Col (repeat)	
				OrgRow	
					Col (repeat)

10.1.1 XML Declaration

The XML declaration is defined as follows.

```
<?xml version="1.0" encoding="utf-8"?>
```

The XML declaration should be placed at the beginning of an xml document. There should be no letter, including blanks, before the declaration.

10.1.2 XML Example

```
<?xml version="1.0" encoding="utf-8"?>
<Root xmlns="http://www.nexacroplatform.com/platform/dataset" ver="4000" >
     <Parameters>
          <Parameter id="service">stock</Parameter>
          <Parameter id="method">search</Parameter>
     </Parameters>
     <Dataset id="output">
          <ColumnInfo>
               <ConstColumn id="market" size="10" type="STRING" value="kse"/>
               <ConstColumn id="openprice" size="10" type="INT"
               value="15000"/>
               <Column id="stockCode" size="5" type="STRING"/>
               <Column id="currentprice" size="10" type="INT"/>
          </ColumnInfo>
          <Rows>
               <Row>
                    <Col id="currentCode">10001</Col>
                    <Col id="currentprice">5700</Col>
               </Row>
               <Row>
                    <Col id="currentCode">10002</Col>
                    <Col id="currentprice">14500</Col>
               </Row>
          </Rows>
     </Dataset>
</Root>
```

10.2 Dataset Elements

The Dataset elements are described here.

10.2.1 Root

Outline

The top-level element that indicates a Dataset.

Child element

Parameters, Dataset

Repeat

One

Property

```
<Root xmlns="http://www.nexacroplatform.com/platform/dataset">
<Root xmlns:nexacro="http://www.nexacroplatform.com/platform/dataset">
<Root xmlns=".." version="1000">
```

Property name	Description
xmlns	Name space, if not using a basic name space then show as xmlns:nexacro
ver	Clarifies Dataset layout version.

10.2.2 Parameters

Outline

An element clarifying the parameters needed for communication. The "child parameter" has the actual value, and the "parameters" element is an assemblage of (child) parameters .

Child element

Parameter

Repeat

None or one

10.2.3 Parameters > Parameter

Outline

Clarifies the value of a parameter

Child element

None

Repeat

None or as many as the number of parameters

Property

```
<Parameter id="ErrorMsg" type="STRING">SUCC</ Parameter >
```

Property Name	Description
id	Parameter name
type	Parameter type

Note

The Nexacro uses the required parameters described below. You need to set parameters for the interconnection with Nexacro.

```
<Parameter id="ErrorCode">1</Parameter>
<Parameter id="ErrorMsg">SUCC</Parameter>
<Parameter id="cachetype">Session</Parameter>
```

ID

ID	Description
ErrorCode	Error Code ● When calling the transaction function: ○ In the case it is less than 0, which is a user defined-error, it will be processed as "fail" without reflecting UpdateStatus into InputDataset ○ In the case it is greater than 0, which is a user defined-normal status, it will be processed as "sucess" as reflecting InputDataset into UpdateStatus

ID	Description
	● When the load function is called: ○ In the case it is less than 0, it is processed as fail defined by the user. ○ In the case it is greater than 0, it is is processed as success as normal defined by the user. ● In case not designated : 0
ErrorMsg	Error Message When ErrorCode is not specified or 0, it is success. Otherwise, fail
CacheType	Defines the cache method type: ● Type is dynamic, static or none ● In case not designated : Session

Type

The data type can be one of the following.

- STRING: Regardless of the column size, there is no limit to the width. (However it is limited to about 2GB since the system can allot 2GB at a time)
  - In the case of an empty string
    <Parameter id="ErrorMsg" type=”STRING”></Parameter>
  - In the case of null
    Is divided by <Parameter id="ErrorMsg" type=”STRING”/>
- INT: Integer type
- FLOAT: Float type
- BIGDECIMAL: Object Type(nexacro.Decimal)
  Floating-point type (supports up to 31 digits in decimal format, with a maximum of 24 digits for the integer part and 15 digits for the fractional part).
- DATE: Type(nexacro.Date)
  YYYYMMDD (0000/01/01 ~ 9999/12/31)
- DATETIME : Type(nexacro.Date)
  YYYYMMDDHHmmssuuu (msec can be included)
- TIME : Type(nexacro.Date)
  6 digit HHmmssuuu(msec can be included)
- BLOB: Binary data

10.2.4 Dataset

Outline

Element that has the value of one Dataset. A Dataset has *ColumnInfo* which defines its structure and *Rows* which has the actual data as child elements.

Child element

ColumnInfo, Rows

Repeat

None or as many as the number of Dataset XML passes

Property

```
<Dataset id="History">
```

Property Name	Description
id	Dataset name

10.2.5 Dataset > ColumnInfo

Outline

It is used to express the Dataset schema. Child elements—ConstColumns and Columns—show the actual data. ColumnInfo is an assemblage of ConstColumns and Columns. ConstColumns show the schema of the columns with fixed values. Columns show the schema of the columns with non-fixed values.

Child element

ConstColumn, Column

Repeat

None or one



As for the child elements of ColumnInfo, ConstColumns must come before Columns.
If ConstColumns and Columns are arranged out of order, the app may not be able to process the data.

```
(X) When ConstColumns do not come first
<ColumnInfo>
    <Column id="stockCode" size="5" type="STRING"/>
    <Column id="currentprice" size="10" type="INT"/>
    <ConstColumn id="market" size="10" type="STRING" value="kse"/>
    <ConstColumn id="openprice" size="10" type="INT" value="15000"/>
</ColumnInfo>
```

```
...
```

```
(X) When ConstColumns and Columns are jumbled
<ColumnInfo>
    <ConstColumn id="market" size="10" type="STRING" value="kse"/>
    <Column id="currentprice" size="10" type="INT"/>
    <ConstColumn id="openprice" size="10" type="INT" value="15000"/>
    <Column id="stockCode" size="5" type="STRING"/>
</ColumnInfo>
...
```

10.2.6 Dataset > ColumnInfo > ConstColumn

Outline

Expresses the schema as *Column* that always has a fixed value.

Child element

None

Repeat

None or as many as the number of fixed value column

Property

```
<ConstColumn id="systemName"
     size="255"
     type="STRING"
     value="Production"/>
```

Property Name	Description
id	Column name
size	Maximum data size
type	Data type of column
value	Fixed value of column

Note

Following are the data type that can come in type.

- STRING: Regardless of the column size there is no limit to the width. (However is limited to around 2GB

which is the size the system can allot at once)

- INT: Integer type
- FLOAT: Float type
- BIGDECIMAL: Object Type(nexacro.Decimal)
  Floating-point type (supports up to 31 digits in decimal format, with a maximum of 24 digits for the integer part and 15 digits for the fractional part).
- DATE: Type(nexacro.Date)
  YYYYMMDD (0000/01/01 ~ 9999/12/31)
- DATETIME : Type(nexacro.Date)
  YYYYMMDDHHmmssuuu (msec can be included)
- TIME : Type(nexacro.Date)
  6 digit HHmmssuuu(msec can be included)
- BLOB: Binary data

10.2.7 Dataset > ColumnInfo > Column

Outline

It clarifies the value of a parameter

Child element

None

Repeat

None or as many as the number of non-fixed value columns

Property

```
<Column id="department"
     size="255"
     type="STRING"
     prop="text"
     sumtext="total"/>
```

Property Name	Description
id	Column name
size	Maximum data size
type	Data type of the column
Prop	Defines the summary type
Sumtext	Character string when prop value is text

Note

The value set in the *type* will be the same as the type in *ColumnInfo*.

The *prop* has the following values.

- count: The number of rows of the column
- sum: The sum of row values of the column
- max: The maximum value of the rows in the column
- min: The minimum value of the rows of the column
- avg: The average of the row value of the column
- Text: Sumtext property value

10.2.8 Dataset > Rows

Outline

It is an assemblage of the rows of a Dataset. It has a row that shows the actual data as its child element.

Child element

Row

Repeat

None or one

10.2.9 Dataset > Rows > Row

Outline

It expresses the data of a row in the Dataset. It has *Col* as its child element, which expresses value of each column. When the type is "update", it has another child element *Org_Row* that has the original values of a row.

Child element

Col, Org_Row

Repeat

None or as many as the number of rows of Dataset

Property

```
<Row type="insert">
     <Col id="currentCode">10001</Col>
     <Col id="currentprice">13400</Col>
</Row>

<Rows>
     <Row type="update">
          <Col id="currentCode">10001</Col>
          <Col id="currentprice">13400</Col>
          <OrgRow>
               <Col id="currentCode">10001</Col>
               <Col id="currentprice">13700</Col>
          </OrgRow>
     </Row>
</Rows>

<Row type="delete">
     <Col id="currentCode">10001</Col>
     <Col id="currentprice">13400</Col>
</Row>
```

Property Name	Description
type	insert : A row added to the original Dataset
	update: A row modified in the original Dataset. It includes Org_Row as its child element. Org_Row is the original row.
	delete: A row deleted from the original Dataset

10.2.10 Dataset > Rows > Row > Col

Outline

It expresses the value of each column of a Dataset

Child element

None.

Repeat

None or as many as the number of columns

Property

```
<Col id="department">management</Col>
```

Property Name	Description
Id	Column name. It Is the same as the name set in the child elements *(ConstColumn, Column)* of *ColumnInfo*

Note

In the case of an empty string:

```
<Col id="department"></Col>
or
<Col id="department"/>
```

In the case of null, it is classified as without "tag".

10.2.11 Dataset > Rows > Row > OrgRow

Outline

It has the original value when the value of Dataset's row is modified. It has Col which has the actual value as its child element.

Child element

Col

Repeat

None or one.

Note

Similar to Row which is its parent element but has no property

10.2.12 Dataset > Rows > Row > OrgRow > Col

Outline

It expresses the original column value before the value is modified

Child element

None

Repeat

None or as many as the number of columns

Property

```
<Col id="department">management</Col>
```

Property Name	Description
id	Column name. It is the same as the name set in the child elements *(ConstColumn, Column)* of *ColumnInfo*

11.

Dataset SSV Format

This describes the layout for processing the Dataset object data in the SSV (Space-Separated Values) form.



Key terminology here are abbreviated as follows.

- (RS): RS(Record Separator) / 0x1E(30)
- (US): US(Unit Separator) / 0x1F(31)



Depending on the operating environment, (RS), (US) can be changed to other codes in the Nexacro Studio.

- Environment.ssvrecordseparator
- Environment.ssvunitseparator

11.1 Dataset SSV layout

Dataset SSV is configured in the following format.

Item	Required
Stream Header	O
Variables	X
Datasets	X

11.1.1 Stream Header

```
SSV{:CodePage}(RS)
```

The Stream Header starts with "SSV", and the CodePage value can be added after the colon (:). It is distinguished from the following items (Variables or Datasets) with (RS).

Item	Required	Description
SSV	O	It is a value fixed with "SSV".
CodePage	X	It starts with the ":" character and is the encoding information.

```
SSV(RS)
SSV:en_US(RS)
SSV:utf-8(RS)
```

11.1.2 Variables

You can omit or have more than 1 Variable. n number of Variables are distinguished with (RS), and it is divided by (RS) and Datasets.

Variable

```
Variable ID{:Type(Length)}{=Value}(RS)
Variable ID{:Type}{=Value}(RS)
```

Item	Required	Description
Variable ID	O	When a duplicate value comes in as a unique ID, only the last value is processed.
Type	X	It starts with the ":" character and can contain the Length information. When omitted, it is processed as "STRING".
Length	X	It is processed as a value between "("character and ")" characters. When omitted, it is processed as a threshold according to Type. However, when Type is "STRING", it is processed as 255.
Value	X	It starts with the "=" character and is processed as a Value until (RS) comes out.

```
name1=value(RS)
name1=value1(RS)name2=value2(RS)
name1:STRING=value1(RS)name2=value2(RS)
```

```
name3=value3(RS)name4:STRING(10)=value4(RS)
```

11.1.3 Datasets

You can omit or have more than 1 Dataset.

Dataset

A Dataset has the following format.

Item	Required
Dataset Header	O
Const Column Infos	X
Column Infos	O
Records	X
Null Record	O



When you have Const Column Infos, it must be listed before Column Info.
If the order is changed or the Const Column Infos information is listed again after Column Info, then the normal operation is not guaranteed.

11.2 Dataset Format Details

This describes each item of the Dataset.

11.2.1 Dataset Header

```
Dataset:Dataset ID(RS)
```

Dataset Header starts with "Dataset" and the Dataset ID is written after the colon (:). It is distinguished from the following items with (RS).

11.2.2 Const Column Infos

```
_Const_(US)Const Column ID{:Type(Length)}{=Value}(RS)
_Const_(US)Const Column ID{:Type}{=Value}(RS)
```

It starts with "_Const_" and can have more than 1 Const Column Info. n number of Const Column Info is distinguished with (US), and it is distinguished from the following items with (RS).

Item	Required	Description
Const	O	It is a value fixed with "_Const_".
Const Column ID	O	It is the ID value to be used as Const Column ID in Dataset.
Type	X	It starts with the ":" character and can contain the Length information. When omitted, it is processed as "STRING".
Length	X	It is processed as a value between "("character and ")" characters. When omitted, it is processed as a threshold according to Type. However, when the Type is "STRING", it is processed as 255.
Value	X	It starts with the "=" character and is processed as Value until (US) or (RS) comes out.

11.2.3 Column Infos

```
_RowType_(US)Column ID{:Type(Length)}{:Sum Type}{:Sum Text}(RS)
_RowType_(US)Column ID{:Type}{:Sum Type}{:Sum Text}(RS)
```

It starts with "_RowType_" and can have more than 1 Column Info. n number of Column Info is distinguished with (US), and it is distinguished from the following items with (RS).

Item	Required	Description
RowType	O	It is a value fixed with "_RowType_"
Column ID	O	It is the ID value to be used as Column ID in Dataset.
Type	X	It starts with the ":" character and can contain the Length information. When omitted, it is processed as "STRING".
Length	X	It is processed as a value between the "("character and the ")" character. When omitted, it is processed as a threshold according to Type. However, when the Type is "STRING", it is processed as 255.
Sum Type	X	It sets the prop property value of the ColumnInfo object with the ":" character.
Sum Text	X	It sets the sumtext property value of the ColumnInfo object with the ":" character.

11.2.4 Records

```
RowType(US)Column Value(RS)
```

The first field value is RowType, and the subsequent field values are written in the Column order set in Column Infos. Each field value is distinguished with (US).

Item	Required	Description
RowType	O	It writes the record type. N: Normal Record I: Inserted Record U: Updated Record D: Deleted Record O: Original Record (This is the original record of the Update Record. It can be used if there is an Update Record).
Column Value	O	This is the value to be used as the Column Value in the Dataset. If you want to process the Column Value as undefined, then use the "ETX (End of Text) / 0x03(03)".

11.2.5 Null Record

Distinguish the end of the Dataset with (RS).

11.3 References

11.3.1 Type

The values that can be used in the Type item in Variable, Const Column Infos, Column Infos are as follows.

Type	Description
STRING	Character string. The maximum value is limited to 2GB, which is the size that can be allocated at one time in the system. It can be used up to the maximum value, regardless of the size of the Column.
INT	Integer type
FLOAT, DECIMAL	Float type
BIGDECIMAL	Object Type(nexacro.Decimal)

Type	Description
	Floating-point type (supports up to 31 digits in decimal format, with a maximum of 24 digits for the integer part and 15 digits for the fractional part).
DATE	Object Type(nexacro.Date) YYYYMMDD (0000/01/01 ~ 9999/12/31)
DATETIME	Object Type(nexacro.Date) YYYYMMDDHHmmssuuu (msec can be included)
TIME	Object Type(nexacro.Date) 6 digit HHmmssuuu(msec can be included)
BLOB	Binary data

11.3.2 Examples



- (RS), (US) are indicated by the symbols below.
  ▼ = Record Separator(RS)
  • = Unit Separator(US)
- Line breaks are added for easy reading. In the actual SSV configuration, no line breaks are added.

When There is 1 Dataset

```
SSV:utf-8?
Dataset:dataset0?
_RowType_•Col1:String(20)•Col2:Int:SUM•Col3:Decimal:AVG?
N•Test•0•1•1?
I•Abc•1•2•2?
U•Def•2•3•3?
O•Chk•2•3•3?
D•Ghi•3•4•4?
?
```

When There is 1 Dataset with a Const Column

```
SSV:utf-8?
Dataset:dataset0?
_Const_•ConstCol1:STRING(20)=Name•ConstCol2:INT=1•ConstCol3:DECIMAL=0.8?
_RowType_•Col1:STRING(20)•Col2:INT:SUM•Col3:DECIMAL:AVG?
N•Test•0•1•1?
```

```
I•Abc•1•2•2?
U•Def•2•3•3?
O•Chk•2•3•3?
D•Ghi•3•4•4?
?
```

When There are Multiple Datasets

```
SSV:utf-8?
Dataset:dataset0?
_Const_•ConstCol1:STRING(20)=Name•ConstCol2:INT=1•ConstCol3:DECIMAL=0.8?
_RowType_•Col1:STRING(20)•Col2:INT:SUM•Col3:DECIMAL:AVG?
N•Test•0•1•1?
I•Abc•1•2•2?
U•Def•2•3•3?
O•Chk•2•3•3?
D•Ghi•3•4•4?
?
Dataset:dataset1?
_RowType_•Col1:INT(4):Summ,Col2:STRING(30):Text?
N•0•test?
I•1•test1?
U•2•test3?
O•2•test3-1?
D•3•test4?
?
```

When There is Only a Variable

```
SSV:utf-8?
Var4=10?Var5=20?
?
```

When There exist both Variable & Dataset

```
SSV:utf-8?
Var4=10?Var5=20?
Dataset:dataset0?
_Const_•ConstCol1:STRING(20)=Name•ConstCol2:INT=1•ConstCol3:DECIMAL=0.8?
_RowType_•Col1:STRING(20)•Col2:INT:SUM•Col3:DECIMAL:AVG?
```

```
N•0•test?
I•1•test1?
U•2•test3?
O•2•test3-1?
D•3•test4?
?
```

12.

Dataset JSON Format

Describe the layout for processing Dataset object data in JavaScript Object Notation (JSON) format.

12.1 Dataset JSON layout

The Dataset JSON is composed in the following format:

Item	Required
version	O
Parameters	X
Datasets	X

12.1.1 version

Specify the version of the Dataset Layout.

```
"version" : "1.0",
```

12.1.2 Parameters

Specify the parameter values.

```
"Parameters":
[
    {"id": "ErrorCode", "value":0},
    {"id": "ErrorMsg", "value":""},
```

```
    {"id": "param1", "value":0},
    {"id": "param2", "value":"0", "type":"string"}
]
```

Item	Required	Description
id	O	"Set the parameter name. ""ErrorCode"" and ""ErrorMsg"" are reserved keywords. If the ""ErrorCode"" field is not specified, the value will be treated as 0. If the ""ErrorMsg"" field is not specified, the ""ErrorCode"" value will be treated as ""SUCCESS"" if it is 0, or ""FAILED:"" otherwise."
value	X	Integer, Float, String
type	X	Set the data type. Refer to the typefield.

- "* ""ErrorCode"" and ""ErrorMsg"" are reserved keywords.
- If the ""ErrorCode"" field is not specified, the value will be treated as 0.
- If the ""ErrorMsg"" field is not specified, the ""ErrorCode"" value will be treated as ""SUCCESS"" if it is 0, or ""FAILED:"" otherwise."

12.1.3 Datasets

A Dataset is an element that includes Dataset object information, and it contains datasets in the form of objects in the arrangement.

Dataset

This is an element that contains information on one Dataset object and includes the ColumnInfo to represent the structure of the Dataset object and rows that contain the actual data of the Dataset object.

Item		Required
id		O
ColumnInfo		O
	ConstColumn	X
	Column	O
Rows		O

12.2 Dataset format details

Describe each item in the Dataset.

12.2.1 id

```
"id":"inData"
```

Item	Required	Description
id	O	This is the Dataset object IDd property value.

12.2.2 ColumnInfo > ConstColumn

```
"ColumnInfo":
{
    "ConstColumn":
    [
        {"id":"ConstCol1", "value":10},
        {"id":"ConstCol2", "type":"string", "size":"256", "value":10},
        {"id":"ConstCol3"}
    ]
}
```

Item	Required	Description
id	O	This is the ID value to be used as the Const Column ID in the Dataset object.
type	X	Set the data type. Refer to the type field.
size	X	"Set the maximum data size. If omitted, it will be treated as a threshold based on the type. However, if the type is ""STRING,"" it will be treated as 255."
value	X	"Set a fixed value. If it is not set, it will be treated as an undefined value."

12.2.3 ColumnInfo > Column

```
"ColumnInfo":
{
    "Column" :
    [
        {"id":"Column0"},
        {"id":"Column1", "type":"string", "size":"256"},
        {"id":"Column2", "type":"string", "size":"256"}
    ]
}
```

Item	Required	Description
id	O	This is the ID value to be used as the Column ID in the Dataset.
type	X	Set the data type. If omitted, it will be treated as "STRING." Refer to the type field.
size	X	"Set the maximum data size. If omitted, it will be treated as a threshold based on the type. However, if the type is ""STRING,"" it will be treated as 255."
prop	X	Sets the value of the prop property of the ColumnInfo object.
sumtext	X	Sets the value of the sumtext property of the ColumnInfo object.

12.2.4 Rows

```
"Rows":
[
    {"_RowType_":"U", "Column0":"", "Column1":"zzz", "Column2":""},
    {"_RowType_":"O", "Column0":"", "Column2":""},
    {"_RowType_":"N", "Column0":"A", "Column1":"B", "Column2":""},
    {"_RowType_":"D", "Column0":"a", "Column1":"b", "Column2":"c"},
    {"_RowType_":"I", "Column0":"", "Column1":"", "Column2":""}
]

"Rows":
[
    {"Column0":"A", "Column1":"B"},
    {"Column0":"a", "Column1":"b", "Column2":"c"},
```

```
    {"Column0":"", "Column1":"", "Column2":""}
]
```

Item	Required	Description
RowType	X	Set the record type. N: Normal Record (Default) I: Inserted Record U: Updated Record D: Deleted Record O: Original Record (This is the original record of the Update Record)
<ConstColumn id>	X	Set the ConstColumn value.
<Column id>	O	Set the Column value.

- "* There is meaning to the order of the row data.
- If the _RowType_ property value is ""O,"" for example, it means that it is the original row of the immediately preceding row.
  It will only work if the value of the _RowType_ property of the immediately preceding row is ""U.""
- If the value of the _RowType_ property is ""O"" and the value of the _RowType_ property of the immediately preceding row is not ""U,"" the row will be ignored. Also, if the value of the _RowType_ property of the first row is ""O,"" the row will be ignored."

12.3 Note

12.3.1 type

The following values can be used for the type fields in Parameters, ConstColumn, and Column.



If Parameters and ConstColumn are not set, the type will be treated as "INT" and "FLOAT" in the case of Integer and Float, depending on the value, and as "STRING" in other cases.

Type	Description
STRING	The string is limited to 2 GB in size, which is the size that the system can allocate at once. The maximum size can be used regardless of the size of the column.
INT	Integer type
FLOAT, DECIMAL	Float type

BIGDECIMAL	Object Type(nexacro.Decimal) Floating-point type (supports up to 31 digits in decimal format, with a maximum of 24 digits for the integer part and 15 digits for the fractional part).
DATE	Object Type(nexacro.Date) YYYYMMDD (0000/01/01 ~ 9999/12/31)
DATETIME	Object Type(nexacro.Date) YYYYMMDDHHmmssuuu (msec can be included)
TIME	Object Type(nexacro.Date) 6 digit HHmmssuuu(msec can be included)
BLOB	Binary data

12.3.2 JSON example

```
{
    "version" : "1.0",
    "Parameters":
    [
        {"id":"ErrorCode", "value":0},
        {"id":"ErrorMsg", "value":""},
        {"id":"param1", "value":0},
        {"id":"param2", "value":"0", "type":"string"}
    ],
    "Datasets" :
    [
        {
            "id":"indata",
            "ColumnInfo":
            {
                "ConstColumn":
                [
                    {"id":"ConstCol1", "value":10},
                    {"id":"ConstCol2", "type":"string", "size":"256", "value":
10},
                    {"id":"ConstCol3"}
                ],
                "Column" :
                [
                    {"id":"Column0"},
                    {"id":"Column1", "type":"string", "size":"256"},
                    {"id":"Column2", "type":"string", "size":"256"}
```

```
                ]
            },
            "Rows":
            [
                {"_RowType_":"U", "Column0":"", "Column1":"zzz", "Column2":""},
                {"_RowType_":"O", "Column0":"", "Column2":""},
                {"_RowType_":"N", "Column0":"A", "Column1":"B", "Column2":""},
                {"_RowType_":"D", "Column0":"a", "Column1":"b", "Column2":"c"},
                {"_RowType_":"I", "Column0":"", "Column1":"", "Column2":""}
            ]
        },
        {
            "id": "indata2",
            "ColumnInfo":
            {
                "Column":
                [
                    {"id":"Column0"},
                    {"id":"Column1", "type":"string", "size":"256"},
                    {"id":"Column2", "type":"string", "size":"256"}
                ]
            },
            "Rows":
            [
                {"Column0":"A", "Column1":"B"},
                {"Column0":"a", "Column1":"b", "Column2":"c"},
                {"Column0":"", "Column1":"", "Column2":""}
            ]
        }
    ]
}
```